

Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Mestrado Integrado em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Sistema de Gestão de um Clube Desportivo

David Silva
a100736

João Costa
ae9271

Sandro Coelho
a105672

Tomás Ramalhete
a98953

Junho, 2026

Data da Receção	
Responsável	
Avaliação	
Observações	

Sistema de Gestão de um Clube Desportivo

David Silva
a100736

João Costa
ae9271

Sandro Coelho
a105672

Tomás Ramalhete
a98953

Junho, 2026

Resumo

Este relatório apresenta o desenvolvimento do Sistema Integrado de Gestão do Codfish United, realizado no âmbito da unidade curricular de Laboratórios de Informática IV. O projeto teve como objetivo a digitalização dos principais processos operacionais de um clube desportivo, abrangendo a gestão de pessoas, modalidades, equipas, treinos, presenças, desempenhos, competições, eventos, quotas, pagamentos e relatórios de apoio à decisão.

A solução foi concebida segundo uma arquitetura cliente-servidor, com uma Single Page Application desenvolvida em React no cliente e uma API REST desenvolvida em Spring Boot no servidor, suportada por uma base de dados PostgreSQL. O sistema foi organizado em sete subsistemas funcionais, permitindo separar responsabilidades e estruturar o domínio de forma modular: Identity, Sportscore, Teams, Activities, Activity Tracking, Finance e Analytics.

A metodologia de desenvolvimento integrou técnicas de Prompt Engineering e o apoio de Modelos de Linguagem de Grande Escala (LLMs) em várias fases do projeto, incluindo a eliciação e refinamento de requisitos, a simulação de stakeholders, a revisão arquitetural, a geração assistida de código, a análise de qualidade e a validação funcional. Este processo resultou na consolidação de requisitos funcionais e não funcionais, posteriormente materializados numa aplicação web funcional.

A implementação final inclui autenticação e controlo de acessos por perfil, gestão temporal de cargos e vínculos, registo de atividades desportivas, acompanhamento de presenças e desempenhos, geração e controlo de quotas, simulação de pagamentos online através de um gateway mock, dashboards analíticos e deployment em ambiente cloud com frontend na Vercel, backend no Render e base de dados no Supabase. Conclui-se que a solução desenvolvida cumpre os objetivos principais definidos para o projeto, demonstrando a viabilidade de uma plataforma modular, escalável e adequada ao contexto académico e funcional de um clube desportivo.

Área de Aplicação: Engenharia de Software, Modelação de Sistemas e Engenharia de Requisitos.

Palavras-Chave: Gestão Desportiva, Aplicação Web, Arquitetura Cliente-Servidor, UML, LLM, Engenharia de Requisitos.

Índice

1. Capítulo 1 - Conceção e Engenharia de Requisitos Assistida por LLM	1
1.1. Contextualização da utilização dos LLMs	1
1.2. Definição do domínio do sistema	1
1.3. Identificação de stakeholders	2
1.4. Análise de contexto e restrições	2
1.5. Eliciação de requisitos	3
1.6. Geração de user stories	5
1.7. Simulação de entrevistas com stakeholders	6
1.7.1. Entrevista — José Treino (Treinador)	6
1.7.2. Entrevista — D. Maria Moeda (Gerente)	7
1.7.3. Entrevista — Tiago Ribeiro (Atleta)	7
1.8. Requisitos não funcionais	8
1.9. Refinamento e Justificação dos Requisitos	9
1.9.1. Explicitação do ator responsável	10
1.9.2. Adição de contexto e restrições técnicas	10
1.9.3. Consolidação e Unicidade	10
1.9.4. Diferenciação de permissões e visibilidade	10
1.10. Especificação do software	14
1.11. Casos de uso	15
1.12. Diagramas UML	17
1.12.1. Dicionário de Entidades	20
1.12.2. Mapeamento de Relacionamentos e Cardinalidades	20
2. Capítulo 2 - Arquitetura e Design de Software utilizando LLM	22
2.1. Arquitetura de Servidor (Backend - Spring Boot)	22
2.2. Arquitetura de Cliente (Frontend - React SPA)	23
2.3. Decisões Arquiteturais: Abstração de Gateways de Pagamento	23
2.4. Explorar LLM na Revisão e Coesão Arquitetural	24
2.5. Diagramas de Classes por Subsistema	24
2.6. Diagramas de Sequência	28
2.7. Diagramas de Componentes	32
2.8. Design de Interfaces e Prototipagem	33
2.8.1. Rastreabilidade: Protótipos e Requisitos Funcionais	34
2.8.2. Avaliação Heurística de Usabilidade (10 Princípios de Nielsen)	37
2.8.3. Exemplo Prático: Refinamento Iterativo (Subsistema Financeiro)	41
2.8.4. Nota sobre a Iteração e Esforço na Prototipagem	42
2.9. Especificação da API e as decisões arquiteturais tomadas	42
2.9.1. Convenções Gerais e Padrões de Contrato	42
2.9.2. Especificação Detalhada por Subsistema	42
2.9.3. Subsistema 1: Identity (Gestão de Pessoal)	42
2.9.4. Subsistema 2: Sportscore (Gestão de Modalidades)	43
2.9.5. Subsistema 3: Teams (Gestão de Equipas)	44
2.9.6. Subsistema 4: Activities (Planeamento)	45
2.9.7. Subsistema 5: Activity Tracking (Terreno)	45
2.9.8. Subsistema 6: Finance (Contabilidade)	46
2.9.9. Subsistema 7: Analytics (Dashboards)	47

2.10. Modelo Físico de Dados e Persistência	47
3. Desenvolvimento Incremental e Integração de LLMs	51
3.1. Estratégia de Gestão de Contexto (Prompt Engineering)	51
3.2. Quantificação do Impacto do LLM (Backend vs. Frontend)	51
3.2.1. Impacto no Servidor (Backend - Spring Boot)	52
3.2.2. Impacto no Cliente (Frontend - React)	52
3.3. Engenharia de Prompts no Frontend: Construção Iterativa (Figma -> React)	53
3.4. Reflexão Crítica sobre a Abordagem	54
3.5. Engenharia de Prompts no backend: Construção iterativa (UML, Documentação das APIs, etc.. -> SpringBoot)	54
3.5.1. Treinos SS4	54
3.5.2. Integração Externa e Simulação (Payment Gateway Mock)	56
3.6. Revisão de Código, Qualidade e Refactoring	57
3.6.1. Estrutura de Pacotes e Convenção de Módulos	58
3.6.2. SS1 (Identity): Segurança e Segregação de Privilégios	58
3.6.3. SS2 (Sportscore): Otimização de Consultas de Validação	59
3.6.4. SS3 (Teams): Integridade de Vínculos Desportivos Temporais	59
3.6.5. SS4 (Activities): Motor de Pesquisa Dinâmico para Calendários	61
3.6.6. SS5 (Tracking): Exclusividade Mútua Lógica (Regra XOR)	62
3.6.7. SS6 (Finance): Idempotência na Receção de Webhooks	63
3.6.8. SS7 (Analytics): Controlo de Acesso e Agregação de Indicadores Executivos	65
3.7. Estratégia de Deployment e Integração Contínua	67
3.7.1. Frontend (Vercel)	67
3.7.2. Backend (Render)	67
3.7.3. Base de Dados (Supabase)	68
3.7.4. Visão Geral do Pipeline e Fluxo de Dados	69
3.7.5. Resumo do Fluxo de Comunicação	69
3.8. Demonstração do Sistema Final	69
3.8.1. Fluxos por Perfil de Utilizador	70
3.8.2. Perfil Gerente	70
3.8.3. Perfil Colaborador	71
3.8.4. Perfil Treinador	73
3.8.5. Perfil Atleta	74
3.9. Vista por Subsistema	75
3.9.1. SS1 — Identidade e Autenticação	75
3.9.2. SS2 — Modalidades Desportivas	76
3.9.3. SS3 — Equipas e Vínculos	77
3.9.4. SS4 — Agendamento de Atividades	77
3.9.5. SS5 — Registo de Presenças	78
3.9.6. SS6 — Módulo Financeiro e Pagamentos	79
3.9.7. SS7 — Analytics e Relatórios Executivos	79
3.10. Síntese	80
4. Capítulo 4 - Verificação, Validação e Avaliação da Qualidade do Software	81
4.1. Estratégia de Validação Baseada na API (API-Driven Testing)	81
4.2. Validação Iterativa e Testes de Aceitação via Swagger UI	81
4.3. Validação da Camada de Apresentação	82
4.4. Testes de Concorrência e Validação de Optimistic Locking	82
4.5. Matriz de Rastreabilidade (Requisitos vs. Endpoints)	83
4.6. Reflexão Crítica: O LLM como Ferramenta de Teste e QA	83
4.7. Avaliação de Qualidade e Requisitos Não Funcionais (ISO/IEC 25010)	83
5. Capítulo 5 - Conclusões e Trabalho Futuro	85
5.1. Reflexão Crítica e Desafios	85
5.2. Trabalho Futuro	85

6. Referências Bibliográficas	87
--	-----------

Lista de Figuras

Figura 1	Modelo de Domínio do Codfish United	18
Figura 2	Diagrama de Use Cases do Codfish United	19
Figura 3	Diagrama de classes do SS1 - Identity	25
Figura 4	Diagrama de classes do SS2 - Sportscore	25
Figura 5	Diagrama de classes do SS3 - Teams	26
Figura 6	Diagrama de classes do SS4 - Activities	27
Figura 7	Diagrama de classes do SS5 - Activity Tracking	27
Figura 8	Diagrama de classes do SS6 - Finance	28
Figura 9	Diagrama de Sequência: Submissão Lote de Presenças (SS5)	29
Figura 10	Diagrama de Sequência: Pagamento Online com Gateway Mock e Simulação de Callback (SS6)	30
Figura 11	Diagrama de Sequência: Integração Real com Gateway de Pagamento Externo (Visão Arquitetural)	31
Figura 12	Diagrama de Sequência: Processamento de Webhook de Pagamento (SS6)	32
Figura 13	Diagrama de Componentes Global da Arquitetura Codfish United	33
Figura 14	Portal Público do Codfish United: Foco na conversão e exposição de modalidades.	34
Figura 15	Ecrã de Autenticação. Comprova o requisito R02 e a fundação do sistema de permissões. .	35
Figura 16	Vista Administrativa de Equipas. Satisfaz os requisitos R07, R11 e R18.	35
Figura 17	Painel de Treinos e Presenças. Comprova o cumprimento dos requisitos R30 e R33.	36
Figura 18	Portal Financeiro do Atleta. Implementa os requisitos R38 e R41.	36
Figura 19	Dashboard Executivo da Gerência. Materializa os requisitos R26 e R46.	37
Figura 20	Notificação de sistema.	37
Figura 21	Terminologia adaptada ao negócio.	38
Figura 22	Botões de ação e cancelamento contíguos.	38
Figura 23	Consistência visual nos cartões de domínio.	39
Figura 24	Omissão de menus (RBAC) para prevenção de erros.	39
Figura 25	Instruções embebidas em campos de texto.	39
Figura 26	Acelerador de interface para marcação em lote.	40
Figura 27	Ausência de ruído visual na vista de Gestão.	40
Figura 28	Mensagem de erro semântica e localizada.	40
Figura 29	Assistência contextual embutida no formulário.	41
Figura 30	Distribuição estimada do esforço de código gerado por LLM na API (Spring Boot).	52
Figura 31	Distribuição estimada do esforço de código gerado por LLM na UI (React).	52
Figura 32	Pipeline de e fluxo de dados entre camadas.	69
Figura 33	Dashboard executivo do Gestor com indicadores KPI agregados (SS7).	70
Figura 34	Gestão de pessoas e papéis.	70
Figura 35	Relatório desportivo do clube.	71
Figura 36	Relatório financeiro do clube.	71
Figura 37	Vista inicial do Colaborador com acesso às funcionalidades administrativas.	72

Figura 38	Gestão de pessoas.	72
Figura 39	Administração de quotas e pagamentos.	72
Figura 40	Calendário de treinos.	73
Figura 41	Gestão de plantéis	73
Figura 42	registo de presenças com regra XOR.	74
Figura 43	Vista pessoal do Atleta com resumo de atividades e estado de pagamentos.	74
Figura 44	Vista para inscrições de competição.	75
Figura 45	Vista pessoal do Atleta com resumo de atividades e estado de pagamentos.	75
Figura 46	Ecrã de autenticação em desktop (identico em mobile).	76
Figura 47	Listagem e gestão do catálogo de modalidades desportivas (SS2).	77
Figura 48	Gestão de plantéis com vínculos temporais ativos e históricos (SS3).	77
Figura 49	Calendário de atividades em desktop (esquerda) e em dispositivo móvel (direita) (SS4).	78
Figura 50	Interface de registo de presenças com validação XOR ativa (SS5).	78
Figura 51	Interface com informação de pagamento	79
Figura 52	Resumos financeiros	79
Figura 53	Resumos financeiros em dispositivo móvel	80

Lista de Tabelas

Tabela 1 Stakeholders do Sistema Codfish United	2
Tabela 2 Lista Consolidada de Requisitos Funcionais (Eliciação Inicial)	4
Tabela 3 Requisitos Não Funcionais (Métricas IEEE 830)	8
Tabela 4 Lista Refinada de Requisitos Funcionais	11

1. Capítulo 1 - Conceção e Engenharia de Requisitos Assistida por LLM

1.1. Contextualização da utilização dos LLMs

A adoção de Grandes Modelos de Linguagem (LLMs) neste projeto não se baseou na geração passiva de artefactos, mas sim numa abordagem metodológica de *Prompt Engineering* e co-criação. Numa fase inicial, o modelo foi alimentado com o contexto do domínio e as restrições arquiteturais pretendidas. Ao longo da fase de eliciação, foram aplicadas técnicas avançadas de interação, destacando-se o *Role-playing* — instruindo o LLM a atuar como diferentes *stakeholders* para simular entrevistas realistas — e o *Iterative Refinement* (refinamento iterativo).

Esta abordagem colaborativa exigiu uma intervenção humana constante: os artefactos gerados pelo modelo raramente foram aceites na sua forma inicial, sendo continuamente criticados, validados face às regras de negócio e reajustados através de *prompts* corretivos. Garantiu-se assim que o resultado final reflete um rigoroso trabalho de Engenharia de Requisitos e avaliação crítica, e não apenas um *output* estocástico do modelo. Em cada um dos pontos seguintes serão apresentados exemplos dos *prompts* fundamentais que guiaram esta construção iterativa.

1.2. Definição do domínio do sistema

O sistema destina-se à gestão integrada do Codfish United, cobrindo inicialmente três grandes áreas funcionais de negócio: gestão de pessoas, gestão desportiva e gestão financeira. O domínio abrange o ciclo de vida completo dos associados — desde a admissão e atribuição de papéis, até ao planeamento de plantéis, treinos e competições — bem como a gestão dos espaços físicos do clube e a automatização da cobrança de quotas e pagamentos. A plataforma visa substituir processos manuais atualmente dispersos em folhas de cálculo, centralizando toda a operação.

Numa fase preliminar, e para orientar o levantamento de requisitos, o domínio foi logicamente particionado nestas três áreas principais de atuação. Esta visão macro servirá de ponto de partida para as sessões de eliciação, permitindo que o sistema suporte, de forma organizada, múltiplas modalidades, equipas e calendários competitivos independentes.

Prompt utilizado:

Prompt submetido: Pretendemos definir o domínio do sistema de forma clara e estruturada. O sistema deverá permitir:

- Registo de atletas, treinadores e staff. Gestão de equipas e modalidades.
- Controlo de treinos e presenças. Gestão de competições e resultados.
- Pagamentos de quotas.

Com base nisto:

1. Define o domínio do sistema.
2. Identifica as principais áreas funcionais.
3. Sugere uma possível decomposição em subsistemas.

1.3. Identificação de stakeholders

O ecossistema do clube é operado por quatro perfis de utilizadores distintos. Para facilitar a modelação dos requisitos e as sessões de eliciação, cada perfil foi materializado através de uma **persona** representativa:

Tabela 1: Stakeholders do Sistema Codfish United

Role	Persona	Responsabilidades
Gerente	D. Maria Moeda	Gestão financeira, controlo de quotas em dívida, faturação mensal e acompanhamento da lista de morosos.
Colaborador	Carlos Pinto	Backoffice diário: admissão de pessoas, criação de modalidades e equipas, registo de pagamentos presenciais.
Treinador	José Treino	Gestão de plantéis, marcação de treinos, registo de presenças e submissão de resultados via dispositivo móvel.
Atleta	Tiago Ribeiro	Auto-serviço: consulta de resultados, receção de notificações de convocatórias e faltas, e pagamento online de quotas.

Cada linha da tabela descreve o papel geral e as preocupações primárias de cada ator, servindo de base sólida para a posterior construção das *User Stories*.

Prompt utilizado:

Refina a lista de stakeholders considerando um clube desportivo de média dimensão. Queremos apenas os perfis essenciais para o funcionamento do sistema. Para cada stakeholder:

- Dá um nome ao papel (cria uma persona)
- Explica as suas responsabilidades no sistema

1.4. Análise de contexto e restrições

O desenvolvimento do sistema é norteado por um conjunto de restrições operacionais e técnicas, identificadas logo nas fases preliminares do projeto, que garantem a viabilidade, segurança e escalabilidade da solução:

- **Segurança e Controlo de Acessos:** Exige-se uma segregação rigorosa de perfis. Apenas utilizadores autorizados (como a Gerência) podem aceder a dados financeiros sensíveis. O sistema deve também assegurar a proteção de credenciais de acesso através de *hashing* criptográfico.
- **Conformidade RGPD:** Os dados demográficos e pessoais dos associados devem ser estritamente protegidos. É obrigatório manter um histórico auditável das funções e privilégios de cada utilizador (com data de início, fim e respetiva justificação das alterações).
- **Integração e Comunicação Externa:** A automatização da tesouraria obriga à comunicação direta com *gateways* de pagamento externos (e.g., SIBS/MBWay). O sistema estará restringido pela necessidade de receber e processar notificações assíncronas (*webhooks*) de forma segura, exigindo validação de assinaturas.
- **Usabilidade no Terreno:** Dado o contexto de utilização da equipa técnica, impõe-se a obrigatoriedade de uma interface ágil e responsiva (orientada a dispositivos móveis) para o registo imediato de presenças e avaliações durante as sessões de treino.
- **Desacoplamento Arquitetural:** Para suportar a evolução contínua do clube, a arquitetura deverá impor fronteiras claras entre as diferentes áreas de atuação (pessoal, desporto, financeiro), garantindo a separação de responsabilidades para futura manutenibilidade.

Prompt utilizado:

Considerando que o sistema lida com dados pessoais, pagamentos e atividade móvel no campo de treino, detalha as principais restrições legais, arquiteturais e de boas práticas de segurança para a plataforma.

1.5. Eliciação de requisitos

A fase de eliciação foi conduzida através de sessões de análise e discussão assistidas por LLM, nas quais foram explorados cenários de utilização do sistema, refinadas descrições funcionais e identificadas dependências entre diferentes componentes do domínio. Este processo resultou num inventário consolidado de 34 requisitos funcionais, organizados pelas três grandes áreas de atuação identificadas na definição do domínio.

A **Gestão de Pessoal (R01–R07)** cobre o ciclo de vida dos utilizadores, desde o registo com dados demográficos e credenciais, até aos mecanismos de autenticação e gestão de papéis (**roles**). Inclui ainda a necessidade crítica de auditar alterações de privilégios para conformidade com o RGPD.

A **Gestão Desportiva (R08–R21, R33–R34)** constitui o núcleo operacional, suportando a definição de modalidades, equipas e épocas. Inclui o registo diário de treinos (com presenças e avaliações) e a gestão de competições oficiais. Foi também identificada a necessidade de gerir as infraestruturas do campus (campos, pavilhões) para evitar conflitos de agendamento.

A **Gestão Financeira e Administrativa (R22–R32)** foca-se na sustentabilidade do clube. Automatiza a geração de quotas (mensalidades e inscrições) e suporta tanto o pagamento tradicional em secretaria como a integração com **gateways** de pagamento digital. Esta área garante ainda a visibilidade sobre a saúde financeira do clube através de relatórios de faturação e listas de morosos.

A lista consolidada de requisitos funcionais resultante desta fase encontra-se apresentada na tabela seguinte:

Tabela 2: Lista Consolidada de Requisitos Funcionais (Eliciação Inicial)

Código	Descrição do Requisito	Data	Levantado por
R01	Registar pessoas com dados demográficos e credenciais de acesso	10/02/2026	Tomás Ramalhete
R02	Autenticar utilizadores via login	10/02/2026	João Costa
R03	Registar saída de uma pessoa do sistema (desativação)	10/02/2026	Tomás Ramalhete
R04	Atualizar o papel (role) de uma pessoa	10/02/2026	João Costa
R05	Alterar a password de uma pessoa	12/02/2026	Tomás Ramalhete
R06	Consultar o histórico de roles de uma pessoa e respetivas justificações	12/02/2026	João Costa
R07	Consultar dados de pessoas do clube	12/02/2026	Tomás Ramalhete
R08	Criar modalidades com caracterização detalhada (individual ou coletiva)	17/02/2026	João Costa
R09	Definir preços por modalidade e limites de faixas etárias	17/02/2026	Tomás Ramalhete
R10	Associar tipos de estatísticas a modalidades	17/02/2026	João Costa
R11	Criar equipas ativas associadas a modalidades e épocas	17/02/2026	Tomás Ramalhete
R12	Inscriver atletas e treinadores em equipas	17/02/2026	João Costa
R13	Registar treinos com indicação de local e duração	19/02/2026	Tomás Ramalhete
R14	Registar presenças de atletas em treinos	19/02/2026	João Costa
R15	Registar avaliações de desempenho de atletas em treinos	19/02/2026	Tomás Ramalhete
R16	Registar competições oficiais	19/02/2026	João Costa
R17	Inscriver equipas em competições	19/02/2026	Tomás Ramalhete
R18	Registar resultados e medalhas (Ouro, Prata, Bronze) em competições	19/02/2026	João Costa
R19	Criar eventos desportivos (ex.: jogos ou provas) dentro de competições	24/02/2026	Tomás Ramalhete
R20	Registar resultados de atletas em eventos	24/02/2026	João Costa
R21	Registar desempenho de atletas em eventos	24/02/2026	Tomás Ramalhete

Código	Descrição do Requisito	Data	Levantado por
R22	Gerar quotas automáticas de inscrição de atletas	24/02/2026	João Costa
R23	Gerar quotas automáticas de mensalidades	24/02/2026	Tomás Ramalhete
R24	Gerar quotas de inscrição de equipas em provas	26/02/2026	João Costa
R25	Consultar estado de quotas (Dívida, Pendente, Pago)	26/02/2026	Tomás Ramalhete
R26	Listar todas as quotas do sistema (visão Gerente)	26/02/2026	João Costa
R27	Processar pagamentos manuais (dinheiro) em secretaria	26/02/2026	Tomás Ramalhete
R28	Iniciar pagamento online via gateway externo (MBWay ou Multibanco)	26/02/2026	João Costa
R29	Consultar estado de um pagamento online	28/02/2026	Tomás Ramalhete
R30	Receber e processar webhooks de confirmação de pagamento	28/02/2026	João Costa
R31	Enviar notificações automáticas relacionadas com atividades desportivas	28/02/2026	Tomás Ramalhete
R32	Gerar relatórios administrativos e financeiros	28/02/2026	João Costa
R33	Registar infraestruturas desportivas do campus	28/02/2026	Tomás Ramalhete
R34	Associar treinos e eventos desportivos a uma infraestrutura	28/02/2026	João Costa

Prompt utilizado:

Pretendemos agora levantar os requisitos do sistema. Já identificámos algumas funcionalidades base: registo de pessoas, login, gestão de equipas/modalidades, treinos/presenças e quotas. Ajuda-nos a expandir esta lista para um conjunto mais completo de requisitos funcionais, tornando-os detalhados e garantindo a consistência entre as áreas de pessoal, desporto e finanças.

1.6. Geração de user stories

Os requisitos foram traduzidos para a perspetiva de cada ator:

- **US01-A (R01-R02, R05):** Como Colaborador/Utilizador, quero registar pessoas e autenticar acessos para garantir a entrada segura no sistema.
- **US01-B (R03-R04, R06):** Como Colaborador, quero gerir os dados demográficos e o estado (ativo/inativo) das pessoas para manter a base de dados atualizada.
- **US01-C (R07-R09):** Como Colaborador, quero atribuir e revogar papéis (roles), garantindo o registo da justificação, para manter os acessos controlados e auditáveis (RGPD).
- **US02 (R08-R12):** Como Colaborador, quero criar modalidades, definir preços e constituir equipas.

- **US03 (R13–R15):** Como Treinador, quero registrar treinos, presenças e avaliações do meu plantel pelo telemóvel, sem necessidade de computador.
- **US04 (R16–R21):** Como Treinador, quero registrar competições, eventos e resultados individuais para manter o histórico desportivo do atleta atualizado.
- **US05 (R31):** Como Sistema, devo enviar notificações automáticas relacionadas com atividades desportivas, como inscrições em equipas, faltas a treinos ou participação em eventos.
- **US06 (R22–R24):** Como Sistema, devo gerar automaticamente as quotas de inscrição, mensalidades e inscrições em competições para alimentar a faturação da tesouraria.
- **US07 (R25, R28):** Como Atleta, quero consultar as minhas quotas em dívida e liquidá-las através de pagamento online (ex.: MBWay ou Multibanco).
- **US08 (R26, R27):** Como Colaborador, quero listar todas as quotas do sistema e registrar pagamentos efetuados presencialmente na secretaria.
- **US09 (R29, R30):** Como Sistema, devo comunicar com o gateway de pagamentos e processar confirmações de pagamento recebidas através de *webhooks*.
- **US10 (R33, R34):** Como Colaborador, quero registrar infraestruturas desportivas do campus e associá-las a treinos e eventos desportivos.
- **US11 (R32):** Como Gerente, quero gerar relatórios administrativos e financeiros sobre quotas, pagamentos e atividade do clube para apoiar a gestão.

Prompt utilizado:

Prendemos agora criar user stories que nos permitam validar e possivelmente identificar novos requisitos, isto é, o que cada utilizador pretende obter do sistema

Transforma os requisitos em user stories no formato:

“Como [tipo de utilizador], quero [funcionalidade], para [objetivo]”

Prompt utilizado:

As user stories geradas estão demasiado fragmentadas. Refaz considerando:

- Cada user story deve representar uma funcionalidade com valor real para o utilizador
- Agrupa requisitos relacionados
- Evita criar uma user story por requisito

1.7. Simulação de entrevistas com stakeholders

Para a extração das necessidades reais de cada ator, utilizou-se a LLM para simular sessões de entrevista em formato *roleplay*. Apresentam-se três excertos representativos.

1.7.1. Entrevista — José Treino (Treinador)

Tomás Ramalhete	Como gere atualmente os treinos e a evolução dos atletas?
-----------------	---

José Treino	Tenho uma caderneta física onde marco as presenças e tiro notas. No fim da época é quase impossível extrair qualquer estatística. Para as competições mando um email para a secretaria e fico à espera — nunca sei se ficou registado.
Tomás Ramalhete	O que precisaria de registar num treino, idealmente?
José Treino	Quem esteve presente e uma avaliação por atleta — um valor numérico ou uma nota. E quero poder fazer isso pelo telemóvel durante o treino, não depois no computador.
Tomás Ramalhete	E nas competições, o que é crítico?
José Treino	As classificações finais — ouro, prata, bronze — e os resultados individuais por jogo. E preciso de poder inscrever a minha equipa numa competição sem depender da secretaria.
Requisitos elicitados	R13–R15: registo de treinos, presenças e avaliações de atletas. R16–R18: registo de competições e inscrições de equipas. R19–R21: eventos desportivos com resultados individuais e desempenho.

1.7.2. Entrevista — D. Maria Moeda (Gerente)

João Costa	Qual é a maior dificuldade na gestão financeira atual?
D. Maria Moeda	Não sei quem deve sem ir pesquisar manualmente no Excel. E as mensalidades tenho de as criar uma a uma todo o mês. Precisava que isso fosse automático e que me avisasse os morosos.
João Costa	E quanto aos pagamentos online?
D. Maria Moeda	Uma transferência bancária pode demorar dias a aparecer. Precisava de saber em tempo real se uma quota foi paga, sem ter de verificar manualmente o banco.
João Costa	Há preocupações com quem pode ver a informação financeira?
D. Maria Moeda	Sim, os dados financeiros são confidenciais. Só a direção deve aceder aos relatórios de faturação — os treinadores não devem ver o que os atletas pagam.
Requisitos elicitados	R22–R24: geração automática de quotas de inscrição, mensalidades e participação em competições. R25–R27: consulta e gestão global das quotas pela gerência. R28–R30: pagamentos online com confirmação assíncrona via <i>webhook</i> . R32: geração de relatórios administrativos e financeiros.

1.7.3. Entrevista — Tiago Ribeiro (Atleta)

João Costa	Qual é a maior frustração com a gestão atual do clube?
-------------------	--

Tiago Ribeiro	Nunca sei se as minhas quotas estão em dia sem ligar para a secretaria, que nem sempre atende. E às vezes fico a saber que havia um treino ou uma convocatória por um colega, porque o aviso chegou tarde ou nem chegou.
João Costa	Como preferiria tratar das suas quotas?
Tiago Ribeiro	Queria poder pagar com MBWay ou referência MB diretamente, sem ter de ir à secretaria nem andar com dinheiro. E saber logo se o pagamento foi confirmado, sem ter de esperar.
João Costa	E quanto às notificações de treinos e convocatórias?
Tiago Ribeiro	Por email seria suficiente — um aviso quando fico inscrito num jogo, e outro se acumulo faltas para poder justificar a tempo.
Requisitos elicitados	R01–R02: autenticação e acesso ao portal do sistema. R25: consulta do estado das quotas. R28–R30: pagamento online com confirmação automática. R31: envio de notificações relacionadas com atividades desportivas.

Prompt utilizado:

Queremos simular uma entrevista para validar e possivelmente levantar novos requisitos

Assume dois papéis:

1. Entrevistador (engenheiro de software)
2. Stakeholder (ex: treinador, gerente ou atleta)

A entrevista deve:

- Ser realista e progressiva
- Incluir perguntas relevantes e não triviais
- Explorar problemas atuais e necessidades
- Conduzir à identificação de requisitos

No final, apresenta uma lista de requisitos funcionais identificados. Começa com o papel de treinador.

1.8. Requisitos não funcionais

A nível de requisitos não funcionais, que ditam as restrições arquiteturais e de qualidade do software, identificaram-se os seguintes indicadores mensuráveis:

Tabela 3: Requisitos Não Funcionais (Métricas IEEE 830)

Código	Categoria	Descrição Mensurável (IEEE 830)
RNF01	Desempenho	As operações frequentes do sistema, como consulta de equipas, treinos, quotas e presenças, devem apresentar

Código	Categoria	Descrição Mensurável (IEEE 830)
		tempos de resposta adequados para utilização interativa em contexto administrativo e desportivo.
RNF02	Usabilidade / Responsividade	As funcionalidades utilizadas em contexto de treino, nomeadamente consulta de plantel e registo de presenças ou desempenhos, devem ser acessíveis através de uma interface responsiva e utilizável em dispositivos móveis.
RNF03	Segurança	O sistema deve proteger o acesso a dados pessoais, financeiros e desportivos através de autenticação, controlo de permissões por perfil de utilizador e armazenamento seguro de credenciais.
RNF04	Integração	O sistema deve ser concebido de forma a permitir integração futura com serviços externos de pagamento, suportando confirmações assíncronas do estado de pagamentos.
RNF05	Manutenibilidade	A arquitetura deve separar claramente as áreas funcionais de gestão de pessoas, gestão desportiva, gestão financeira e análise de dados, facilitando a evolução e manutenção independente de cada componente.

Prompt utilizado:

Considerando que a nossa aplicação terá um frontend web, operações de campo (mobile) e integrações com pagamentos externos: Identifica os principais requisitos não funcionais para o sistema. Organiza-os por categorias (ex: desempenho, segurança, manutenibilidade, integração e RGPD) cruzando com as necessidades expressas pelos stakeholders.

1.9. Refinamento e Justificação dos Requisitos

Após a validação inicial dos requisitos funcionais através das entrevistas com os *stakeholders*, tornou-se evidente que a divisão preliminar em apenas três macro-áreas (Gestão de Pessoas, Desportiva e Financeira) resultava num elevado acoplamento lógico. Para preparar o sistema para uma arquitetura de *software* robusta e respeitar o Princípio da Responsabilidade Única (*Single Responsibility Principle*), foi necessário particionar o domínio.

A macro-área Desportiva foi logicamente decomposta, separando a estrutura estática (Modalidades e Equipas) do planeamento (Competições e Treinos) e da execução no terreno (*Tracking* de presenças e avaliações). Simultaneamente, a gestão de permissões foi isolada num módulo de Identidade para garantir a conformidade com o RGPD. Esta reestruturação originou a divisão final do sistema em sete (7) subsistemas coesos.

O refinamento dos requisitos, orientado por esta nova visão modular, seguiu quatro eixos principais de detalhe:

1.9.1. Explicitação do ator responsável

Cada requisito passou a identificar claramente qual (ou quais) o perfil de utilizador que desencadeia a ação (Colaborador, Gerente, Treinador ou Atleta). Por exemplo, o requisito **R03** — referente à gestão do ciclo de vida do pessoal no **SS1 (Identity)** — especifica que apenas o **Colaborador** e o **Gerente** podem executar a inativação lógica do registo, garantindo a preservação do histórico e a conformidade com o RGPD, em vez de uma eliminação física de dados.

1.9.2. Adição de contexto e restrições técnicas

Para suportar uma implementação robusta, foram introduzidas restrições de lógica de negócio e segurança que omitiam condições de execução críticas:

- **Regras de Exclusividade (XOR):** Os requisitos **R31** e **R35** no subsistema de **Tracking** passaram a exigir que qualquer presença ou avaliação de desempenho pertença inequivocamente a um Treino ou a um Evento, impedindo a sobreposição de dados estatísticos.
- **Idempotência Financeira:** O requisito **R44** no **SS6 (Finance)** foi introduzido para garantir que o processamento de *webhooks* externos seja idempotente, prevenindo duplicações de pagamentos no sistema em caso de falhas de rede ou reenvios do *gateway*.
- **Consistência Temporal:** Requisitos como **R22** e **R24** passaram a validar rigorosamente que datas de fim (de vínculos ou competições) nunca precedam as respetivas datas de início.

Nota de Refinamento Arquitetural: Durante a transição para a listagem final (Tabela 4), reconheceu-se que certas regras geradas pela IA (como a restrição não-negativa em presenças ou a regra XOR de Treinos/Eventos repetida em Presenças e Desempenhos) atuam primariamente como Restrições de Domínio (*Business Rules*) e não como novos Casos de Uso funcionais. Optou-se por mantê-las na especificação funcional de forma atomizada para facilitar a rastreabilidade e a posterior conversão em Testes Unitários de validação nas camadas *Request DTO* e *Service*.

1.9.3. Consolidação e Unicidade

Alguns requisitos foram expandidos para garantir a integridade referencial entre os módulos:

- O requisito **R20** no **SS3 (Teams)** passou a assegurar a unicidade de vínculos, impedindo que a mesma pessoa seja associada à mesma equipa, com o mesmo papel, na mesma data de início.
- A lógica de notificações, anteriormente dispersa, foi consolidada no requisito **R10**, integrando-a como um serviço central do subsistema de **Identity**.

1.9.4. Diferenciação de permissões e visibilidade

A separação de responsabilidades foi refinada para distinguir o acesso global do acesso privado. Enquanto o **Gerente** possui visibilidade sobre os KPIs globais no **SS7 (Analytics)** através do requisito **R46**, o acesso do **Atleta** (requisitos **R49** e **R50**) foi estritamente limitado à consulta do seu próprio perfil e histórico estatístico, assegurando a privacidade dos dados desportivos e financeiros.

Nota sobre Controlo de Acessos: O sistema implementa uma hierarquia de privilégios. Salvo indicação em contrário, assume-se que o perfil de Gerente herda todas as permissões de leitura e escrita atribuídas aos perfis de Colaborador, Treinador e Atleta, possuindo acesso total e irrestrito às funcionalidades opera-

cionais do sistema, tendo como exceção a visibilidade e acesso das notificações, onde apenas o Atleta tem acesso.

O resultado deste processo foi uma lista consolidada de **50 requisitos funcionais** com elevado grau de completude, testabilidade e rastreabilidade, totalmente organizada e com a arquitetura modular implementada. A lista refinada encontra-se apresentada na tabela seguinte, organizada por subsistema.

Tabela 4: Lista Refinada de Requisitos Funcionais

Código	Descrição do Requisito
SS1 - Identity (Gestão de Pessoal)	
R01	Um Colaborador pode registar uma nova pessoa no sistema, introduzindo dados demográficos básicos.
R02	O sistema deve autenticar os utilizadores (via username e password), validando credenciais.
R03	Um Colaborador pode ativar ou desativar o registo de uma pessoa, preservando o seu histórico.
R04	Um Colaborador pode atualizar os dados pessoais de qualquer utilizador.
R05	Qualquer utilizador autenticado pode alterar a sua própria palavra-passe.
R06	Um Colaborador pode forçar a redefinição da palavra-passe de outro utilizador.
R07	Um Colaborador pode atribuir um papel (Role) a um utilizador, definindo datas de início.
R08	O sistema deve garantir que um utilizador apenas possui, no máximo, um único “Role Principal” ativo num dado momento.
R09	Um Colaborador pode terminar o papel de um utilizador, registando obrigatoriamente a justificação.
R10	O utilizador pode consultar as suas notificações e marcá-las individualmente como lidas.
SS2 - Sportscore (Gestão de Modalidades)	
R11	Um Colaborador pode criar e atualizar modalidades, definindo se são treinadas e o limite máximo de presenças.
R12	O sistema deve validar que o limite máximo de presenças semanais numa modalidade, quando definido, é estritamente não negativo.
R13	Um Colaborador pode registar e atualizar os Complexos desportivos (nome, morada e telefone).

R14	Um Colaborador pode definir Tipos de Estatísticas (ex: golos, tempo), especificando a sua unidade e se são obrigatórios.
R15	Um Colaborador pode associar múltiplos Tipos de Estatística a uma dada Modalidade.
R16	Um Colaborador pode configurar a Tabela de Preços por modalidade, definindo a taxa de inscrição e mensalidade para faixas etárias específicas.
R17	O sistema deve impedir a criação de preçários ilógicos, garantindo que a Idade Mínima <= Idade Máxima, e que os valores não são negativos.
SS3 - Teams (Gestão de Equipas)	
R18	Um Colaborador pode criar uma Equipa, definindo o seu tipo, época desportiva e associando-a a uma Modalidade.
R19	Um Colaborador pode adicionar um membro a uma equipa, definindo a sua relação (Atleta/Treinador) e a data de início de vínculo.
R20	O sistema deve impedir a duplicação de vínculos, garantindo a unicidade do conjunto (Equipa, Pessoa, Relação, Data de Início).
R21	Um Colaborador pode terminar o vínculo de um membro a uma equipa, registando a data de fim.
R22	O sistema deve garantir a consistência temporal dos plantéis, rejeitando datas de fim de vínculo anteriores às datas de início.
SS4 - Activities (Competições, Eventos e Treinos)	
R23	Um Treinador pode criar uma Competição com data de início e fim, associada a uma modalidade e com taxa de inscrição não negativa.
R24	O sistema deve validar a coerência do calendário competitivo, impedindo que a data de fim de uma competição preceda a sua data de início.
R25	Um Treinador pode inscrever uma Equipa numa Competição, assegurando o sistema a unicidade da inscrição.
R26	Um Treinador pode registar o resultado final e classificativo (notas/pontos) da sua Equipa numa Competição.
R27	Um Treinador pode calendarizar um Evento específico, definindo a sua duração, data e o Complexo onde ocorre.
R28	Um Treinador pode registar o resultado numérico e descritivo de uma Equipa num Evento.
R29	Um Treinador pode agendar uma sessão de Treino para a sua equipa, especificando data, duração estritamente positiva, e Complexo.

SS5 - Activity Tracking (Presenças e Desempenho)	
R30	Um Treinador pode registrar a Presença de um Atleta num Treino agendado ou num Evento oficial.
R31	O sistema deve garantir através de exclusividade mútua lógica (XOR) que um registo de Presença remete inequivocamente para um Treino OU para um Evento, nunca para ambos.
R32	O sistema deve permitir o registo de presenças associadas a contexto de “Treino Livre”, fora do planeamento regular da equipa.
R33	Um Treinador pode registrar métricas de Desempenho de um Atleta durante um Treino, com base nos Tipos de Estatística da modalidade.
R34	Um Treinador pode registrar métricas de Desempenho de um Atleta durante um Evento.
R35	O sistema deve aplicar a mesma exclusividade mútua lógica (XOR) aos registos de Desempenho, isolando dados de prestação em Treino dos dados de Evento.
SS6 - Finance (Contabilidade)	
R36	O sistema deve gerir Quotas (inscrição, mensalidade, competição), definindo a data de vencimento e identificando se são recorrentes.
R37	O sistema deve garantir que não existem Quotas criadas com valores base (taxas/mensalidades) negativos.
R38	O sistema deve permitir consultar se uma Quota está vencida com base na data atual e no seu ciclo.
R39	Um Colaborador pode confirmar manualmente o pagamento de uma quota (ex: dinheiro presencial), registando o valor e a data de confirmação.
R40	O sistema deve suportar a geração de referências Multibanco, estipulando a entidade, referência e data limite.
R41	Um Atleta pode iniciar o pagamento online de uma quota através de um Gateway, transitando para um estado transitório de “Pendente”.
R42	O sistema deve criar um registo de rastreabilidade sempre que invoca a API de pagamentos externa, guardando o sucesso ou detalhe do erro técnico.
R43	O sistema deve fornecer um endpoint isolado e protegido para a receção e validação de webhooks (notificações) enviados pelo Gateway de pagamento.
R44	O sistema deve processar os webhooks garantindo a idempotência da transação (via chave única de notificação do gateway), impedindo duplicações.

R45	O sistema ou utilizador pode cancelar pagamentos que ainda se encontrem num estado que permita cancelamento.
SS7 - Analytics (Dashboards e Relatórios)	
R46	O sistema deve dispor de uma Facade de leitura que calcule e agregue KPIs para o Dashboard da Gerência.
R47	O sistema deve agregar dados financeiros transversais para suportar o Relatório Financeiro Global do clube.
R48	O sistema deve agregar métricas de presenças e competições para suportar o Relatório Desportivo do clube.
R49	O sistema deve compilar um portefólio read-only com os dados demográficos e histórico do utilizador autenticado para consulta no seu Perfil.
R50	O sistema deve consolidar o histórico estatístico e as avaliações de Desempenho de um Atleta para visualização na sua página de perfil.

Prompt utilizado:

Tendo em conta as falhas e restrições identificadas nas entrevistas com os stakeholders (como a necessidade de exclusividade mútua entre atividades e a exigência de idempotência nos webhooks de pagamento), refina a nossa lista inicial de requisitos funcionais. Diretrizes rigorosas para o refinamento:

-Reorganiza e agrupa os requisitos por subsistemas. -Explicita sempre o ator que desencadeia a ação. -Transforma restrições de negócio implícitas em requisitos explícitos (ex: regras de validação temporal, garantia de unicidade de vínculos e impedimento de valores negativos). -Aumenta a granularidade para cobrir casos de exceção e fluxos técnicos.

Gera a lista consolidada final.

1.10. Especificação do software

A especificação segue a norma IEEE 830/29148 e está organizada de forma modular, refletindo o refinamento efetuado na fase de requisitos. O sistema é decomposto em sete (7) subsistemas independentes, cada um responsável por uma área de domínio específica.

Esta organização garante que a lógica interna, as entidades de domínio e os repositórios de persistência de cada módulo permaneçam isolados. A comunicação inter-subsistemas é efetuada estritamente através das interfaces de aplicação, recorrendo ao uso de objetos de transferência de dados (*DTOs*) para assegurar o baixo acoplamento. Esta estrutura modular permite que o sistema mantenha a coesão necessária para uma gestão desportiva e financeira rigorosa, facilitando a testabilidade e a manutenção evolutiva do software.

Prompt utilizado:

Com base na lista final de 50 requisitos e na necessidade de isolamento entre as áreas de gestão desportiva, pessoal e financeira:

1. Sugere uma organização de especificação seguindo a norma IEEE 830/29148.
2. Define como deve ser feita a comunicação entre os diferentes módulos funcionais (do SS1 ao SS7).
3. Propõe o uso de padrões de desenho (como Facade e DTOs) que garantam o baixo acoplamento e a robustez técnica necessária.

1.11. Casos de uso

Os casos de uso identificados cobrem todas as interações entre os atores externos e o sistema, derivando diretamente dos requisitos funcionais levantados. O diagrama de casos de uso encontra-se na secção de Diagramas UML. Apresentam-se de seguida quatro casos de uso representativos, cobrindo os diferentes atores e fluxos do sistema.

UC-P01: Autenticar no Sistema

Campo	Descrição
Identificador	UC-P01
Ator Principal	Colaborador / Gerente / Treinador / Atleta
Descrição	Qualquer utilizador registado autentica-se no sistema através de username e password, estabelecendo uma sessão com as respetivas permissões.
Pré-condições	O utilizador está registado no sistema.
Pós-condições	A sessão é estabelecida com as permissões do utilizador.
Fluxo Normal	1. O utilizador acede à página de login. 2. Introduce username e password. 3. O sistema valida as credenciais. 4. A sessão é criada e o utilizador entra na parte autenticada da aplicação.
Fluxo de Exceção: Credenciais Inválidas	3.1. As credenciais são inválidas. 3.2. O sistema apresenta mensagem de erro e permite nova tentativa.

UC-P02: Registar Pessoa

Campo	Descrição
Identificador	UC-P02
Ator Principal	Colaborador

Campo	Descrição
Descrição	O colaborador regista uma nova pessoa no sistema, introduzindo os dados demográficos e criando as respetivas credenciais de acesso.
Pré-condições	O colaborador está autenticado. Não existe –a pessoa registada com o mesmo username ou email.
Pós-condições	A pessoa fica registada no sistema com um role inicial e credenciais de acesso.
Fluxo Normal	1. O colaborador acede ao formulário de registo. 2. Preenche os dados demográficos (nome, sexo, data de nascimento, email, telemóvel). 3. O sistema valida os dados e cria o registo.
Fluxo de Exceção: Dados Duplicados	4.1. O username ou email já existe no sistema. 4.2. O sistema apresenta erro e solicita correção.

UC-D05: Inscrever Equipa em Competição

Campo	Descrição
Identificador	UC-D05
Ator Principal	Treinador
Descrição	O treinador inscreve uma equipa numa competição oficial
Pré-condições	O utilizador está autenticado. Existe uma competição compatível com a modalidade da equipa e a equipa tem de estar ativa.
Pós-condições	A equipa fica inscrita na competição. A quota de inscrição em competição é gerada automaticamente para todos os atletas.
Fluxo Normal	1. O utilizador acede à competição pretendida. 2. Seleciona a equipa a inscrever. 3. O sistema valida a modalidade e o estado da época. 4. Regista a inscrição e gera a quota de participação automaticamente. 5. O sistema notifica os atletas da equipa da inscrição na competição.
Fluxo de Exceção: Modalidade Incompatível	3.1. A modalidade da equipa não coincide com a da competição. 3.2. O sistema rejeita a inscrição e informa o utilizador.

UC-C03: Efetuar Pagamento Online

Campo	Descrição
Identificador	UC-C03
Ator Principal	Atleta

Campo	Descrição
Descrição	O atleta inicia o pagamento online de uma quota em dívida. O sistema invoca o gateway externo e aguarda confirmação assíncrona.
Pré-condições	O atleta está autenticado. Existe pelo menos uma quota com estado “Dívida” associada ao atleta.
Pós-condições	A quota transita para “Pendente”. Após confirmação do gateway, transita para “Pago”.
Fluxo Normal	1. O atleta acede às suas quotas e seleciona uma em dívida. 2. Seleciona pagar com Multibanco. 3. O sistema invoca o gateway externo e transita a quota para “Pendente”. 4. O utilizador recebe entidade, referência e montante. 5. Após o pagamento, o sistema valida a assinatura do <i>webhook</i> e atualiza a quota para “Pago”. 6. O atleta recebe confirmação visual e notificação por email.
Fluxo de Exceção: Falha no Gateway	3.1. O gateway não responde dentro do tempo limite. 3.2. A quota mantém-se “Pendente” e o atleta é notificado. 3.3. O atleta pode repetir a operação mais tarde ou optar pelo fluxo alternativo.

Prompt utilizado:

Com base na lista de requisitos funcionais do Sistema de Gestão do Codfish United e nos atores identificados — Gerente, Colaborador, Treinador e Atleta — gera um conjunto de casos de uso representativos.

Para cada caso de uso, apresenta:

- identificador;
- ator principal;
- descrição;
- pré-condições;
- pós-condições;
- fluxo normal;
- fluxos de exceção relevantes.

Os casos de uso não devem repetir todos os requisitos individualmente, mas sim representar fluxos de negócio completos e relevantes para o sistema. Devem cobrir diferentes áreas funcionais, incluindo autenticação, gestão de pessoas, gestão desportiva e pagamentos.

1.12. Diagramas UML

Nesta fase foi criado um modelo de domínio para melhor perceber quais as entidades presentes no sistema bem como as relações entre elas, permitindo uma visão global das interações que irão acontecer

no sistema. Após o desenho do diagrama de domínio foi consultado o LLM para validar as relações e os atores presentes neste.

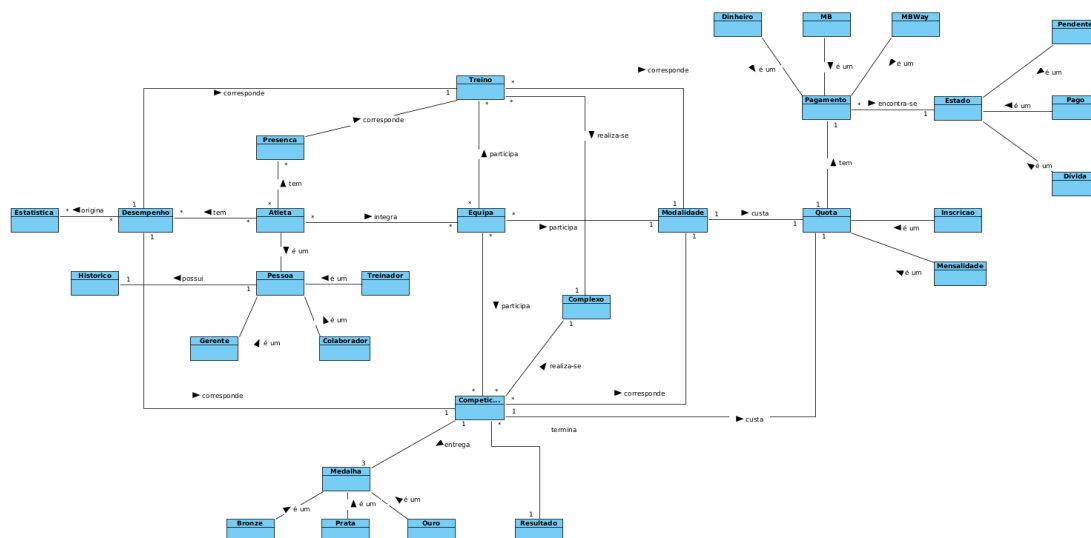


Figura 1: Modelo de Domínio do Codfish United

Prompt utilizado:

Desenvolvi manualmente um modelo de domínio para um sistema de gestão de um clube desportivo. O modelo inclui entidades relacionadas com pessoas, papéis, modalidades, equipas, vínculos temporais, treinos, competições, eventos, presenças, desempenhos, quotas e pagamentos.

Quero que faças uma validação crítica do modelo, sem o redesenhar do zero. Analisa se:

- as entidades principais do domínio estão corretamente representadas;
- existem entidades relevantes em falta;
- há relações redundantes ou mal posicionadas;
- as cardinalidades parecem coerentes com as regras de negócio;
- a separação entre gestão de pessoas, gestão desportiva e gestão financeira está clara;
- existem ambiguidades que possam causar problemas na fase de implementação.

Apresenta a resposta em formato de revisão técnica, indicando problemas encontrados, sugestões de melhoria e justificação para cada sugestão.

Nesta fase, foi também concebido um diagrama de casos de uso para mapear de forma clara as funcionalidades disponibilizadas pelo sistema e identificar as interações exatas entre os diferentes atores (internos e externos) e a plataforma, permitindo uma visão global do comportamento e do escopo operacional do clube. Após a modelação inicial, o LLM foi novamente consultado para realizar uma avaliação crítica do diagrama, o que permitiu validar a correta atribuição de privilégios aos perfis, refinar a separação lógica dos sete subsistemas e assegurar a representação técnica adequada de sistemas externos.

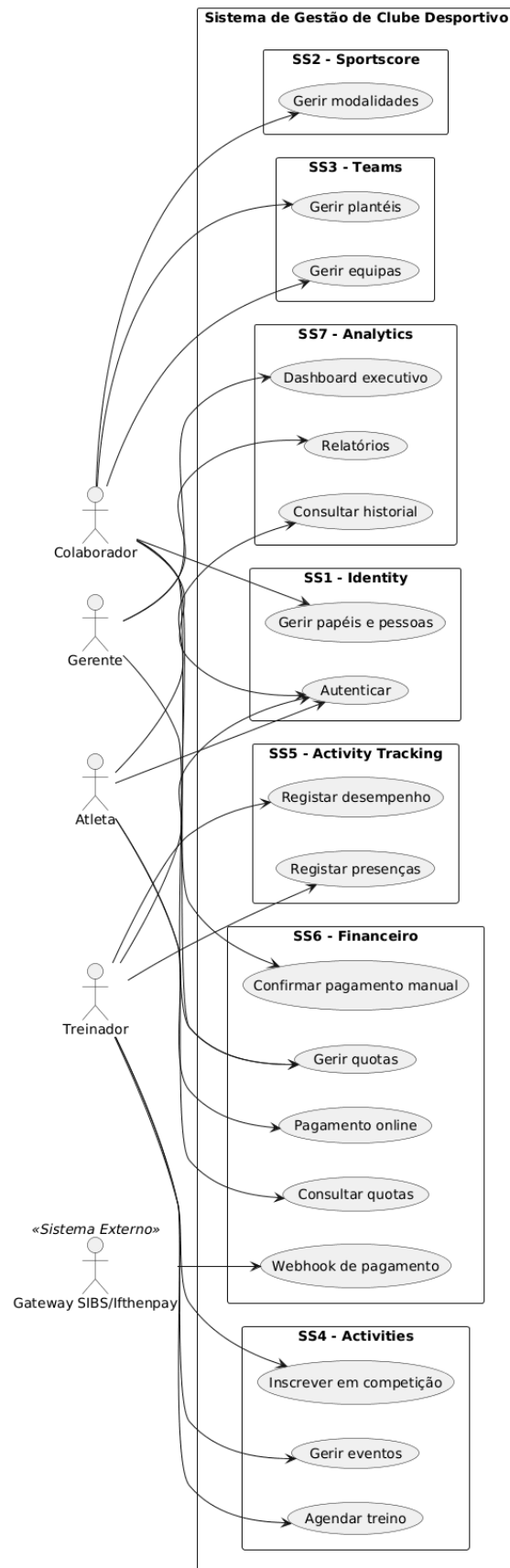


Figura 2: Diagrama de Use Cases do Codfish United

1.12.1. Dicionário de Entidades

A seguinte secção define individualmente cada entidade identificada no Modelo de Domínio, focando-se no seu propósito funcional e abstraindo a sua conectividade:

1. Pessoal e Identidade

- **Pessoa:** Entidade raiz que centraliza os dados demográficos (nome, data de nascimento, etc.) e as credenciais de autenticação de qualquer membro.
- **Gerente:** Perfil com privilégios de supervisão financeira e acesso a indicadores de gestão global.
- **Colaborador:** Perfil responsável pela execução das tarefas administrativas e de secretariado diário.
- **Treinador:** Perfil focado na gestão técnica de equipas, planeamento de treinos e avaliação de atletas.
- **Atleta:** Perfil que representa o praticante desportivo, sendo o sujeito principal das avaliações e quotas.
- **Histórico:** Registo cronológico e auditável que preserva a evolução dos papéis e estados de cada pessoa no sistema.

2. Estrutura e Atividades Desportivas

- **Modalidade:** Unidade de negócio que define uma disciplina desportiva específica (ex: Futebol, Judo).
- **Equipa:** Agrupamento organizacional de atletas e treinadores constituído para uma época desportiva.
- **Complexo:** Representação física das infraestruturas (campos, pavilhões) onde decorrem as atividades.
- **Treino:** Sessão agendada de preparação e instrução desportiva para uma determinada equipa.
- **Competição:** Prova ou torneio oficial, externo ou interno, com vertente classificativa.
- **Resultado:** Registo formal do desfecho final da participação de uma equipa num evento competitivo.
- **Medalha:** Distinção de mérito desportivo, categorizada tipologicamente em **Ouro**, **Prata** ou **Bronze**.

3. Avaliação e Métricas

- **Presença:** Registo que confirma a comparência efetiva de um atleta a uma atividade planeada.
- **Desempenho:** Avaliação da prestação de um atleta, podendo ser baseada em observações ou dados métricos.
- **Estatística:** Configuração da métrica ou parâmetro de medição (ex: golos, assistências, tempo) a aplicar no desempenho.

4. Gestão Financeira e Estados

- **Quota:** Obrigação financeira (débito) gerada pelo sistema, que pode ser de **Inscrição** ou **Mensalidade**.
- **Pagamento:** Registo da transação financeira, que pode ser liquidada via **Dinheiro**, **MB presencial** ou **MB online**.
- **Estado:** Classificação situacional do pagamento, podendo assumir os valores de **Pendente**, **Pago** ou **Dívida**.

1.12.2. Mapeamento de Relacionamentos e Cardinalidades

As interações entre as entidades são asseguradas pelos seguintes relacionamentos lógicos e multiplicidades, conforme representado no Modelo de Domínio:

Entidade Origem	Relacionamento	Entidade Destino	Cardinalidade
Pessoa	possui	Histórico	1 : 1
Atleta	integra	Equipa	N : N

Entidade Origem	Relacionamento	Entidade Destino	Cardinalidade
Atleta	tem	Presença / Desempenho	1 : N
Equipa	participa em	Modalidade	N : 1
Equipa	participa em	Treino / Competição	1 : N
Modalidade	corresponde a	Treino / Competição	1 : N
Modalidade / Competição	custa	Quota	1 : N
Treino / Competição	realiza-se em	Complexo	N : 1
Competição	termina em	Resultado	1 : 1
Competição	entrega	Medalha	1 : 3
Presença	corresponde a	Treino	N : 1
Desempenho	corresponde a	Treino / Competição	N : 1
Estatística	origina	Desempenho	1 : N
Quota	tem	Pagamento	1 : 1
Pagamento	encontra-se em	Estado	1 : 1

Prompt utilizado:

Com base na lista final de requisitos funcionais e nas regras de negócio identificadas (como a segregação de permissões e a gestão de quotas):

1. Identifica as principais entidades de domínio do sistema Codfish United.
2. Define os relacionamentos e as cardinalidades (ex: 1 para N) entre elas.
3. Garante que as seguintes restrições lógicas ficam representadas no modelo:
 - Uma Pessoa deve ter um histórico temporal de Papéis (Roles).
 - As Presenças e Avaliações aplicam-se a Treinos ou Eventos de Competição, havendo uma exclusividade mútua.
 - O pagamento de Quotas deve prever o registo de Notificações de Gateway Externo para efeitos de auditoria.

2. Capítulo 2 - Arquitetura e Design de Software utilizando LLM

A fase de desenho arquitetural teve como objetivo principal a tradução do modelo conceitual, estabelecido no Capítulo 1, para uma solução técnica robusta, escalável e de fácil manutenção. Dada a complexidade operacional — desde integrações financeiras externas até à avaliação desportiva em dispositivos móveis — tornou-se imperativa a adoção de uma arquitetura modular baseada no modelo clássico Cliente-Servidor, com separação física e lógica entre o *Frontend* e o *Backend*.

2.1. Arquitetura de Servidor (Backend - Spring Boot)

A API RESTful foi desenvolvida em **Java 21** com **Spring Boot**, adotando o padrão de **Monólito estruturado por subsistemas funcionais**. Em vez de espalhar lógicas por pacotes genéricos (como controllers ou services globais), o sistema foi dividido em sete (7) diretorias de domínio independentes (identity, sportscore, teams, activities, activitytracking, finance, analytics).

Cada um destes subsistemas obedece a uma arquitetura interna em camadas rigorosa:

- **Controller:** O ponto de entrada REST. Lida exclusivamente com o roteamento HTTP, códigos de estado e validação sintática (@Valid) dos *RequestDTOs*.
- **Application (Facade):** A única porta de entrada para a lógica de negócio do módulo. Por exemplo, a *SS4FacadeImpl* orquestra chamadas a múltiplos serviços internos, garantindo a atomicidade das operações através da anotação @Transactional.
- **Service:** Onde residem as regras de domínio puras (ex: *TeamRulesService* ou *SS2RulesService*). Estes serviços não conhecem a web (HTTP) nem comunicam com o exterior, focando-se apenas em validar o estado das entidades.
- **Domain e DTOs:** As entidades JPA (@Entity) estão isoladas no pacote domain e nunca são expostas na API. A transformação entre Entidades e DTOs é assegurada por classes *Mapper* dedicadas, garantindo um contrato de dados blindado.
- **Repository:** O acesso e persistência à base de dados PostgreSQL são abstraídos por interfaces do **Spring Data JPA**, eliminando a necessidade de DAOs manuais e prevenindo injeções de SQL.

Prompt utilizado:

Prompt submetido: Atua como um Arquiteto de Software Spring Boot (Java). Considerando os nossos 7 módulos lógicos, desenha a “package structure” para um monólito organizado por módulos (subsistemas) em Spring Boot.

1. Queremos isolar cada subsistema de forma a que alterações nas Finanças não afetem os Treinos.

2. Recomenda Padrões de Desenho (como Facade e DTO) para garantir que as Entidades JPA nunca sejam expostas na API REST.
3. Explica como os Repositórios do Spring Data JPA devem ser organizados dentro desta estrutura.

2.2. Arquitetura de Cliente (Frontend - React SPA)

Para garantir que o *Frontend* acompanhasse a escalabilidade do *Backend* sem degenerar em código esparguete (*spaghetti code*), a *Single Page Application* (SPA) em **React (TypeScript e TailwindCSS)** abandonou a estruturação tradicional (agrupar todos os componentes ou todos os *hooks* globais) em prol de uma organização orientada a domínios de negócio, inspirada no **Feature-Sliced Design**.

A diretoria `src/` está dividida de forma cirúrgica:

- **Features:** O coração da aplicação. Aqui replicam-se os 7 subsistemas do *backend* (ex: `finance/`, `teams/`, `identity/`). Cada funcionalidade encapsula os seus próprios *endpoints* HTTP (`api/`), páginas (`pages/`), lógica de acesso a dados (`hooks/`), tipagens e *mappers* (`model/`) e componentes visuais específicos (`components/`).
- **Shared:** Contém elementos transversais e reutilizáveis, agnósticos à lógica de negócio. Inclui a biblioteca de UI base (botões, *cards*, diálogos em `components/ui/`), *hooks* globais de navegação e formatação de datas.
- **Core e Infraestrutura:** Gere a configuração central do *Router*, a inicialização do tema, a gestão de *tokens* JWT no *Local Storage* e a configuração base do *Axios* (`http-client.ts`), incluindo intercetores genéricos para o tratamento automático de erros HTTP (401, 403).

Prompt utilizado:

Prompt submetido: Atua como um Arquiteto Frontend Sênior. Precisamos de organizar uma aplicação React/TypeScript complexa que irá consumir uma API com 7 subsistemas distintos.

1. Desenha uma “package structure” escalável (ex: Feature-Sliced Design) que espelhe a separação dos módulos do backend.
2. Detalha o que deve constar dentro de cada “feature” (API, Hooks, Pages, Componentes lógicos).
3. Sugere como isolar configurações globais de rede e componentes de UI genéricos (botões/modais) para máxima reutilização.

2.3. Decisões Arquiteturais: Abstração de Gateways de Pagamento

Um dos desafios arquiteturais mais relevantes do projeto (focado no Subsistema Financeiro - SS6) foi a integração com métodos de pagamento externos (e.g., referências Multibanco). Em vez de acoplar rigidamente o sistema a um fornecedor específico na fase de desenvolvimento, a equipa optou por aplicar o **Princípio da Inversão de Dependência**.

O fluxo foi desenhado em ambas as arquiteturas (Cliente e Servidor) para suportar o ciclo de vida real de um *Gateway* externo: o Spring Boot gera o pedido e transita a dívida para “Pendente”, o React exibe a Entidade/Referência ao utilizador, e o sistema disponibiliza um *endpoint* (/api/gateway/notifications) para receber a confirmação assíncrona (*webhook*).

Contudo, para efeitos de validação académica e para garantir a total autonomia da plataforma sem dependência de credenciais reais ou conectividade de terceiros, a implementação utiliza um *Mock* (simulador) interno. Este simulador imita o comportamento do *Gateway* e permite o envio de um *webhook* de sucesso através da interface do React. Esta abordagem garante que a transição para o “mundo real” é um processo linear e não intrusivo: não exige a reescrita da lógica de apresentação ou do motor transacional, bastando injetar o URL e as chaves de API do fornecedor definitivo nas variáveis de ambiente da infraestrutura.

2.4. Explorar LLM na Revisão e Coesão Arquitetural

Nesta etapa de *design*, a IA serviu como ponto de charneira para garantir a consistência entre a arquitetura do cliente e a do servidor. A interação focou-se na auditoria da coesão do sistema:

1. **Alinhamento de Contratos (Tipagens End-to-End):** Apresentou-se a proposta de DTOs do Java ao modelo para garantir que os ficheiros TypeScript (`types.ts`) e os *mappers* no `src/features/*/model/` correspondiam perfeitamente aos objetos JSON gerados pela API, mitigando quebras contratuais.
2. **Revisão de Dependências Circulares:** O modelo analisou os diagramas de pacotes gerados para verificar se a separação em *Monólito Modular* era efetiva. Validou-se que não existiam invocações não autorizadas, confirmando que, por exemplo, o SS4 (Activities) não importa repositórios do SS6 (Finance) no Spring Boot, nem cruza *Hooks* incorretamente no React.

Prompt utilizado:

Prompt submetido: Atua como um Arquiteto de Software Sênior Full-Stack. Com base nas estruturas de diretórios que definimos para o React e para o Spring Boot:

1. Avalia se a nossa estratégia de comunicação Cliente-Servidor via DTOs previne o sobrecarregamento de dados (over-fetching).
2. Avalia a implementação do *Mock* interno para simular o Gateway de Pagamentos, confirmando se o contrato desenhado permite uma transição “plug-and-play” para uma API real em produção sem refatorizar a UI ou a Camada de Serviço.

2.5. Diagramas de Classes por Subsistema

A modelação detalhada do *design* do sistema foi fragmentada em sete diagramas de classes, um para cada subsistema lógico, refletindo a adoção de *Domain-Driven Design* (DDD) sobre a *framework* Spring Boot. Todos os diagramas partilham a mesma fundação arquitetural: uma interface pública de serviço, a implementação concreta contendo a lógica de negócio, as entidades JPA mapeadas para a base de dados relacional, e as interfaces de acesso a dados nativas do Spring Data JPA.

1. SS1 - Identity (Gestão de Pessoal)

Foca-se na segurança, autenticação e conformidade com o RGPD. Expõe a `ISS1IdentityFacade`, que

orquestra os fluxos de *login* e a segregação de perfis (RBAC - *Role-Based Access Control*). No domínio, destaca-se a entidade Pessoa acedida via PessoaRepository. A gestão do ciclo de vida dos utilizadores é suportada pela entidade PessoaRole (gerida pelo PessoaRoleRepository), que garante a imutabilidade e a rastreabilidade temporal dos papéis atribuídos, um requisito crítico para auditorias.

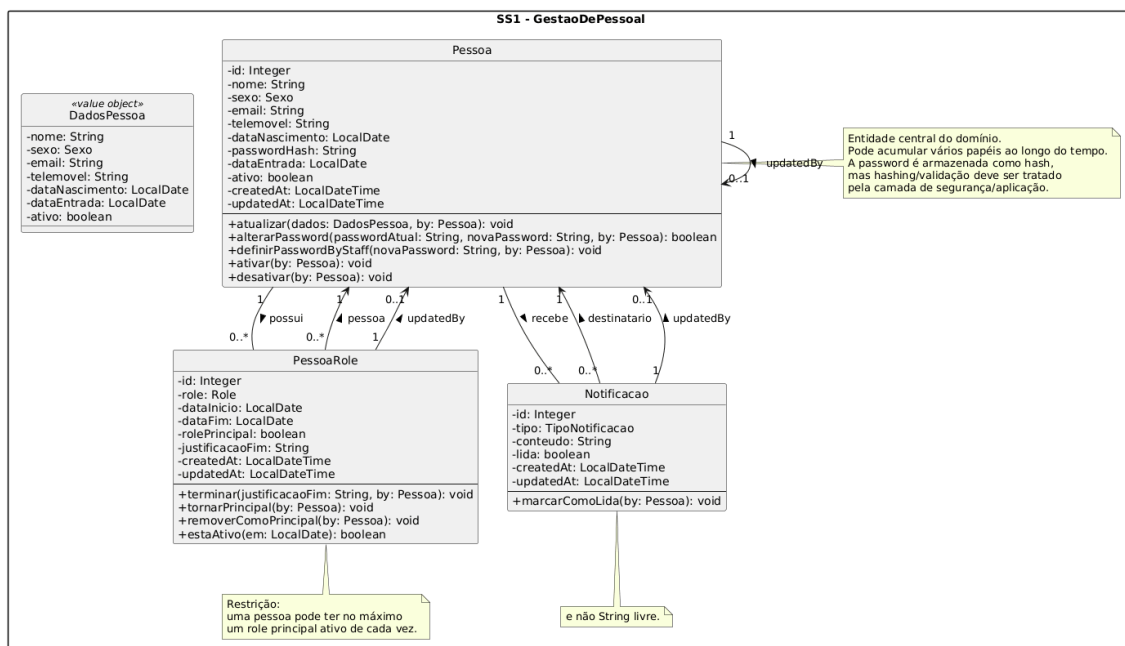


Figura 3: Diagrama de classes do SS1 - Identity

2. SS2 - Sportscore (Modalidades e Sistema)

Este diagrama ilustra o núcleo de parametrização do clube. A ISS2SportscoreFacade permite configurar as regras estruturais do negócio. Interage com o ModalidadeRepository para definir preçários base e escalões (garantindo valores não-negativos), com o EstatisticaRepository para parametrizar as métricas de avaliação dinâmica, e com o ComplexoRepository para gerir o inventário de infraestruturas. Sendo dados de baixa mutabilidade, este módulo foi desenhado para ser facilmente cacheável pela camada de serviço.

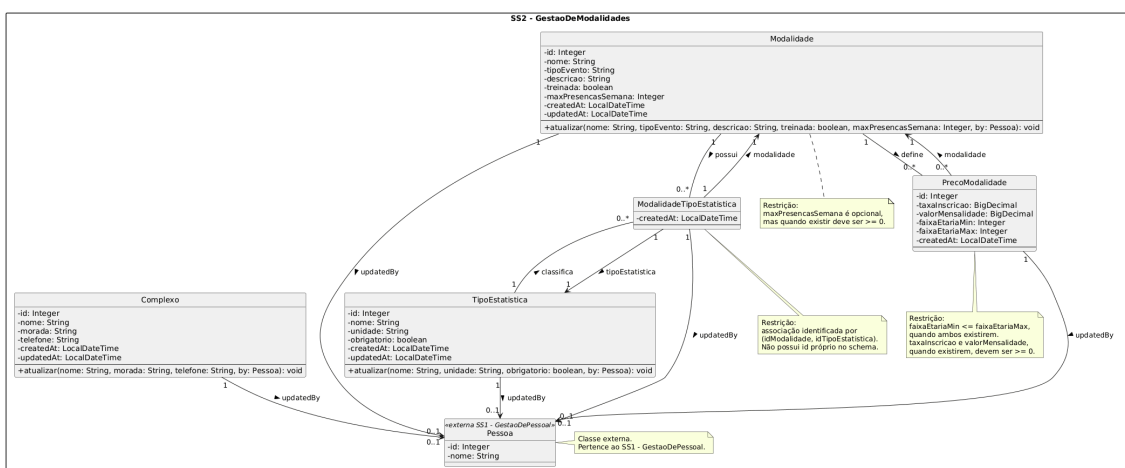


Figura 4: Diagrama de classes do SS2 - Sportscore

3. SS3 - Teams (Gestão de Equipas)

Dedicado exclusivamente à alocação de recursos humanos, este diagrama expõe a ISS3TeamsFacade. A complexidade deste módulo reside na integridade relacional temporal. A lógica de negócio assegura, atra-

vés do EquipaRepository e do motor de persistência do Hibernate, que as associações N:M entre atletas, treinadores e equipas não contêm sobreposições cronológicas ilógicas durante uma época desportiva.

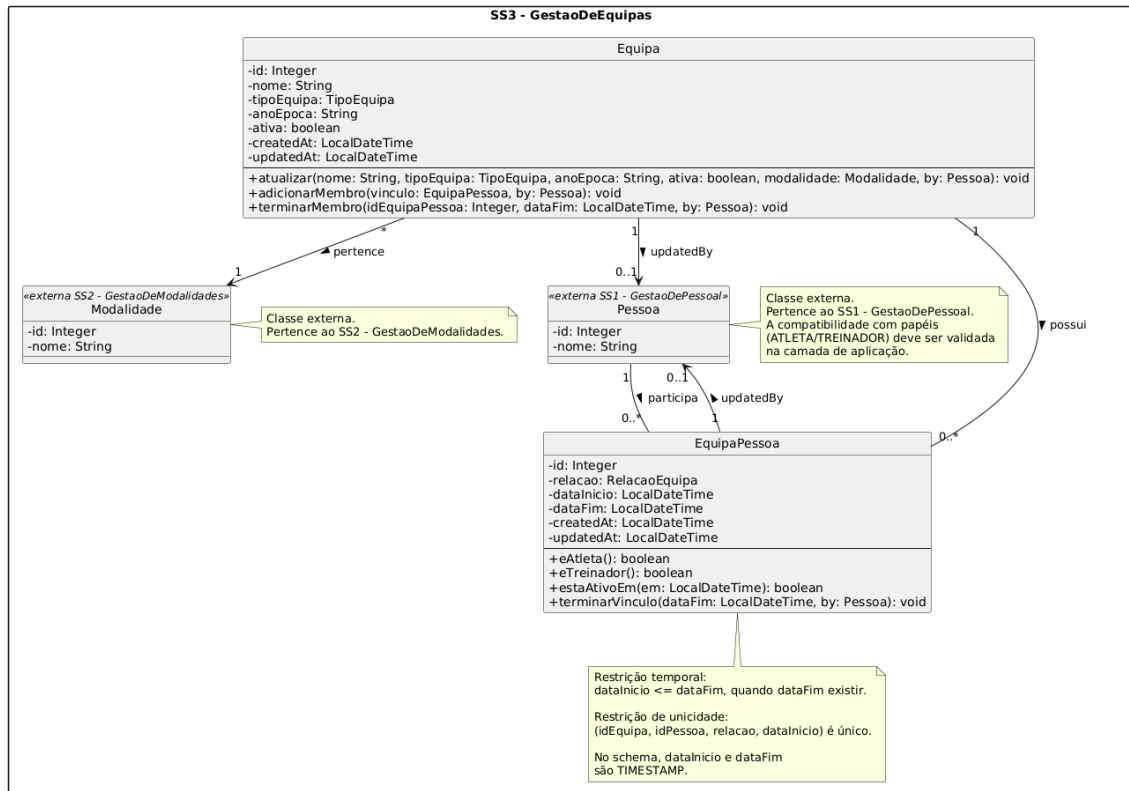


Figura 5: Diagrama de classes do SS3 - Teams

4. SS4 - Activities (Competições, Eventos e Treinos)

Módulo responsável pelo motor de agendamento e disponibilidade. A ISS4ActivitiesFacade coordena o ciclo de vida dos eventos desportivos. O CompeticaoRepository e o TreinoRepository persistem as marcações, associando-as obrigatoriamente aos complexos do SS2. O serviço garante validações rigorosas antes da persistência, rejeitando anomalias como sobreposição de horários no mesmo pavilhão ou datas de fim anteriores às datas de início.

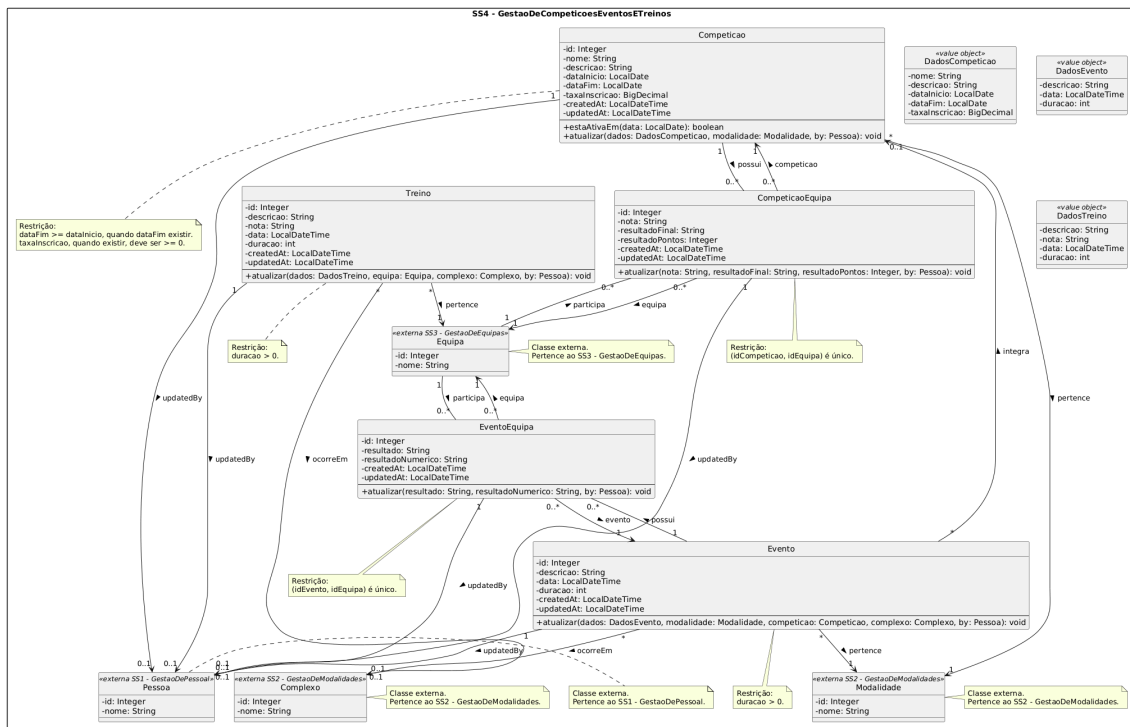


Figura 6: Diagrama de classes do SS4 - Activities

5. SS5 - Activity Tracking (Presenças e Desempenho)

Desenhado para suportar o elevado volume de transações gerado pelas operações no terreno. A ISS5TrackingFacade abstrai a complexidade da submissão em lote (*batch processing*) de presenças e avaliações. O aspeto mais crítico deste *design*, suportado pelo PresencaRepository, é a implementação algorítmica da regra XOR: o serviço garante que os dados submetidos pelo treinador são injetados exclusivamente no contexto de um Treino **ou** de um Evento, abortando a transação em caso de ambiguidade.

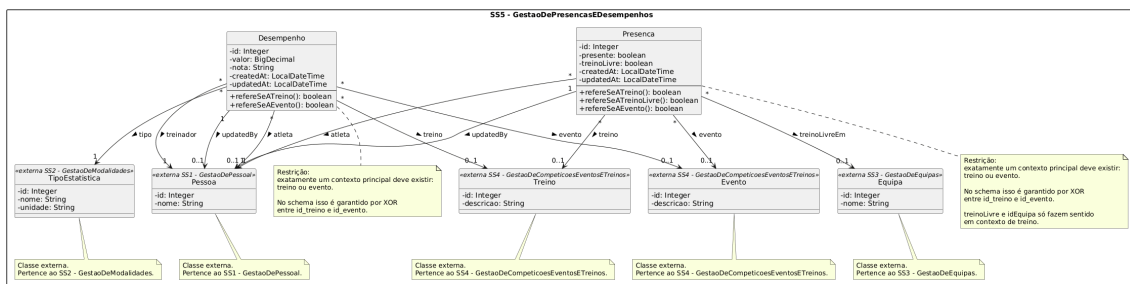


Figura 7: Diagrama de classes do SS5 - Activity Tracking

6. SS6 - Finance (Contabilidade e Finanças)

O motor transacional do sistema. A `ISS6FinanceFacade` orquestra a geração de faturas e o processamento de débitos. Interage com o `QuotaRepository` para monitorizar o incumprimento e com o `PagamentoRepository` para transições de estado. O método `processarWebhook()` é o ponto central deste diagrama, exigindo anotações `@Transactional` estritas para garantir propriedades ACID e a idempotência na comunicação assíncrona com gateways externos (ex: SIBS), evitando duplicação de pagamentos.

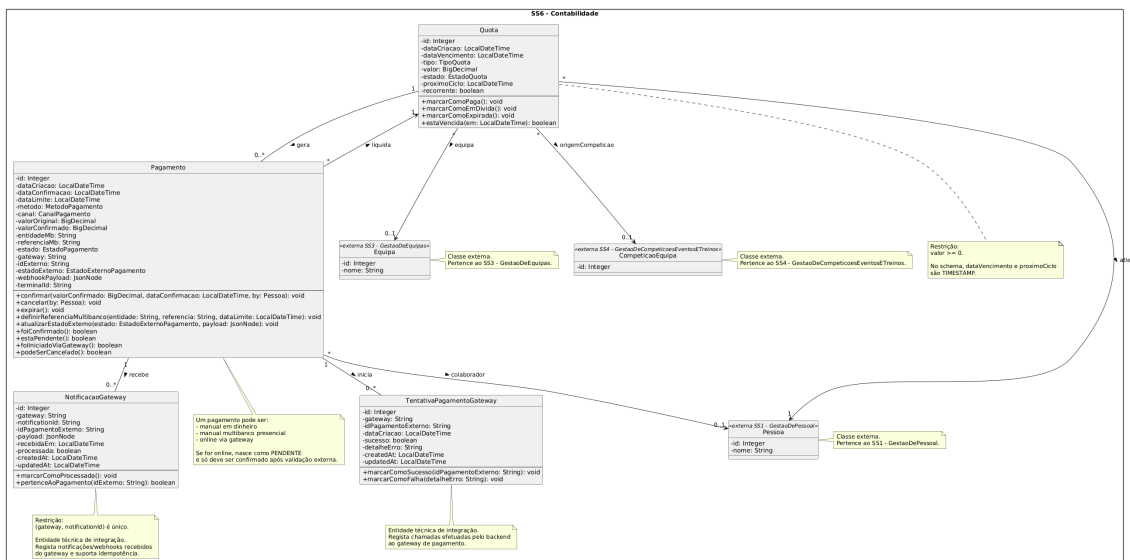


Figura 8: Diagrama de classes do SS6 - Finance

7. SS7 - Analytics (Dashboards e Relatórios)

Este subsistema destaca-se pelo seu padrão puramente arquitetural de leitura (*Read-Model*). A *ISS7AnalyticsFacade* omite repositórios de escrita tradicionais. Em vez disso, utiliza *Spring Data Projections* e *queries* nativas otimizadas para agregar dados transversais (como taxas de retenção, valor em dívida global e evolução de performance). Estes dados são projetados diretamente em DTOs, garantindo tempos de resposta ultrarrápidos para o consumo dos *dashboards* na aplicação React.

Prompt utilizado:

Atua como um Arquiteto de Software Java/Spring Boot. Vou fornecer-te as entidades da modelação de domínio e os requisitos do subsistema SS[X]. Gera o código PlantUML para um Diagrama de Classes de Design contemplando:

1. Uma interface `ISS[X]Facade` e a sua implementação.
 2. As Entidades da lógica de negócio JPA correspondentes.
 3. As interfaces genéricas Spring Data JPA que injetam dependências no Service.
 4. Oculta os detalhes de métodos `getters/setters` e foca-te nas assinaturas relevantes para o negócio.
- Adiciona notas explicativas para as regras de negócio mais complexas (ex: validações XOR, gestão de transações ACID).

2.6. Diagramas de Sequência

Para validar o fluxo de dados entre a *Single Page Application* (React), a API REST (Spring Boot) e a base de dados PostgreSQL, foram modelados diagramas de sequência focados nos fluxos de negócio mais críticos. A geração destes diagramas foi assistida por LLM, garantindo a representação correta das chamadas assíncronas, injeção de dependências e fronteiras transacionais. Considerou-se relevante modelar o processo de pagamento online, tanto na solução efetivamente implementada em contexto académico, baseada num gateway mock com simulação de callback, como na solução ideal de integração real com um gateway externo.

1. Atribuição de Desempenho a um Atleta (SS5 - Activity Tracking)

Este fluxo representa o registo ou atualização de desempenhos desportivos por parte de um treinador. O pedido chega ao SS5Controller, onde é aplicada a autorização necessária para garantir que o treinador tem acesso ao treino em causa. De seguida, a SS5Facade valida o contexto do pedido e processa os desempenhos submetidos. Para cada atleta e tipo de estatística, o sistema verifica se já existe um registo anterior; caso exista, o valor é atualizado, caso contrário é criado um novo desempenho. A operação decorre numa fronteira transacional, garantindo consistência entre validação e persistência.

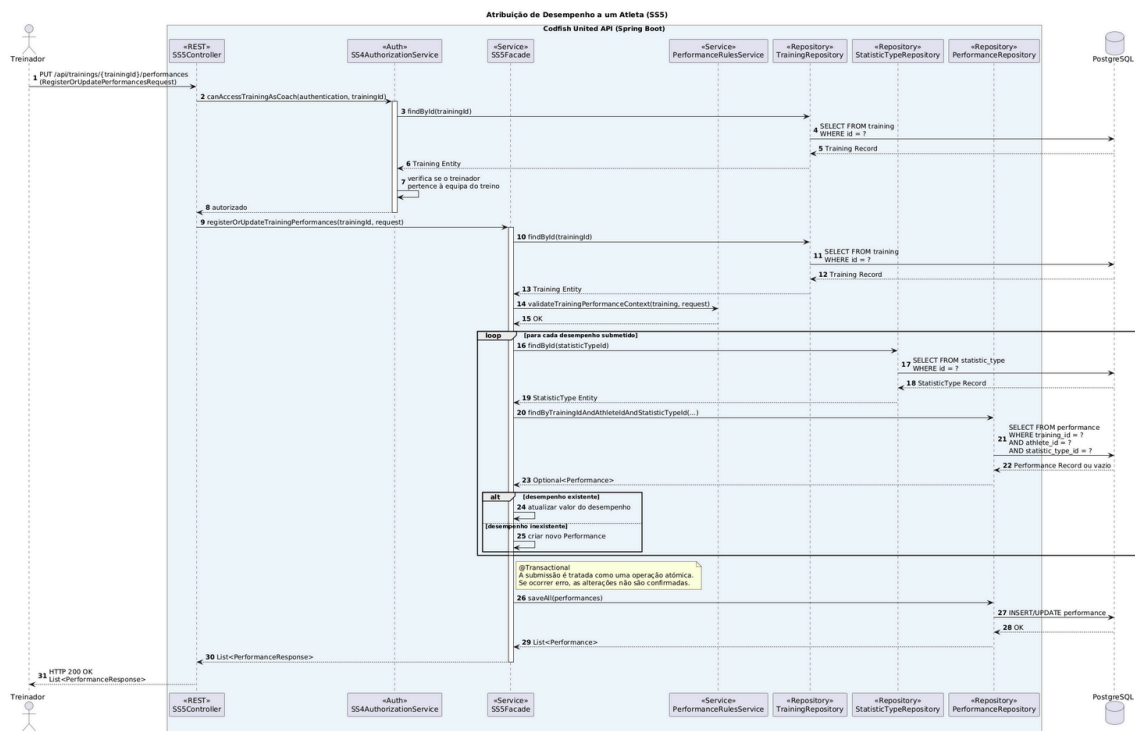


Figura 9: Diagrama de Sequência: Submissão Lote de Presenças (SS5)

2. Pagamento Online com Gateway Mock e Simulação de Callback (SS6 – Contabilidade)

Este fluxo representa a solução efetivamente implementada no projeto académico. Em vez de depender de credenciais reais de um fornecedor externo, o backend gera dados de pagamento simulados (entidade e referência) e expõe um mecanismo de simulação de callback. O diagrama demonstra como a aplicação preserva a mesma lógica nuclear do processamento assíncrono — criação de pagamento, registo de tentativa, processamento de notificação, verificação e atualização transacional do estado da quota — sem depender de uma integração externa real.

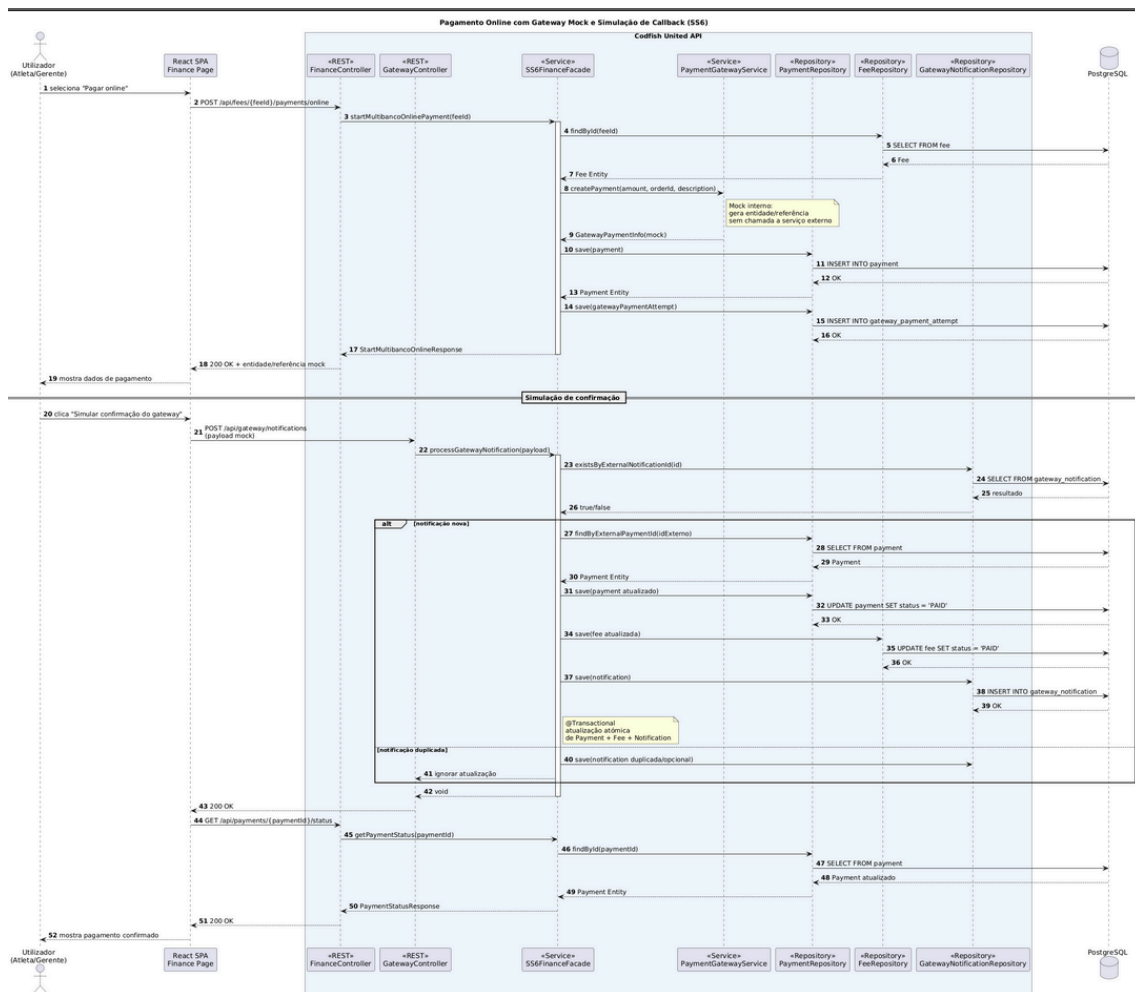


Figura 10: Diagrama de Sequência: Pagamento Online com Gateway Mock e Simulação de Callback (SS6)

3. Integração Real com Gateway de Pagamento Externo (Visão Arquitetural)

Este fluxo representa a solução ideal de produção, na qual o backend comunica diretamente com um gateway de pagamento externo (por exemplo, em ambiente sandbox ou produção). O diagrama evidencia a criação remota do pagamento, a devolução de identificadores e referências externas, e a receção assíncrona de uma notificação de confirmação. Embora esta variante não tenha sido utilizada na implementação académica final, foi modelada para demonstrar que a arquitetura adotada suporta uma migração futura para uma integração real com mínimo impacto estrutural.

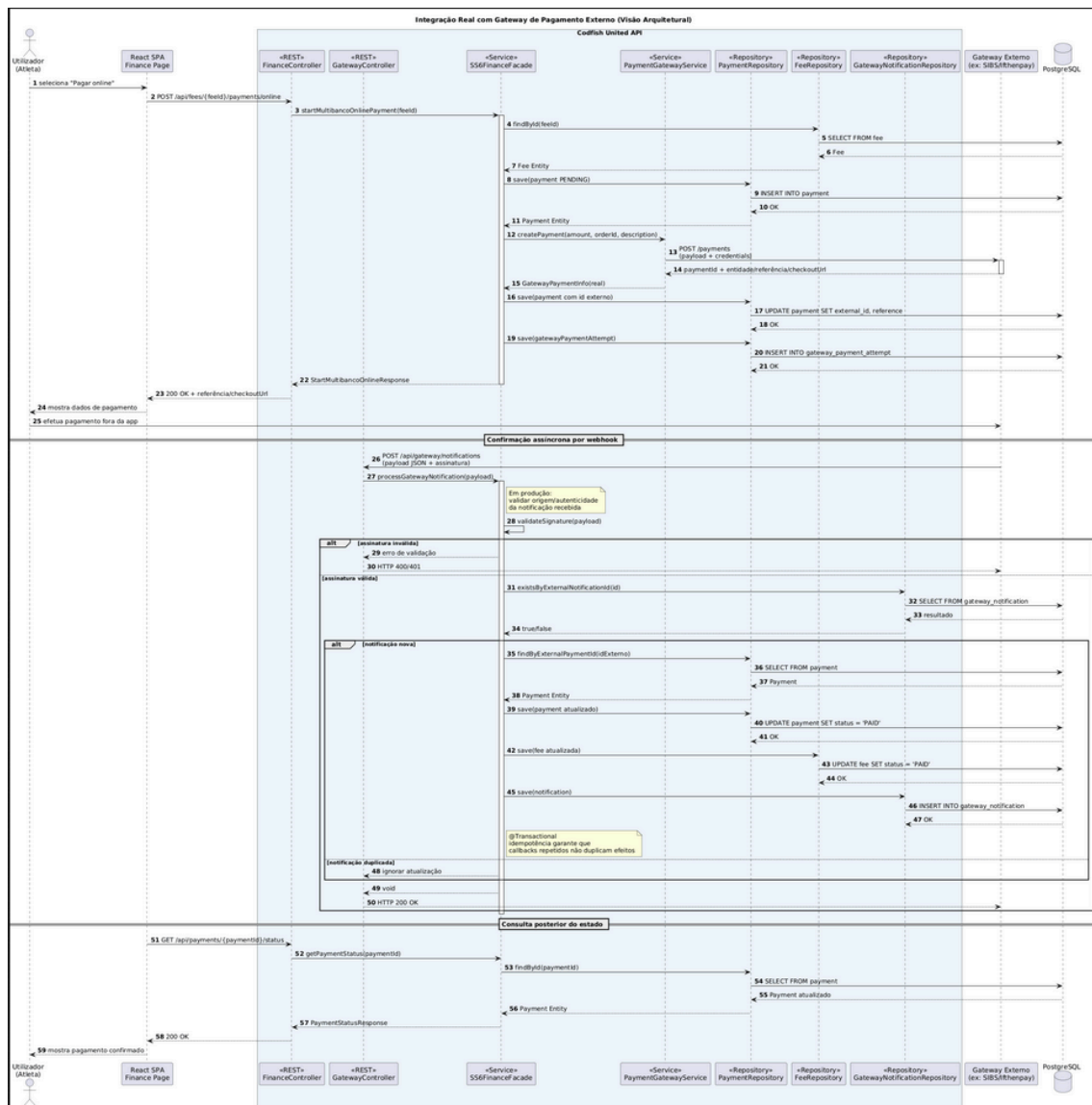


Figura 11: Diagrama de Sequência: Integração Real com Gateway de Pagamento Externo (Visão Arquitetural)

4. Processamento de Pagamento via Webhook (SS6 - Contabilidade) Este fluxo ilustra a comunicação assíncrona com o *gateway* de pagamentos externo (ex: SIBS). O diagrama demonstra como a ISS6FinanceFacade recebe o *payload* JSON no *controller*, valida a assinatura criptográfica, verifica a idempotência da notificação e, sob uma anotação *@Transactional*, atualiza o estado da Quota e do Pagamento, garantindo que não ocorrem atualizações parciais em caso de falha.

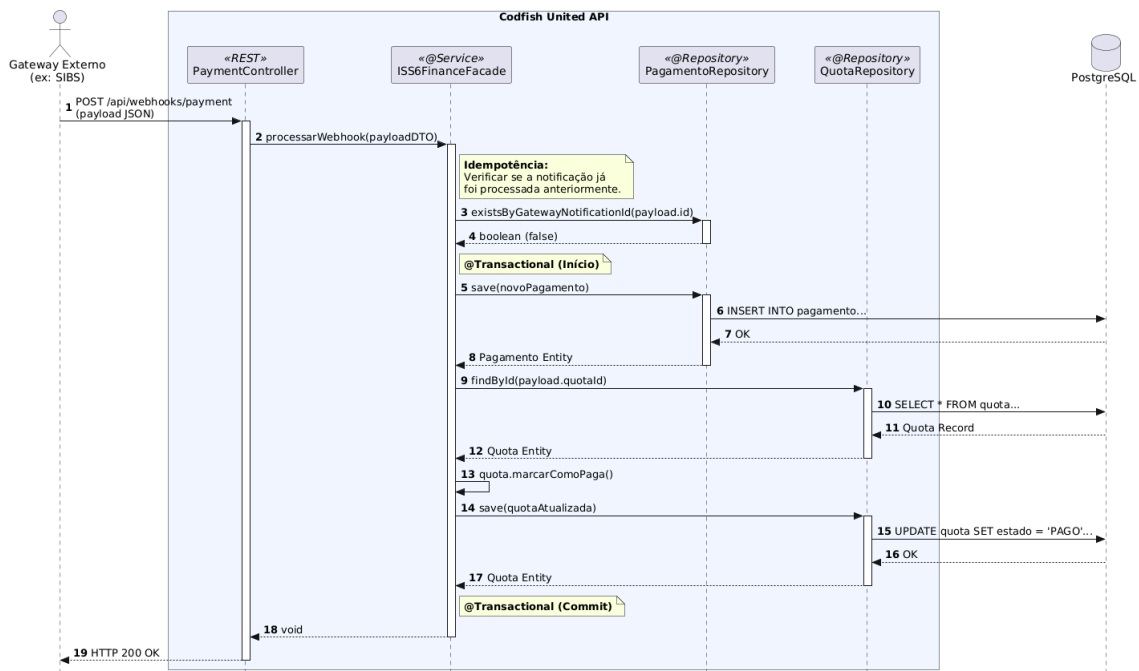


Figura 12: Diagrama de Sequência: Processamento de Webhook de Pagamento (SS6)

Prompt utilizado:

Atua como Arquiteto de Software. Com base no diagrama de classes do SS6, gera o código PlantUML para o Diagrama de Sequência do Caso de Uso “Receber Webhook de Pagamento”. Mostra a interação desde o Sistema Externo (Gateway) -> REST Controller -> ISS6FinanceFacade -> QuotaRepository. Representa explicitamente a fronteira transacional e a validação de idempotência.

2.7. Diagramas de Componentes

Para clarificar a topologia de *deployment* e a organização dos artefactos de *software*, foi desenhado o Diagrama de Componentes. Este modelo reflete a arquitetura cliente-servidor adotada:

- **Componente Cliente (Frontend):** Uma aplicação web desenvolvida em React e TypeScript, empacotada via *bundler* (ex: Vite/Webpack), que consome exclusivamente a API REST.
- **Componente Servidor (Backend API):** A aplicação Spring Boot, executada num contentor Java (ex: Tomcat embebido). Este componente engloba os *Controllers* REST, a camada de Segurança (filtros JWT) e os 7 subsistemas lógicos (módulos JAR internos ou pacotes Java isolados).
- **Componentes de Infraestrutura:** A base de dados PostgreSQL e os nós de integração externos (Gateways de Pagamento e Serviços de Email/Notificação).

O diagrama evidencia o padrão de portas e adaptadores, onde o núcleo de negócio do Codfish United se encontra isolado de detalhes de interface (HTTP) e de persistência (JDBC).

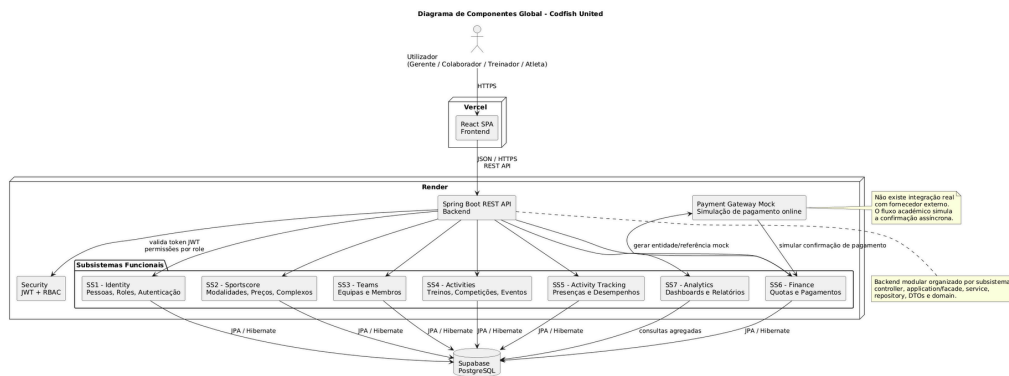


Figura 13: Diagrama de Componentes Global da Arquitetura Codfish United

Prompt utilizado:

Atua como um Arquiteto de Software Sênior. Com base na nossa arquitetura de Monólito estruturado por módulos funcionais (7 subsistemas em Spring Boot), *Frontend* em React e dependências externas (PostgreSQL, Gateway SIBS, Servidor SMTP), gera o código PlantUML para o Diagrama de Componentes Global.

Garante que o modelo reflete o padrão de Portas e Adaptadores:

1. Representa as aplicações Cliente (SPA Web e Mobile) a consumir a API REST via HTTP.
2. Modela o Servidor (JVM) isolando os *Controllers*, a camada de Segurança (filtros JWT) e os 7 pacotes lógicos de negócio (SS1 a SS7).
3. Demonstra as ligações físicas de persistência (JDBC/JPA) à base de dados e a integração com sistemas externos.

2.8. Design de Interfaces e Prototipagem

O *design* da interface gráfica do sistema Codfish United foi materializado através de um protótipo funcional desenvolvido com o auxílio da ferramenta **Figma Make**. Esta abordagem permitiu a criação célere de uma *Single Page Application* (SPA) de alta fidelidade, focada na validação antecipada da experiência de utilizador (UX) e na garantia de que a arquitetura *Role-Based Access Control* (RBAC) e os 50 requisitos funcionais delineados no Capítulo 1 são integralmente satisfeitos.

Prompt utilizado:

Atua como um Especialista em UI/UX e Frontend. Com base na nossa lista de 50 requisitos e na matriz de visibilidade por perfis em anexo (Gerente, Colaborador, Treinador e Atleta), utiliza o Figma Make para gerar um protótipo funcional em React/TailwindCSS.

1. Desenha uma Landing Page pública e um ecrã de Login com simulação de perfis.
2. Estrutura uma área autenticada com barra lateral adaptativa que oculte módulos sensíveis consoante o utilizador autenticado.
3. Cria diversas páginas compostas por diferentes componentes específicas para: Dashboard de Gestão, Lista de Equipas, Registo de Presenças e Área Financeira do Atleta, etc

4. Na barra lateral cria um seletor de vista ativa

2.8.1. Rastreabilidade: Protótipos e Requisitos Funcionais

A interface não foi concebida apenas sob critérios estéticos, mas como a materialização das regras de negócio do sistema. Abaixo, detalha-se o cruzamento entre os ecrãs produzidos e os requisitos (R) que estes comprovam:

A. Portal Público e Onboarding A *Landing Page* constitui o ponto de entrada para utilizadores não autenticados, focando-se na visibilidade das modalidades e na captação de sócios, estabelecendo a identidade visual do clube.

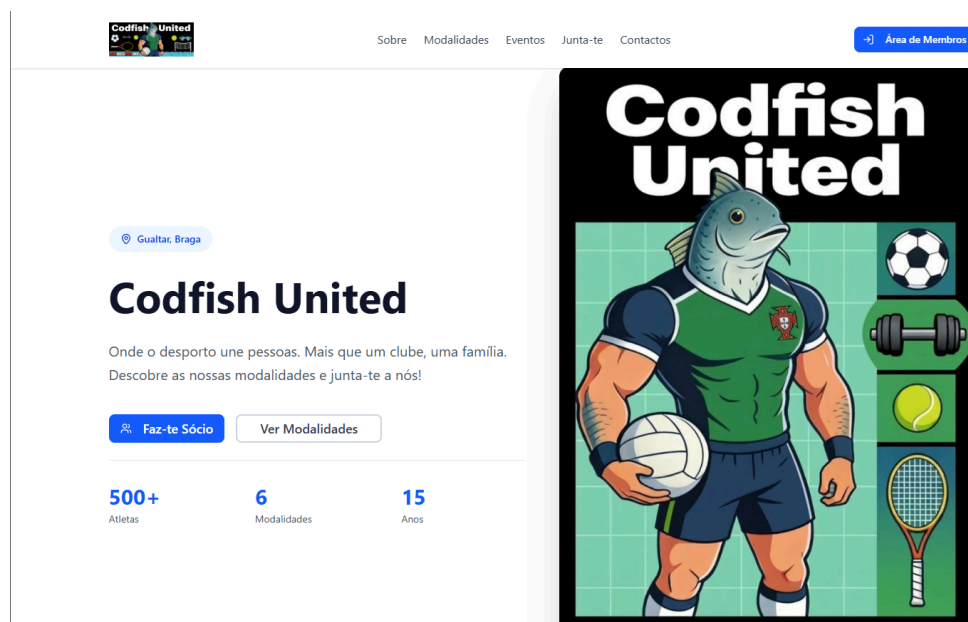


Figura 14: Portal Público do Codfish United: Foco na conversão e exposição de modalidades.

B. Autenticação e Fundação RBAC (R02) O ecrã de *Login* materializa o requisito R02 (Autenticação) e assegura o isolamento imediato de contexto consoante a *Role* do utilizador. O protótipo inclui mecanismos de simulação para validar as diferentes árvores de navegação de cada ator.

Figura 15: Ecrã de Autenticação. Comprova o requisito R02 e a fundação do sistema de permissões.

C. Gestão Estrutural e Backoffice (R07, R11, R18) A vista administrativa centraliza a gestão de recursos. O ecrã de Equipas permite a consulta de dados (R07) e fornece acesso direto à criação de novas instâncias desportivas (R18) através do botão “Nova Equipa”. A barra lateral adapta-se para expor módulos como “Pessoas” e “Modalidades” (R11) apenas a utilizadores autorizados.

Figura 16: Vista Administrativa de Equipas. Satisfaz os requisitos R07, R11 e R18.

D. Operações de Terreno - Treinador (R30, R33) A interface do Treinador oculta ruído administrativo para focar na atividade desportiva. O ecrã de “Treinos e Presenças” permite o controlo operacional do plantel (R30), apresentando taxas de assiduidade e históricos recentes para apoiar a avaliação de desempenho (R33).

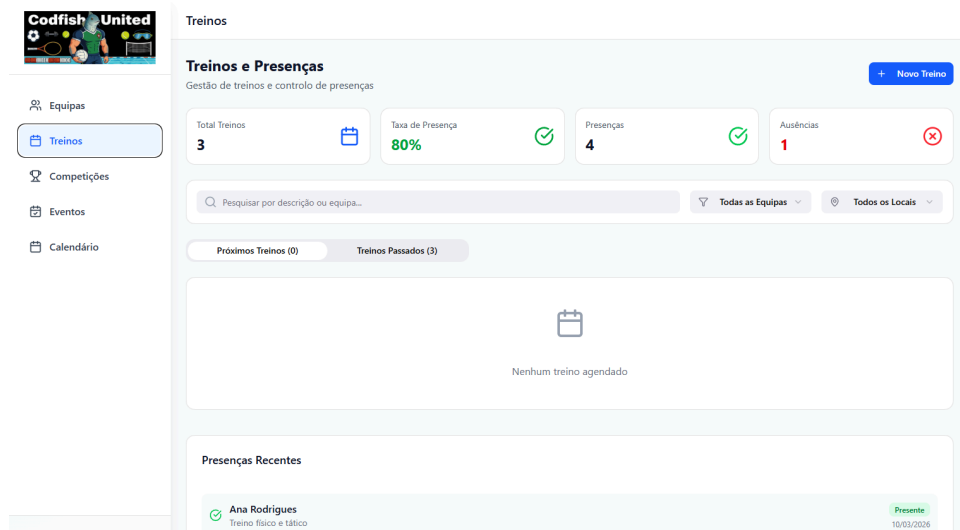


Figura 17: Painel de Treinos e Presenças. Comprova o cumprimento dos requisitos R30 e R33.

E. Gestão Financeira Self-Service - Atleta (R38, R41) O portal do Atleta garante a privacidade absoluta, limitando o acesso apenas às suas próprias obrigações financeiras. O ecrã “As minhas finanças” permite a consulta do estado de quotas (R38) e prepara o fluxo para a liquidação de dívidas via *gateway* (R41).

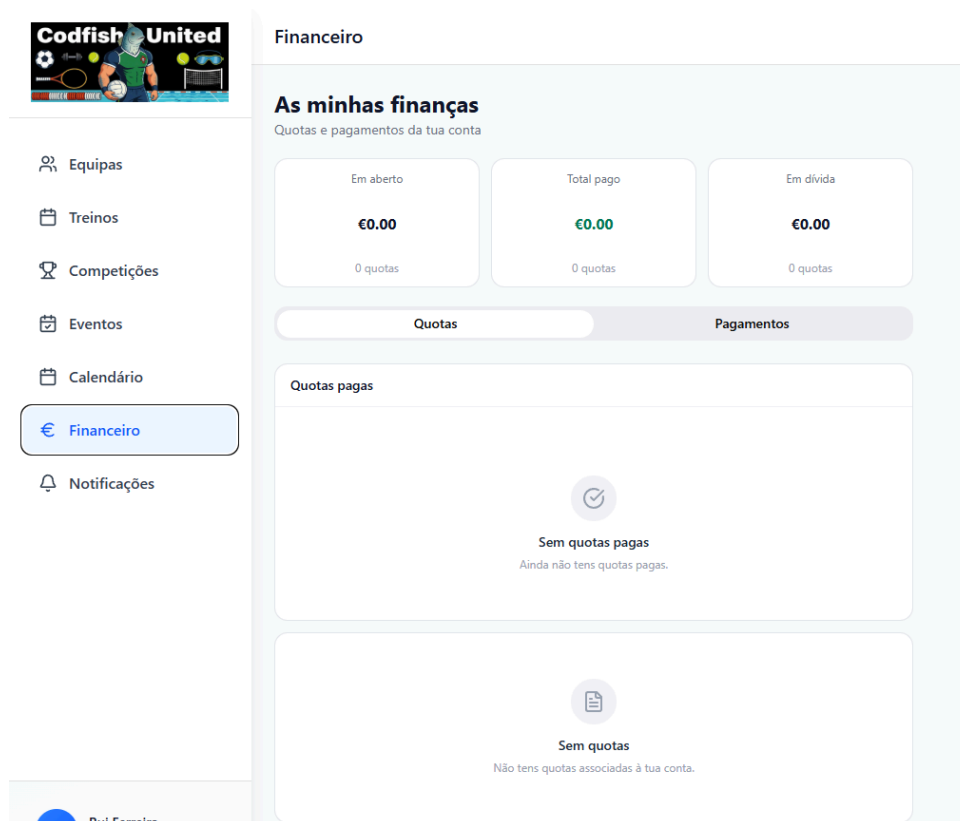


Figura 18: Portal Financeiro do Atleta. Implementa os requisitos R38 e R41.

F. Agregação Analítica - Gerência (R26, R46) Exclusivo da Gerência, o *Dashboard* consome os dados agregados para expor KPIs globais (R46). Gráficos de *Snapshot* Financeiro e métricas de adesão permitem uma visão holística e em tempo real do estado do clube (R26).

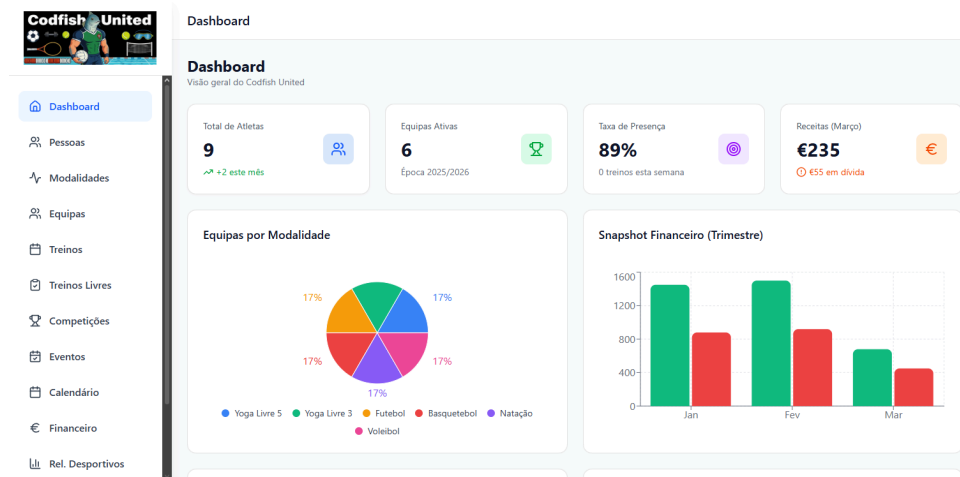


Figura 19: Dashboard Executivo da Gerência. Materializa os requisitos R26 e R46.

2.8.2. Avaliação Heurística de Usabilidade (10 Princípios de Nielsen)

Para assegurar a qualidade ergonómica da interface e garantir que esta reflete os padrões da indústria, o protótipo foi sujeito a uma auditoria visual focada em pormenores de interação. Abaixo, evidenciam-se os 10 princípios de Jakob Nielsen aplicados aos componentes da *Single Page Application*:

1. Visibilidade do Estado do Sistema

O *frontend* fornece *feedback* visual imediato. A utilização de *Toasts* (notificações flutuantes) informa o utilizador sobre o sucesso da transação em tempo real.

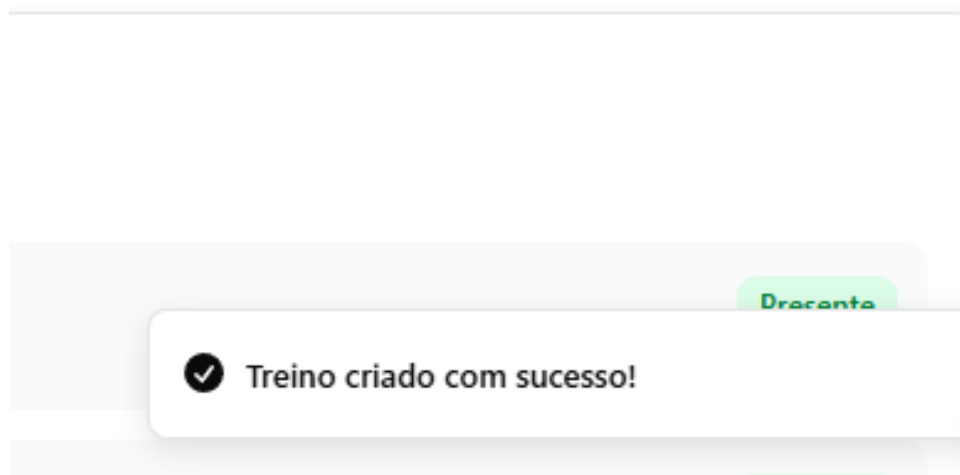
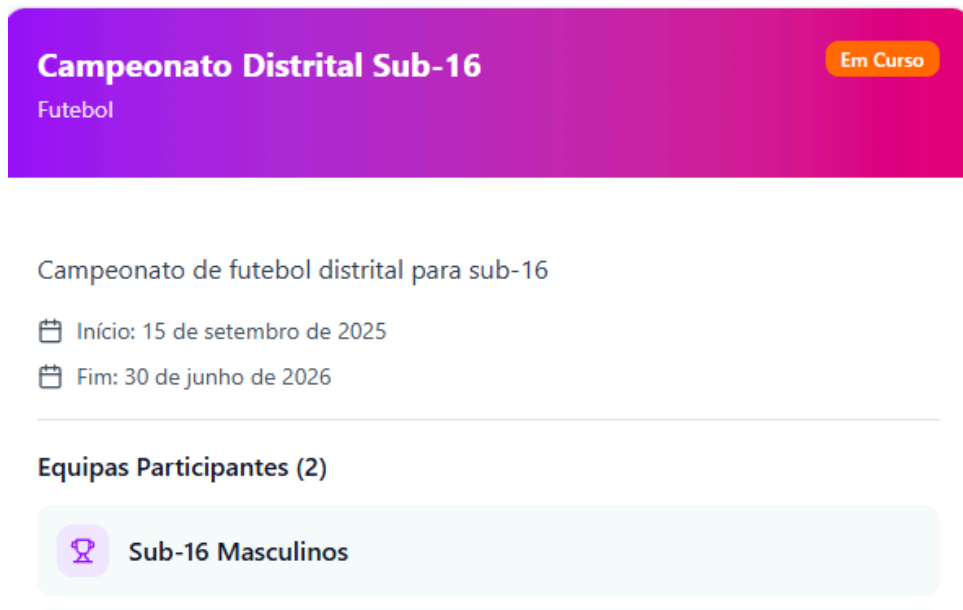


Figura 20: Notificação de sistema.

2. Correspondência com o Mundo Real

A arquitetura substitui terminologia de base de dados por léxico do domínio desportivo (“Campeonato Distrital”, “Em Curso”), alinhando-se com o modelo mental do utilizador.

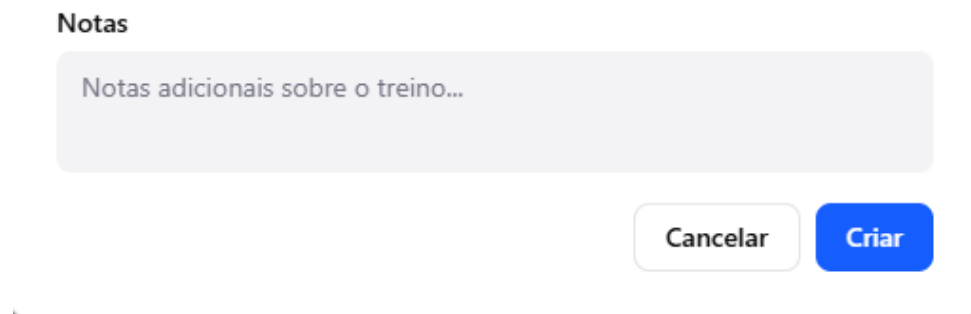


The screenshot shows a web interface for a football tournament. At the top, there is a purple header bar with the text "Campeonato Distrital Sub-16" in white, and a smaller orange button labeled "Em Curso" to its right. Below the header, the word "Futebol" is displayed. The main content area has a light gray background and contains the following elements: the title "Campeonato de futebol distrital para sub-16", two date entries with calendar icons ("Início: 15 de setembro de 2025" and "Fim: 30 de junho de 2026"), a section header "Equipas Participantes (2)", and a single participant entry "Sub-16 Masculinos" with a purple trophy icon.

Figura 21: Terminologia adaptada ao negócio.

3. Controlo e Liberdade do Utilizador

Em todas as vistas de formulário, são disponibilizadas “saídas de emergência” explícitas (botão “Cancelar”) para permitir a reversão de fluxos iniciados por engano.



The screenshot shows a form with a section titled "Notas" in bold. Below the title is a light gray text input field containing the placeholder text "Notas adicionais sobre o treino...". At the bottom right of the form, there are two buttons: a white button with a gray border labeled "Cancelar" and a solid blue button labeled "Criar".

Figura 22: Botões de ação e cancelamento contíguos.

4. Consistência e Padrões

A interface segue convenções rigorosas no desenho de componentes. Os cartões informativos utilizam os mesmos identificadores de cor e tipografia em toda a plataforma.

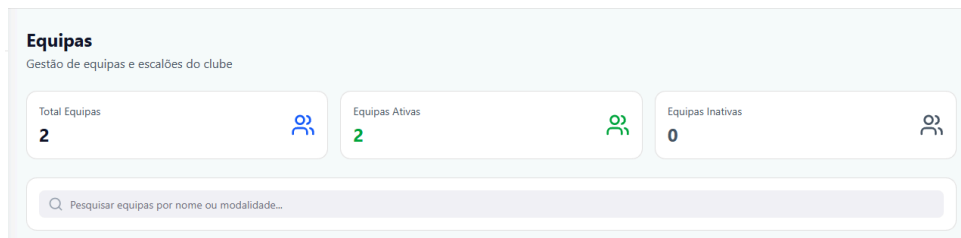


Figura 23: Consistência visual nos cartões de domínio.

5. Prevenção de Erros

A interface previne falhas de autorização omitindo ativamente rotas proibidas. A barra de navegação adapta-se ao contexto, impedindo acessos indevidos logo na navegação.

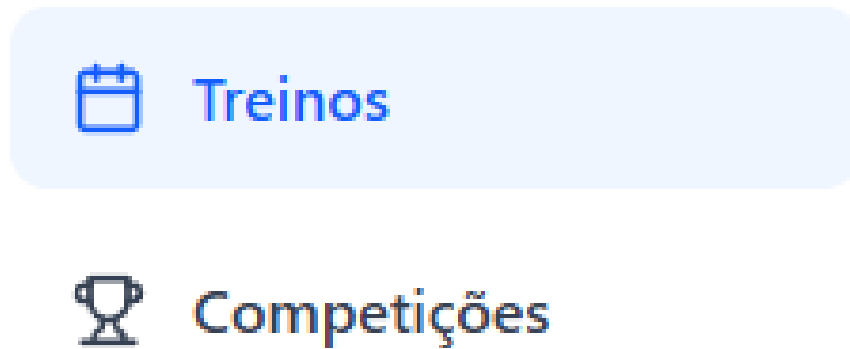


Figura 24: Omissão de menus (RBAC) para prevenção de erros.

6. Reconhecimento em vez de Lembrança

Os campos de *input* apresentam *placeholders* descritivos e exemplos de preenchimento, reduzindo a carga cognitiva ao não obrigar o utilizador a memorizar formatos.

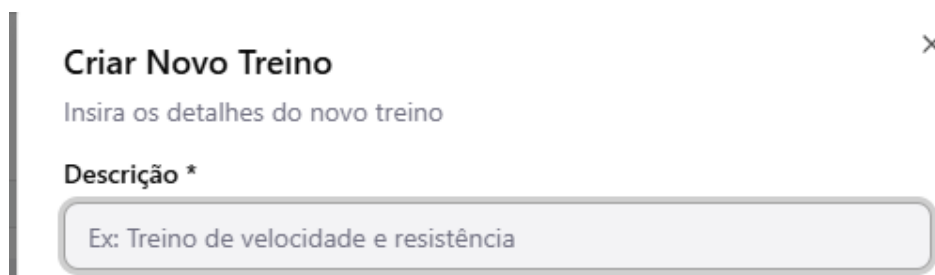


Figura 25: Instruções embebidas em campos de texto.

7. Flexibilidade e Eficiência de Utilização

Para suportar fluxos intensivos de terreno (Treinador), foram introduzidos aceleradores. A funcionalidade de marcação em lote permite submeter presenças massivas rapidamente.



Figura 26: Acelerador de interface para marcação em lote.

8. Estética e Design Minimalista

Painéis administrativos e listagens removem elementos decorativos, maximizando o espaço útil para focar a atenção exclusivamente nos KPIs e dados estruturais.

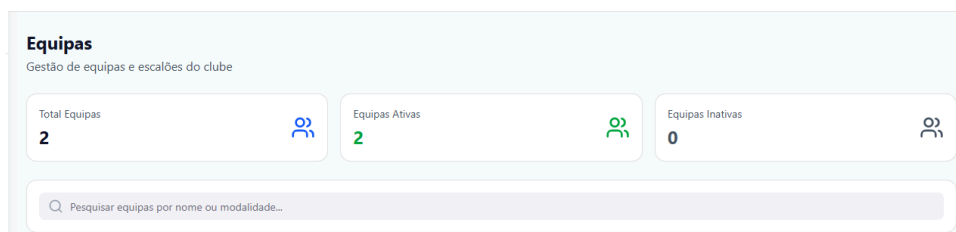


Figura 27: Ausência de ruído visual na vista de Gestão.

9. Reconhecer, Diagnosticar e Recuperar de Erros

Em caso de falha de validação, a interface apresenta mensagens de erro humanizadas e diretamente afixadas ao campo problemático (“Preencha este campo”), facilitando a correção imediata.

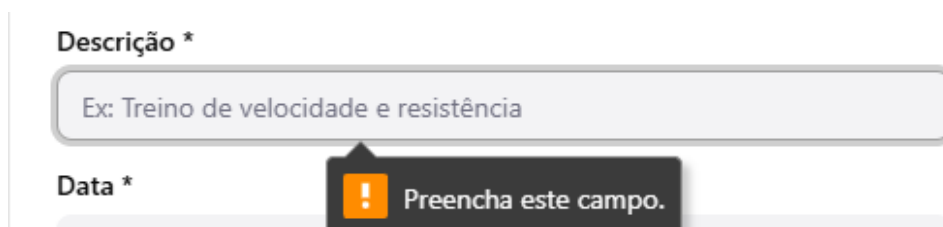


Figura 28: Mensagem de erro semântica e localizada.

10. Ajuda e Documentação

Em vez de forçar o recurso a manuais, a interface utiliza texto de suporte contextual diretamente por baixo dos campos complexos, prestando assistência no momento da ação.

Resultado

Ex: V, E, D, 1º, 2º, 3º...

V (Vitória), E (Empate), D (Derrota) ou posições (1º, 2º, 3º, 4º...)

Resultado Numérico

Ex: 2-3, 15-10, 85-72...

Resultado numérico do jogo/prova (opcional)

Figura 29: Assistência contextual embutida no formulário.

2.8.3. Exemplo Prático: Refinamento Iterativo (Subsistema Financeiro)

A transposição das regras de negócio do *backend* para a interface gráfica do *frontend* exigiu uma abordagem de prototipagem faseada. Em vez de delegar a criação visual de forma abstrata à IA, a equipa utilizou técnicas de *Prompting* Estruturado, fornecendo as entidades de domínio exatas, as regras de controlo de acessos (RBAC) e as restrições do *Design System* existente.

O exemplo seguinte ilustra o fluxo iterativo utilizado para desenhar a página do Subsistema Financeiro (/Financeiro), demonstrando como a equipa dividiu a tarefa em duas etapas: primeiro a fundação estrutural, seguida do refinamento visual (*polish*).

Prompt utilizado:

Prompt Iteração 1 (Estrutura e Lógica): Quero redesenhar a página /Financeiro mantendo a mesma *app shell* e *design system* da nossa aplicação React. **Contexto Funcional:** A página deve gerir as entidades Quota (estado: PENDENTE, PAGO, DIVIDA) e Pagamento (metodo, canal, estado). **Perfis e permissões:** A UI deve refletir o perfil do utilizador. GERENTE/COLABORADOR veem uma página operacional (KPI cards de totais, filtros, tabelas de gestão). O ATLETA vê uma vista simplificada apenas com as suas quotas pendentes e histórico, sem acesso a ações administrativas. **Output:** Layout completo e estrutura da informação, sem aprofundar modais nesta fase.

Prompt Iteração 2 (Refinamento e Polish): Agora faz o *polish* final da página /Financeiro, sem alterar a arquitetura principal já construída. Quero uma versão refinada, consistente e pronta para parecer produção real. Evita um visual de “checkout de e-commerce”; privilegia um estilo de *dashboard* administrativo (sóbrio e profissional). Melhora a hierarquia visual, clareza funcional e os *loading skeletons*.

— **Análise e Impacto da Resposta:** Esta abordagem iterativa foi crucial para o sucesso da interface. O primeiro *prompt* garantiu que o modelo não inventava dados fictícios, consumindo exatamente os DTOs projetados no Spring Boot e aplicando dinamicamente a segregação de ecrãs entre a Tesouraria e o Atleta. O segundo *prompt* atuou como uma revisão de *Design* (UI/UX), forçando a IA a abandonar

tendências visuais genéricas em prol de um *layout* utilitário, consistente com um *software* de gestão desportiva real.

2.8.4. Nota sobre a Iteração e Esforço na Prototipagem

De salientar que a arquitetura e os protótipos finais apresentados não resultaram de respostas diretas ou únicas (*zero-shot*) por parte do Modelo de Linguagem, tal como é mencionado no início do documento. A fase de prototipagem exigiu um processo intensivo e altamente iterativo de validação cruzada, ajustes arquiteturais e reescrita de requisitos. Estima-se que a equipa tenha submetido e refinado cerca de 100 *prompts* distintos apenas durante esta etapa de desenho e prototipagem, sublinhando o papel da IA não como um gerador automático de soluções fechadas, mas sim como uma ferramenta de trabalho colaborativa que exige orientação, contexto e escrutínio humano contínuo para atingir o rigor exigido num sistema de nível empresarial.

2.9. Especificação da API e as decisões arquiteturais tomadas

A arquitetura de comunicação do Codfish United assenta numa API RESTful estrita, desenhada sob o princípio de “API-First”. Esta interface funciona como o contrato imutável entre a *Single Page Application* (React) e os sete subsistemas de domínio.

2.9.1. Convenções Gerais e Padrões de Contrato

Para garantir a consistência transversal aos 50 requisitos implementados, estabeleceram-se diretrizes técnicas rigorosas que guiam todos os contratos da API:

- **Isolamento de Persistência:** – objeto de base de dados é exposto. A API comunica exclusivamente via DTOs, garantindo que alterações no esquema SQL não quebrem o contrato com o *frontend*.
- **Separação de Preocupações (Request vs Response):** Adotou-se a prática de separar DTOs de entrada e saída. Isto permite que os modelos de escrita (Requests) contenham apenas campos editáveis e validações, enquanto os modelos de leitura (Responses) incluam metadados, IDs e campos computados.
- **Inferência de Atores:** O parâmetro *by* (quem executa a ação) é omitido nos *payloads* JSON. O sistema extrai a identidade do utilizador a partir do contexto de segurança do pedido (JWT), impedindo ataques de falsificação de identidade (*ID Spoofing*).
- **Normalização via Enums:** Estados críticos como *EstadoPagamento* ou *Role* são validados via tipos enumerados (pasta *ENUMs*), forçando a integridade dos dados na fronteira da API.

2.9.2. Especificação Detalhada por Subsistema

Abaixo detalham-se as interfaces e decisões de *design* tomadas para cada um dos sete módulos funcionais, evidenciando a separação de DTOs e as responsabilidades de cada rota.

2.9.3. Subsistema 1: Identity (Gestão de Pessoal)

Este módulo gere o ciclo de vida das pessoas e a segurança. A decisão principal foi separar a gestão demográfica (Pessoa) da gestão de papéis (Roles).

Operação	Endpoint	Modelo (Req / Res)
Obter dados de perfil	GET /api/profile/{personId}	— / ProfileDataResponse
Listar pessoas	GET /api/people	PersonFilter / List
Obter pessoa por ID	GET /api/people/{personId}	— / PersonResponse
Criar pessoa	POST /api/people	CreatePersonRequest / PersonResponse
Atualizar pessoa	PUT /api/people/{personId}	UpdatePersonRequest / —
Alterar password	POST /api/people/{personId}/password	AlterPasswordRequest / AlterPasswordResponse
Definir password por staff	POST /api/people/{personId}/password/reset	SetPasswordByStaffRequest / —
Listar roles ativas	GET /api/people/{personId}/roles	on (opcional) / List
Atribuir role	POST /api/people/{personId}/roles	AssignRoleRequest / PersonRoleResponse
Terminar role	POST /api/person-roles/{personRoleId}/termination	TerminateRoleRequest / —
Definir role como principal	POST /api/person-roles/{personRoleId}/primary	— / —
Verificar role ativa	GET /api/people/{personId}/roles/{role}/active	on (opcional) / BooleanResponse
Listar notificações	GET /api/people/{personId}/notifications	read (opcional) / List
Marcar notificação como lida	POST /api/notifications/{notificationId}/read	— / —
Remover notificação	DELETE /api/notifications/{notificationId}	— / —

2.9.4. Subsistema 2: Sportscore (Gestão de Modalidades)

Módulo de parametrização. Utiliza PUT para atualizações por substituição total de agregados (modalidades e preços).

Operação	Endpoint	Modelo (Req / Res)
Listar modalidades de um treinador	GET /api/coaches/{coachId}/modalities	— / List
Listar modalidades	GET /api/modalities	ModalityFilter / List
Obter modalidade	GET /api/modalities/{modalityId}	— / ModalityResponse
Criar modalidade	POST /api/modalities	CreateModalityRequest / ModalityResponse
Atualizar modalidade	PUT /api/modalities/{modalityId}	UpdateModalityRequest / —
Eliminar modalidade	DELETE /api/modalities/{modalityId}	— / —
Listar tipos de estatística de uma modalidade	GET /api/modalities/{modalityId}/statistic-types	— / List

Operação	Endpoint	Modelo (Req / Res)
Listar preços de uma modalidade	GET /api/modalities/{modalityId}/prices	– / List
Calcular preço de inscrição	GET /api/modalities/{modalityId}/registration-price	birthdate (query) / Calculated-PriceResponse
Calcular mensalidade	GET /api/modalities/{modalityId}/monthly-fee	birthdate (query) / Calculated-PriceResponse
Listar tipos de estatística	GET /api/statistic-types	– / List
Criar tipo de estatística	POST /api/statistic-types	CreateStatisticTypeRequest / StatisticTypeResponse
Atualizar tipo de estatística	PUT /api/statistic-types/{statisticTypeId}	UpdateStatisticTypeRequest / –
Eliminar tipo de estatística	DELETE /api/statistic-types/{statisticTypeId}	– / –
Listar complexos	GET /api/complexes	– / List
Obter complexo	GET /api/complexes/{complexId}	– / ComplexResponse
Criar complexo	POST /api/complexes	CreateComplexRequest / ComplexResponse
Atualizar complexo	PUT /api/complexes/{complexId}	UpdateComplexRequest / –
Eliminar complexo	DELETE /api/complexes/{complexId}	– / –
Listar equipas de uma modalidade	GET /api/modalities/{modalityId}/teams	– / List

2.9.5. Subsistema 3: Teams (Gestão de Equipas)

Gere os vínculos temporais entre pessoas e equipas. A API introduz o EquipaResumoResponse para otimizar listagens, evitando o carregamento pesado de plantéis inteiros.

Operação	Endpoint	Modelo (Req / Res)
Listar equipas	GET /api/teams	TeamFilter / List
Obter equipa	GET /api/teams/{teamId}	– / TeamResponse
Listar equipas de um treinador	GET /api/coaches/{coachId}/teams	TeamFilter / List
Listar equipas de um atleta	GET /api/athletes/{athleteId}/teams	TeamFilter / List
Criar equipa	POST /api/teams	CreateTeamRequest / TeamResponse
Atualizar equipa	PUT /api/teams/{teamId}	UpdateTeamRequest / –
Adicionar atleta à equipa	POST /api/teams/{teamId}/athletes	AddTeamMemberRequest / TeamMemberResponse
Adicionar treinador à equipa	POST /api/teams/{teamId}/coaches	AddTeamMemberRequest / TeamMemberResponse
Listar atletas numa data	GET /api/teams/{teamId}/athletes	at (opcional) / List
Listar treinadores numa data	GET /api/teams/{teamId}/coaches	at (opcional) / List

Operação	Endpoint	Modelo (Req / Res)
Terminar vínculo de membro	POST /api/team-members/{team-MemberId}/termination	EndMembershipRequest / –

2.9.6. Subsistema 4: Activities (Planeamento)

Gere competições, eventos e treinos. Optou-se por uma política de *Hard Delete* para eventos, simplificando a limpeza de agendamentos obsoletos.

Operação	Endpoint	Modelo (Req / Res)
Listar equipas	GET /api/teams	TeamFilter / List
Obter equipa	GET /api/teams/{teamId}	– / TeamResponse
Listar equipas de um treinador	GET /api/coaches/{coachId}/teams	TeamFilter / List
Listar equipas de um atleta	GET /api/athletes/{athleteId}/teams	TeamFilter / List
Criar equipa	POST /api/teams	CreateTeamRequest / TeamResponse
Atualizar equipa	PUT /api/teams/{teamId}	UpdateTeamRequest / –
Adicionar atleta à equipa	POST /api/teams/{teamId}/athletes	AddTeamMemberRequest / TeamMemberResponse
Adicionar treinador à equipa	POST /api/teams/{teamId}/coaches	AddTeamMemberRequest / TeamMemberResponse
Listar atletas numa data	GET /api/teams/{teamId}/athletes	at (opcional) / List
Listar treinadores numa data	GET /api/teams/{teamId}/coaches	at (opcional) / List
Terminar vínculo de membro	POST /api/team-members/{team-MemberId}/termination	EndMembershipRequest / –

2.9.7. Subsistema 5: Activity Tracking (Terreno)

Desenhado para registos massivos (lotes). Implementa o padrão *Upsert* no endpoint de desempenhos, permitindo que o treinador corrija avaliações sem gerar duplicados na BD.

Operação	Endpoint	Modelo (Req / Res)
Listar presenças de treino	GET /api/trainings/{trainingId}/attendances	– / List
Listar presenças de evento	GET /api/events/{eventId}/attendances	– / List
Listar presenças semanais de treino livre	GET /api/teams/{teamId}/free-trainings/attendances	on (opcional) / List
Registar ou atualizar presença	POST /api/attendances	RegisterOrUpdateAttendanceRequest / AttendanceResponse
Listar performances de treino	GET /api/trainings/{trainingId}/performances	– / List

Operação	Endpoint	Modelo (Req / Res)
Listar performances de evento	GET /api/events/{eventId}/performances	– / List
Registrar ou atualizar performances	POST /api/performances	RegisterOrUpdatePerformancesRequest / –
Calcular estatísticas de presenças	GET /api/statistics/attendances	– / AttendanceStatisticsResponse
Calcular estatísticas de presenças de um treinador	GET /api/coaches/{coachId}/statistics/attendances	– / AttendanceStatisticsResponse
Registrar ou atualizar presença de treino livre	POST /api/attendances/free-trainings	RegisterOrUpdateAttendanceRequest / AttendanceResponse
Listar presenças de treino livre	GET /api/attendances/free-trainings	teamId, date (query) / List

2.9.8. Subsistema 6: Finance (Contabilidade)

Focado na robustez financeira e integrações. O endpoint de *webhooks* é estritamente idempotente.

Operação	Endpoint	Modelo (Req / Res)
Listar mensalidades	GET /api/fees	FeeFilterQuery / List
Listar mensalidades de um atleta	GET /api/athletes/{athleteId}/fees	FeeFilterQuery / List
Listar dívidas de um atleta	GET /api/athletes/{athleteId}/debts	– / List
Listar pagamentos	GET /api/payments	– / List
Listar pagamentos de um atleta	GET /api/athletes/{athleteId}/payments	– / List
Registrar pagamento em dinheiro	POST /api/payments/cash	RegisterCashPaymentRequest / PaymentResponse
Registrar pagamento Multibanco presencial	POST /api/payments/multibanco/in-person	RegisterMultibancoPresencialRequest / PaymentResponse
Iniciar pagamento Multibanco online	POST /api/payments/multibanco/online	StartMultibancoOnlineRequest / StartMultibancoOnlineResponse
Obter dados de pagamento Multibanco	GET /api/payments/{paymentId}/multibanco	– / MultibancoPaymentDataResponse
Obter estado do pagamento	GET /api/payments/{paymentId}/status	– / PaymentStatusResponse
Cancelar pagamento pendente	POST /api/payments/{paymentId}/cancellation	PaymentVersionRequest / –
Obter mensalidade associada ao pagamento	GET /api/payments/{paymentId}/fee	– / FeeResponse
Obter pagamento associado à mensalidade	GET /api/fees/{feeId}/payment	– / PaymentResponse
Atualizar pagamento presencial	PUT /api/payments/{paymentId}/in-person	UpdateInPersonPaymentRequest / PaymentResponse

Operação	Endpoint	Modelo (Req / Res)
Eliminar pagamento presencial	DELETE /api/payments/{paymentId}/in-person	PaymentVersionRequest / –
Processar notificação do gateway	POST /api/gateway/notifications	GatewayNotificationRequest / –

2.9.9. Subsistema 7: Analytics (Dashboards)

Subsistema puramente consultivo (Read-Only). Agrega dados de todos os módulos para fornecer visões executivas (KPIs) e o perfil do atleta sem necessitar de parâmetros ID, inferindo o utilizador via JWT.

Operação	Endpoint	Modelo (Req / Res)
Calcular dashboard	GET /api/dashboard	– / DashboardResponse
Calcular relatório desportivo	GET /api/reports/sport	– / SportReportResponse
Calcular relatório financeiro	GET /api/reports/financial	– / FinancialReportResponse
Obter estatísticas de perfil	GET /api/profile/{personId}/statistics	– / ProfileStatisticsResponse
Obter atividades do calendário	GET /api/calendar	month, year, personId, view (query) / List

Prompt utilizado:

Atua como Arquiteto Backend. Analisa os nossos ficheiros de especificação REST (SS1 a SS7).

1. Justifica a decisão de manter o subsistema SS7 como Read-Only, explicando como isso beneficia a performance dos dashboards.
2. Descreve o mecanismo de tratamento de erros global que converte exceções de negócio (ex: violação de regra XOR) em respostas HTTP 400 amigáveis.
3. Gera a API completa com base nos pontos anteriores.

2.10. Modelo Físico de Dados e Persistência

A persistência de dados do sistema Codfish United assenta num modelo relacional implementado em PostgreSQL. A arquitetura de dados foi desenhada para garantir a integridade referencial, o suporte a transações ACID e a conformidade com as exigências de auditoria (RGPD), incorporando campos de rastreabilidade temporal (`created_at`, `updated_at`, `updated_by`) em todas as tabelas de domínio.

A conversão do Modelo de Domínio para o modelo físico resultou na criação de 21 tabelas e 11 tipos estritos (ENUMs como `role_enum`, `estado_quota_enum`) que forçam a integridade dos dados diretamente no motor de base de dados. A segregação de dados reflete a arquitetura de Monólito Modular: o núcleo demográfico encontra-se na tabela `pessoa`, a hierarquia desportiva em tabelas como `modalidade` e `equipa`, e o motor financeiro assenta nas tabelas `quota` e `notificacao_gateway` (recorrendo a restrições `UNIQUE` para garantir a idempotência transacional no processamento de *webhooks*).

Abaixo apresenta-se o dicionário de dados detalhado, derivado do esquema físico, contemplando as principais entidades relacionais do sistema, as suas cardinalidades e restrições.

E01 - Pessoa

- **id** (PK)
- **nome** (Varchar)
- **sexo** (Enum)
- **email** (Varchar, UNIQUE)
- **telemovel** (Varchar, NULLABLE)
- **data_nascimento** (Date, NULLABLE)
- **password_hash** (Varchar, NULLABLE)
- **ativo** (Boolean)
- **created_at** (Timestamp)
- **updated_by** (FK → E01, NULLABLE)

E02 - Modalidade

- **id** (PK)
- **nome** (Varchar)
- **tipo_evento** (Varchar, NULLABLE)
- **descricao** (Varchar, NULLABLE)
- **treinada** (Boolean)
- **max_presencas_semana** (Integer, NULLABLE)
- **Restrição:** max_presencas_semana >= 0

E03 - TipoEstatistica

- **id** (PK)
- **nome** (Varchar)
- **unidade** (Varchar, NULLABLE)
- **obrigatorio** (Boolean)

E04 - Equipa

- **id** (PK)
- **nome** (Varchar)
- **tipo** (Enum: Individual/Coletiva)
- **epoca_desportiva** (Varchar, NULLABLE)
- **ativa** (Boolean)
- **id_modalidade** (FK → E02)

E05 - Competicao

- **id** (PK)
- **nome** (Varchar)
- **data_inicio** (Date)
- **data_fim** (Date, NULLABLE)
- **taxa_inscricao** (Decimal, NULLABLE)
- **id_modalidade** (FK → E02)
- **Restrição:** data_fim >= data_inicio

E06 - Evento

- **id** (PK)

E12 - Pessoa_Role

- **id** (PK)
- **role** (Enum: Gerente/Treinador/...)
- **data_inicio** (Timestamp)
- **data_fim** (Timestamp, NULLABLE)
- **id_pessoa** (FK → E01)

E13 - Modalidade_TipoEstatistica

- **id_modalidade** (PK, FK → E02)
- **id_tipoEstatistica** (PK, FK → E03)

E14 - Equipa_Pessoa

- **id** (PK)
- **relacao** (Enum: Atleta/Treinador)
- **data_inicio** (Timestamp)
- **data_fim** (Timestamp, NULLABLE)
- **id_equipa** (FK → E04)
- **id_pessoa** (FK → E01)
- **UNIQUE:** (id_equipa, id_pessoa, relacao, data_inicio)

E15 - Presenca

- **id** (PK)
- **presente** (Boolean)
- **id_atleta** (FK → E01)
- **id_treino** (FK → E07, NULLABLE)
- **id_evento** (FK → E06, NULLABLE)
- **Restrição XOR:** Check Apenas Treino OU Evento

E16 - Desempenho

- **id** (PK)
- **valor_numerico** (Decimal, NULLABLE)
- **id_atleta** (FK → E01)
- **id_tipo_estatistica** (FK → E03)
- **id_treino** (FK → E07, NULLABLE)
- **id_evento** (FK → E06, NULLABLE)

E17 - Competicao_Equipa

- **id** (PK)
- **nota** (NULLABLE)
- **resultado_final** (NULLABLE)
- **resultado_pontos** (NULLABLE)
- **id_competicao** (FK → E05)
- **id_equipa** (FK → E04)
- **UNIQUE:** (id_competicao, id_equipa)

- **descricao** (Varchar)
- **data_evento** (Timestamp)
- **duracao_minutos** (Integer)
- **id_modalidade** (FK → E02)
- **id_competicao** (FK → E05, NULLABLE)
- **Restrição:** duracao_minutos > 0

E07 - Treino

- **id** (PK)
- **descricao** (Varchar)
- **data_treino** (Timestamp)
- **duracao_minutos** (Integer)
- **id_equipa** (FK → E04)
- **id_complexo** (FK → E08, NULLABLE)

E08 - ComplexoDesportivo

- **id** (PK)
- **nome** (Varchar)
- **morada** (Varchar, NULLABLE)

E09 - Quota

- **id** (PK)
- **tipo** (Enum: Inscrição/Mensalidade)
- **valor_base** (Decimal)
- **data_emissao** (Date)
- **data_vencimento** (Date)
- **estado** (Enum: Pendente/Pago/Dívida)
- **id_atleta** (FK → E01)
- **Restrição:** valor_base >= 0

E10 - Pagamento

- **id** (PK)
- **metodo** (Enum: Dinheiro/MB)
- **canal** (Enum: Presencial/Online)
- **valor_pago** (Decimal, NULLABLE)
- **estado** (Enum: Pendente/Confirmado)
- **id_quota** (FK → E09, UNIQUE)

E11 - Notificacao_Gateway

- **id** (PK)
- **gateway** (Varchar)
- **id_transacao_externa** (Varchar, UNIQUE)
- **estado_externo** (Varchar)
- **processada** (Boolean)
- **id_pagamento** (FK → E10)

E18 - Preco_Modalidade

- **id** (PK)
- **taxa_inscricao**(NULLABLE)
- **valor_mensalidade** (NULLABLE)
- **faixa_etaria_min**(NULLABLE)
- **faixa_etaria_max** (NULLABLE)
- **id_modalidade** (FK → E02)

E19 - Evento_Equipa

- **id** (PK)
- **resultado** (NULLABLE)
- **resultado_numerico** (NULLABLE)
- **id_evento** (FK → E06)
- **id_equipa** (FK → E04)
- **UNIQUE:** (id_evento, id_equipa)

E20 - Notificacao

- **id** (PK)
- **tipo**
- **conteudo**
- **lida**
- **id_atleta** (FK → E01)

E21 - Tentativa_Pagamento_Gateway

- **id** (PK)
- **gateway**
- **id_pagamento_externo** (NULLABLE)
- **data_criacao**
- **sucesso**
- **detalhe_erro** (NULLABLE, TEXT)
- **id_pagamento** (FK → E11)

Prompt utilizado:

Atua como Arquiteto de Dados. Analisa o nosso Modelo Físico de Dados implementado em PostgreSQL.

1. Explica como a utilização de tipos ENUM (ex: `role_enum`, `estado_quota_enum`) e restrições UNIQUE (ex: em `id_transacao_externa`) contribui para a integridade dos dados e a idempotência transacional no processamento de webhooks.
2. Justifica a decisão de incluir campos de rastreabilidade temporal (`created_at`, `updated_at`, `updated_by`) em todas as tabelas de domínio, relacionando esta escolha com os requisitos de auditoria do RGPD.
3. Descreve como a restrição XOR implementada na tabela `Presenca` (através do CHECK que valida `id_treino XOR id_evento`) garante a consistência semântica do modelo, e como o sistema deve tratar violações desta regra ao nível da aplicação.

3. Desenvolvimento Incremental e Integração de LLMs

O desenvolvimento do *software* seguiu uma estratégia de implementação incremental, abordando a lógica subsistema a subsistema. Desde o início, a equipa adotou deliberadamente a Inteligência Artificial Generativa como um elemento central do fluxo de trabalho de engenharia — não como um substituto do raciocínio técnico, mas como um acelerador da transposição dos diagramas UML, protótipos de ecrã e da especificação REST para código funcional.

Esta abordagem caracteriza-se como *Human-in-the-Loop Development*: o LLM atuava como executor de código estrutural e rotineiro (*boilerplate*), enquanto a equipa retinha a responsabilidade de definir as restrições arquiteturais, validar a correção semântica e tomar as decisões de *design* que exigem compreensão aprofundada do domínio desportivo. — excerto de código foi integrado sem um ciclo rigoroso de revisão e adaptação manual.

3.1. Estratégia de Gestão de Contexto (Prompt Engineering)

Para mitigar o risco de “alucinações” e garantir que o código gerado respeitava a arquitetura do projeto, a equipa aplicou uma técnica rigorosa de ancoragem de contexto.

Em todos os *prompts* de implementação, o ficheiro de documentação `package-structure.md` (detalhando a árvore de diretórios isolada do *Frontend* e do *Backend*) era fornecido como anexo obrigatório. Adicionalmente, as instruções operacionais transmitidas ao LLM incluíam uma diretiva explícita de comportamento assistente: “*Antes de tomar qualquer decisão de design arquitetural ou de gerar/corrigir lógicas complexas, analisa a estrutura e solicita os ficheiros de código-fonte que consideres necessários para compreender as dependências*”.

Esta instrução evitou que o modelo assumisse importações inexistentes ou violasse as fronteiras dos 7 subsistemas lógicos, assegurando que as sugestões eram cirúrgicas e contextualmente precisas.

3.2. Quantificação do Impacto do LLM (Backend vs. Frontend)

Para tornar tangível o contributo da IA no ciclo de desenvolvimento, a equipa realizou uma estimativa da distribuição de esforço por camada arquitetural, separando as bases de código do Servidor (Spring Boot) e do Cliente (React). A metodologia baseou-se na classificação de cada bloco de código segundo a sua origem predominante: gerado por LLM com edições mínimas, gerado por LLM com refatoração significativa, ou escrito estritamente de raiz pela equipa.

3.2.1. Impacto no Servidor (Backend - Spring Boot)

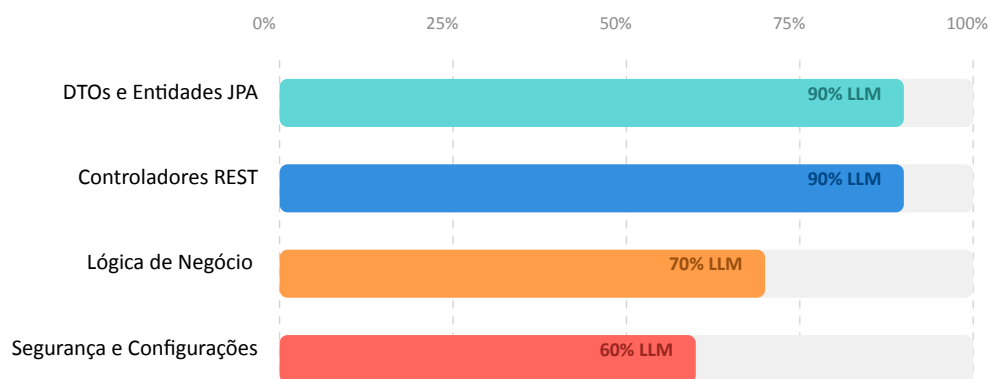


Figura 30: Distribuição estimada do esforço de código gerado por LLM na API (Spring Boot).

Os resultados no *Backend* revelam um padrão ditado pela previsibilidade da camada:

- **DTOs e Entidades JPA (90% LLM):** Tratando-se de estruturas de dados repetitivas, o LLM transcreveu o diagrama de classes quase de forma perfeita. A intervenção humana limitou-se à correção pontual de anotações de cardinalidade (`@OneToMany`) ou *fetch types*.
- **Controladores REST (90% LLM):** Sendo contratos estritos definidos por anotações padrão (`@PostMapping`, `@RequestBody`), foram amplamente automatizados. A equipa apenas ajustou as restrições de injeção de dependência.
- **Lógica de Negócio e Facades (70% LLM):** Esta fase já requiriu maior esforço humano. A validação de regras de treinos ou a orquestração transacional de pagamentos (SS6) requerem um rigor que o LLM não domina de forma isolada, servindo apenas para criar a “espinha dorsal” dos métodos.
- **Segurança e Configurações (60% LLM):** Filtros JWT, configurações CORS e a mecânica de *Role-Based Access Control* (RBAC) foram programados predominantemente à mão, devido à criticidade e ao risco de vulnerabilidades silenciosas no tratamento de sessões.

3.2.2. Impacto no Cliente (Frontend - React)

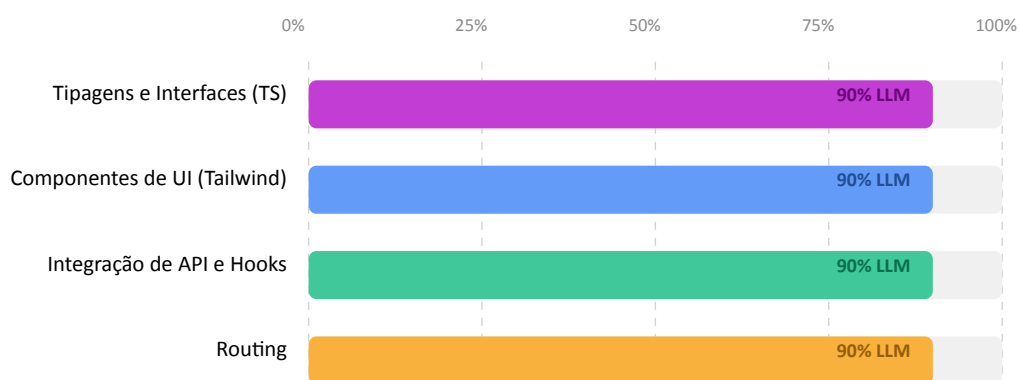


Figura 31: Distribuição estimada do esforço de código gerado por LLM na UI (React).

No ecossistema da *Single Page Application*, a assistência distribuiu-se da seguinte forma:

- **Tipagens e Interfaces TS (90% LLM):** A IA demonstrou eficácia a converter os DTOs em Java diretamente para interfaces rigorosas em TypeScript, garantindo o alinhamento de contratos sem atrito.

- **Componentes de UI e Tailwind (90% LLM):** Através de *prompts* de design estruturado, a IA gerou os esqueletos dos ecrãs, tabelas e *cards* com as classes Tailwind necessárias. A equipa dedicou o seu esforço estritamente ao refinamento visual (*polish*).
- **Integração de API e Custom Hooks (90% LLM):** A criação da base dos ficheiros de comunicação via Fetch (ex: `api/teams.ts`) e o mapeamento das *Promises* foi delegada à IA. O toque humano assegurou a correção do que foi gerado.
- **Roteamento (90% LLM):** À semelhança dos anteriores, a Navegabilidade (Rotas) foram bem geradas pelo LLM e o toque humano assegurou a correção do que foi gerado.

3.3. Engenharia de Prompts no Frontend: Construção Iterativa (Figma -> React)

Embora a ferramenta **Figma Make** tenha acelerado drasticamente a fase de prototipagem visual (como documentado no Capítulo 2), o código gerado automaticamente revelou-se estruturalmente inviável para um ambiente de produção. O código apresentava uma ausência total de separação de responsabilidades (lógica misturada com UI), estilos em linha arbitrários, dependência de dados estáticos (*mock data*) e –a preparação para consumo de APIs REST.

Para aproveitar o *design* dos componentes sem herdar a dívida técnica gerada pelo Figma, a equipa optou por fazer o frontend de raiz, de forma assistida por LLM. Em vez de *prompts ad-hoc*, foi desenvolvido um **Master Prompt** (um contrato estrito de instruções) que definia a *persona* da IA como um engenheiro *frontend* sénior especializado na *stack* tecnológica do projeto.

Este sistema de instruções baseava-se na submissão de iterações modulares, onde o modelo recebia o código desorganizado do Figma e os *endpoints* do *backend*, e era forçado a devolver código limpo e arquiteturalmente correto.

Prompt utilizado:

Master Prompt (Instruções de Sistema) És um *senior frontend engineer* especializado em React, Vite, TypeScript e Tailwind CSS. O teu objetivo é transformar código desorganizado gerado pelo Figma Make em código limpo e modular.

Regras de Linguagem: O diálogo é em português de Portugal, mas o código, variáveis e comentários gerados têm de ser estritamente em inglês.

Regras de Arquitetura: Utiliza a arquitetura *feature-first* (ex: `src/features/*`). Respeita sempre a *package structure* fornecida e não crie ficheiros fora da árvore sem permissão explícita.

Formato de Output Obrigatório: O output deve conter as Decisões tomadas, a lista de ficheiros a alterar com os respetivos *paths* exatos, e o código final completo sem atalhos ou pseudocódigo.

Template de Iteração (Submetido a cada passo)

Tarefa: [Descrição do que fazer]

Package Structure Atual: [Árvore de diretórios atualizada]

Código Figma para refatorar: [Código UI desorganizado]

Endpoints REST do Backend e estruturas de dados: [Contratos de Request/Response]

Nota: Se precisares de ficheiros adicionais para teres contexto, pede-os em vez de inventares dependências

3.4. Reflexão Crítica sobre a Abordagem

A análise da distribuição de esforço evidencia uma conclusão central: **o LLM não substituiu o engenheiro, mas alterou fundamentalmente o tipo de trabalho que o engenheiro realiza.** O tempo anteriormente dedicado à escrita de código estrutural e repetitivo foi redirecionado para atividades de maior valor: revisão arquitetural, deteção de vulnerabilidades, orquestração de lógica transacional crítica e garantia de coerência entre os 7 subsistemas.

Esta redistribuição de esforço traduz-se numa aceleração mensurável do ciclo *design, implement, validate*, sem comprometer a qualidade do produto final — desde que a equipa mantenha um processo rigoroso de revisão do código gerado.

3.5. Engenharia de Prompts no backend: Construção iterativa (UML, Documentação das APIs, etc.. -> SpringBoot)

Para garantir que o código gerado pelo modelo respeitava a arquitetura rigorosa do projeto, adotou-se uma técnica de *Context-Rich Prompting*. Em vez de pedidos genéricos, os *prompts* forneciam o código base existente, as regras de negócio específicas e as restrições de injeção de dependências.

Os exemplos seguintes demonstram exemplos de *prompts* submetidos, começando com o *prompt* para a inicialização do módulo de agendamento de treinos no Subsistema 4:

3.5.1. Treinos SS4

Prompt utilizado:

Master Prompt (Instruções de Sistema) És um *senior backend engineer* especializado em Java, Spring Boot, Spring Security e Spring Data JPA. O teu objetivo é gerar código backend limpo, seguro e modular, respeitando a arquitetura do projeto.

Regras de Linguagem: O diálogo é em português de Portugal, mas o código, variáveis e comentários gerados têm de ser estritamente em inglês.

Regras de Arquitetura:

- Respeita a estrutura de pacotes por subsistema (ex: `com.clubedesportivo.{subsistema}.*`).
- Cada subsistema expõe exclusivamente uma interface Facade pública (ex: `ISS4ActivitiesFacade`), diretamente aos repositórios de outro módulo.
- Nenhuma entidade `@Entity` deve ultrapassar a camada de serviço — usa sempre DTOs (`RequestDTO` / `ResponseDTO`) na fronteira REST.

- O parâmetro by (ator da ação) é sempre extraído do contexto de segurança via SecurityUtils, nunca do payload JSON.
- Toda a lógica de negócio com escrita na base de dados deve estar anotada com @Transactional ao nível do serviço (Facade).

Formato de Output Obrigatório: O output deve conter:

1. Decisões arquiteturais tomadas e justificação.
2. Lista de ficheiros a criar/alterar com os paths exatos.
3. Código final completo, sem atalhos nem pseudocódigo.

Template de Iteração (Submetido a cada passo)

- **Tarefa:** [Descrição da funcionalidade ou regra de negócio a implementar]
- **Package Structure Atual:** [Árvore de diretórios do subsistema relevante]
- **Código existente para contexto:** [Entidades JPA, Facade interface, repositórios já criados]
- **Documentação específica:** [Ex: validação XOR, idempotência, restrição temporal]
- **Contratos de API (Request / Response DTOs):** [Estrutura esperada dos objetos de entrada e saída]
- **Nota:** Se precisares de ficheiros adicionais para teres contexto , pede-os em vez de inventares dependências ou assumires assinaturas de métodos.

Análise do Código Resultante: O modelo compreendeu as restrições arquiteturais impostas e produziu um código que respeitou integralmente a separação entre Controladores e a injeção do contexto de segurança, prevenindo proativamente vulnerabilidades de *ID Spoofing*.

```
// Excerto gerado via LLM: SS4Controller.java (Camada REST)
@RestController
@RequestMapping("/api/treinos")
public class SS4Controller {

    private final SS4Facade ss4Facade;

    public SS4Controller(SS4Facade ss4Facade) {
        this.ss4Facade = ss4Facade;
    }

    @PostMapping
    public ResponseEntity<TrainingResponse> createTraining(
        @Valid @RequestBody CreateTrainingRequest request) {
        // Restrição 2 respeitada: Extração segura do ID do treinador via sessão
        Integer treinadorId = SecurityUtils.getAuthenticatedPersonId();

        // Injeção limpa na Facade
        TrainingResponse response = ss4Facade.createTraining(request, treinadorId);

        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
            .path("/{id}").buildAndExpand(response.id()).toUri();

        return ResponseEntity.created(location).body(response);
    }
}
```

3.5.2. Integração Externa e Simulação (Payment Gateway Mock)

Contexto e Desafio: Para o processo de pagamentos online (SS6), a arquitetura exigia a comunicação com uma API REST externa para gerar Entidade/Referência e a recepção do respetivo *webhook* de confirmação. Para garantir a testabilidade académica (sem dependência de credenciais reais de fornecedores), optou-se por implementar um *Mock Gateway* interno. O desafio era desenhar a classe de forma genérica, para que uma futura transição para o “mundo real” exigisse apenas a alteração de variáveis de ambiente, sem refatorização de código estrutural.

Prompt utilizado:

Tarefa:

No módulo Financeiro, cria o serviço `PaymentGatewayService`. A classe deve estar preparada para comunicar com uma API de pagamentos externa, injetando o `RestTemplate` e variáveis de ambiente (`@Value`) para o URL base e API Key. Contudo, cria também um método interno que simule a resposta (Mock) gerando uma referência fictícia para efeitos de testes académicos. Usa *Records* para os DTOs.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

// Excerto: `PaymentGatewayService.java` (SS6)

```
@Service
public class PaymentGatewayService {

    private final RestTemplate restTemplate;
    private final String sandboxUrl;
    private final String entityId;
    private final String apiKey;
    private final boolean mockEnabled;

    public PaymentGatewayService(
        RestTemplate restTemplate,
        @Value("${payment-gateway.sandbox-url}") String sandboxUrl,
        @Value("${payment-gateway.entity}") String entityId,
        @Value("${payment-gateway.api-key}") String apiKey,
        @Value("${payment-gateway.mock-enabled:true}") boolean mockEnabled) {
        this.restTemplate = restTemplate;
        this.sandboxUrl = sandboxUrl;
        this.entityId = entityId;
        this.apiKey = apiKey;
        this.mockEnabled = mockEnabled;
    }

    public GatewayPaymentInfo createPayment(int amountCents, String orderId, String
description) {
        if (mockEnabled || apiKey == null || apiKey.isBlank() || entityId ==
null || entityId.isBlank()) {
```

```

        return createMockPayment(orderId);
    }

    GatewayCreateRequest request = new GatewayCreateRequest(
        entityId,
        amountCents,
        orderId,
        description);

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    headers.setBearerAuth(apiKey);

    HttpEntity<GatewayCreateRequest> entity = new HttpEntity<>(request,
headers);

    try {
        GatewayCreateResponse response = restTemplate.postForObject(
            sandboxUrl + "/multibanco/api/createPayment",
            entity,
            GatewayCreateResponse.class);

        if (response == null) {
            throw new IllegalStateException("Empty response from payment
gateway");
        }

        return new GatewayPaymentInfo(
            response.paymentId(),
            response.referenciaEntidade(),
            response.referencia(),
            parseDateTime(response.validade()));

    } catch (RestClientException ex) {
        throw new IllegalStateException("Error creating payment in
payment gateway", ex);
    }
}

```

3.6. Revisão de Código, Qualidade e Refactoring

A garantia de qualidade do código não dependeu exclusivamente de processos de revisão manual. O ciclo de desenvolvimento integrou, de forma sistemática, sessões de *pair programming* assistido por LLM, nas quais o modelo atuava como interlocutor técnico para a resolução de bloqueios arquiteturais, refatorização de lógicas complexas e verificação do cumprimento das restrições de domínio definidas na fase de requisitos. Esta secção documenta, subsistema a subsistema, os desafios concretos enfrentados, os *prompts* submetidos e a análise crítica das soluções geradas.

3.6.1. Estrutura de Pacotes e Convenção de Módulos

Para garantir a coesão interna do Monólito Organizado em Módulos(subsistemas), a estrutura de pacotes no *backend* espelha rigorosamente a separação de domínios definida na arquitetura. Cada subsistema (identificado pelo prefixo SS seguido do respetivo índice) é encapsulado num pacote independente, isolando as suas *Facades*, Controladores e Repositórios dos restantes módulos:

```
src/main/java/com/clubedesportivo/
├─ activities/          # SS4 – Treinos e Eventos
├─ activitytracking/    # SS5 – Presenças e Regra XOR
├─ analytics/          # SS7 – Dashboards e Agregações
├─ finance/            # SS6 – Pagamentos e Webhooks
├─ identity/           # SS1 – Acessos e Autorizações
├─ sportscore/         # SS2 – Modalidades
└─ teams/              # SS3 – Equipas e Vínculos
```

Esta convenção de nomenclatura foi mantida de forma consistente ao longo de todo o desenvolvimento, facilitando a navegação no repositório e a atribuição de responsabilidades entre os membros da equipa.

3.6.2. SS1 (Identity): Segurança e Segregação de Privilégios

Contexto e Desafio: O módulo de Identidade constitui a fundação de segurança de todo o sistema. A gestão de papéis (*roles*) exigia uma blindagem rigorosa contra escalada de privilégios — designadamente, impedir que um colaborador administrativo (EMPLOYEE) pudesse revogar ou alterar os acessos de um elemento da gerência (MANAGER), cenário que representaria uma falha de segurança crítica.

Prompt utilizado:

Tarefa: Atua como um *Senior Java Security Engineer*. No SS1, precisamos de um serviço *PersonAuthorizationService* que impeça escalada de privilégios. Um funcionário (EMPLOYEE) nunca pode alterar os dados ou o papel de um MANAGER. Lança uma *IllegalStateException* caso a regra seja violada.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

Análise da Solução: O modelo adotou o padrão de *Guard Clauses*, que substitui blocos *if-else* aninhados por verificações antecipadas com saída imediata, resultando num fluxo de leitura linear e sem ambiguidade. Do ponto de vista da integração sistémica, o lançamento de *IllegalStateException* delega ao *GlobalExceptionHandler* a responsabilidade de converter a violação numa resposta HTTP 403 (*Forbidden*), mantendo a separação entre a lógica de negócio e o protocolo de comunicação REST.

```
// Excerto: PersonAuthorizationService.java (SS1)
@Service
public class PersonAuthorizationService {
    public void ensureCanManageRole(Person actor, PersonRole targetRole) {
        validateActor(actor);
        if (isManager(actor)) return;
    }
}
```

```

// Proteção contra escalada de privilégios
if (targetRole.getRole() == Role.MANAGER || isManager(targetRole.getPerson())) {
    throw new IllegalStateException("Employee cannot alter manager roles.");
}
}
}

```

3.6.3. SS2 (Sportscore): Otimização de Consultas de Validação

Contexto e Desafio: O SS2 serve como catálogo mestre de modalidades desportivas do clube. A criação de modalidades com designações duplicadas corromperia as chaves lógicas do sistema e comprometeria a integridade referencial de subsistemas dependentes. A abordagem ingênua — carregar todas as modalidades para memória e comparar em Java — é ineficiente e não escalável.

Prompt utilizado:

Tarefa: No nosso SS2RulesService, precisamos de garantir que não se criam Modalidades em duplicado. Utiliza os repositórios do Spring Data JPA para validar a existência do registo e evitar constrangimentos silenciosos de base de dados, sem carregar a entidade para a memória.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

Análise da Solução: A solução gerada recorreu aos *Derived Query Methods* do Spring Data JPA, nomeadamente `existsByNameIgnoreCase`. Este mecanismo traduz a validação diretamente para uma instrução SQL `SELECT EXISTS (...)`, executada inteiramente ao nível do motor de base de dados. A eficiência desta abordagem é dupla: elimina a transferência desnecessária de dados para a JVM e tira partido dos índices do PostgreSQL, garantindo uma validação de complexidade constante independentemente da dimensão do catálogo.

```

// Excerto: SS2RulesService.java (SS2)
@Service
public class SS2RulesService {
    public void validateUniqueModalityName(String name) {
        if (modalityRepository.existsByNameIgnoreCase(name)) {
            throw new IllegalStateException("Modality name already exists.");
        }
    }
}

```

3.6.4. SS3 (Teams): Integridade de Vínculos Desportivos Temporais

Contexto e Desafio: A gestão de plantéis exige coerência cronológica. Um atleta pode ter pertencido a uma equipa no passado e voltar a ingressar na mesma, pelo que o sistema não pode simplesmente impedir vínculos repetidos — tem de distinguir vínculos históricos de vínculos ativos, prevenindo exclusivamente a coexistência de dois registos ativos em simultâneo.

Prompt utilizado:

Tarefa: O nosso `TeamRulesService` gere os vínculos (Atleta/Treinador). Escreve um método `ensureNoActiveMembership` que aceda ao `TeamMemberRepository` e verifique se a Pessoa já tem uma relação ativa com aquela Equipa específica. Considera um vínculo “ativo” se a data de fim (`endDate`) for nula.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

Análise da Solução: A solução produzida traduziu o conceito semântico de “vínculo ativo” na condição de persistência `endDateIsNull`, uma adaptação do padrão de *Temporal Records*. Esta abordagem preserva o historial completo do atleta na base de dados para efeitos de auditoria, ao mesmo tempo que impõe a restrição de unicidade sobre o estado presente.

O repositório resultante evidencia a expressividade desta abordagem: para além do método de validação de unicidade, o LLM gerou um conjunto de queries JPQL que operam sobre o eixo temporal, permitindo reconstituir o estado exato do plantel em qualquer instante passado:

```
// Excerto: TeamMemberRepository.java (SS3)
public interface TeamMemberRepository extends JpaRepository<TeamMember, Integer> {

    // Validação de unicidade de vínculo ativo – utilizado pelo TeamRulesService
    boolean existsByTeamIdAndPersonIdAndRelationshipAndEndDateIsNull(
        Integer teamId,
        Integer personId,
        TeamRelation relationship);

    // Query temporal: reconstituição do plantel numa data específica
    @Query("""
        select tm
        from TeamMember tm
        where tm.team.id = :teamId
              and tm.relationship = :relationship
              and tm.startDate <= :at
              and (tm.endDate is null or tm.endDate >= :at)
        order by tm.startDate desc
        """)
    List<TeamMember> findActiveMembersAt(
        Integer teamId,
        TeamRelation relationship,
        LocalDateTime at);

    // Verificação de pertença ativa num instante específico (suporta múltiplas
    relações)
    @Query("""
        select (count(tm) > 0)
        from TeamMember tm
    """)
    boolean isAnyActiveAt(LocalDateTime at);
}
```

```

        where tm.team.id = :teamId
            and tm.person.id = :personId
            and tm.relationship in :relations
            and tm.startDate <= :at
            and (tm.endDate is null or tm.endDate >= :at)
        """
    boolean existsActiveMembershipAt(
        Integer teamId,
        Integer personId,
        List<TeamRelation> relations,
        LocalDateTime at);
}

```

A condição `tm.endDate is null or tm.endDate >= :at` é o núcleo da lógica temporal: um registro com `endDate` nulo representa um vínculo ainda ativo, enquanto um registro com `endDate` preenchido pode ser histórico ou futuro, dependendo da sua relação com o instante de consulta. Esta semântica permite que o sistema responda corretamente a perguntas como “quem fazia parte desta equipa no dia X?”, funcionalidade relevante para auditorias e relatórios históricos.

3.6.5. SS4 (Activities): Motor de Pesquisa Dinâmico para Calendários

Contexto e Desafio: A apresentação de treinos e competições na interface do treinador requeria um motor de pesquisa capaz de filtrar agendamentos por intervalos temporais arbitrários (ex: visualização semanal ou mensal). Uma abordagem baseada em métodos de repositório fixos implicaria a proliferação de assinaturas para cada combinação de filtros possível, tornando o repositório progressivamente ingerível.

Prompt utilizado:

Tarefa: No SS4, cria a interface `EventRepository` que estenda o `JpaRepository`. Além dos métodos normais, precisamos de suporte para filtros dinâmicos e paginação. Gera também assinaturas que nos permitam procurar eventos por Competição (ordenados por data) e um método para contagem num intervalo temporal.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

Análise da Solução: A decisão arquitetural de maior relevo na solução gerada foi a herança adicional de `JpaSpecificationExecutor<Event>`. Esta interface habilita a construção de *queries* dinâmicas através de `Specification` e `Predicate` na camada de negócio, dissociando os critérios de filtragem da definição do repositório. O resultado é um repositório com uma superfície de API mínima e estável, capaz de suportar combinações arbitrárias de filtros sem qualquer alteração estrutural.

```

// Excerto: EventRepository.java (SS4)
public interface EventRepository
    extends JpaRepository<Event, Integer>, JpaSpecificationExecutor<Event> {
    List<Event> findByCompetitionIdOrderByDateAsc(Integer competitionId);
}

```

```

    long countByDateBetween(LocalDateTime start, LocalDateTime end);
}

```

3.6.6. SS5 (Tracking): Exclusividade Mútua Lógica (Regra XOR)

Contexto e Desafio: No momento de submeter presenças, o *payload* pode conter referências a Treinos ou a Eventos. Para garantir a coesão estatística dos registos, o sistema tem de impor uma exclusividade estrita: cada registo de presença pertence a exatamente um dos dois contextos, nunca a ambos nem a — uma restrição formalmente equivalente a uma operação lógica XOR.

Prompt utilizado:

Tarefa: Atua como um *Backend Developer*. No nosso *AttendanceRulesService* (SS5), precisamos de garantir uma regra XOR lógica estrita. Cria um método que valide as chaves primárias e lance uma *IllegalArgumentException* caso ambos os IDs (*trainingId* e *eventId*) sejam nulos ou caso venham preenchidos em simultâneo.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

Análise da Solução: O design do DTO *RegisterOrUpdateAttendanceRequest* reflete a intencionalidade desta regra: ambos os campos *trainingId* e *eventId* são declarados sem anotação *@NotNull*, tornando-os explicitamente opcionais ao nível da deserialização JSON. A validação estrutural do DTO limita-se aos campos verdadeiramente obrigatórios — *present*, *athleteId* e *freeTraining* — delegando a verificação da regra de negócio XOR para a camada de serviço:

```

// Excerto: RegisterOrUpdateAttendanceRequest.java (SS5)
public record RegisterOrUpdateAttendanceRequest(
    @NotNull(message = "Present flag is required.") Boolean present,
    Integer trainingId, // opcional por design – validação XOR na camada de
serviço
    Integer eventId, // opcional por design – validação XOR na camada de
serviço
    @NotNull(message = "Athlete id is required.") Integer athleteId,
    @NotNull(message = "Free training flag is required.") Boolean freeTraining,
    Integer teamId) {
}

```

Esta separação de responsabilidades é uma decisão arquitetural deliberada: colocar a regra XOR no DTO através de anotações personalizadas seria possível, mas acoplaria uma regra de negócio central a uma estrutura de transporte. Ao mantê-la no *AttendanceRulesService*, a regra permanece agnóstica à interface REST e reutilizável em qualquer contexto de invocação interna. A implementação gerada cobre exaustivamente os quatro estados possíveis do par de identificadores, rejeitando os dois casos inválidos — ambos nulos ou ambos preenchidos — e aceitando os dois casos válidos.

```

// Excerto: AttendanceRulesService.java (SS5)
@Service

```

```

public class AttendanceRulesService {
    public void validateContext(Integer trainingId, Integer eventId) {
        // Regra XOR: a presença pertence a um Treino OU a um Evento, nunca a ambos nem
a --.
        if ((trainingId == null && eventId == null)
            || (trainingId != null && eventId != null)) {
            throw new IllegalArgumentException(
                "Exactly one of trainingId or eventId must be provided.");
        }
    }
}

```

3.6.7. SS6 (Finance): Idempotência na Receção de Webhooks

Contexto e Desafio: A integração com *gateways* financeiros externos impõe um requisito de robustez específico: as notificações de pagamento concluído (*webhooks*) podem ser entregues múltiplas vezes pelo operador externo, em consequência de retransmissões por falha de rede ou *timeout*. O sistema receptor tinha, por isso, de ser concebido como idempotente — produzindo o mesmo resultado independentemente do número de vezes que o mesmo *webhook* fosse recebido.

Prompt utilizado:

Tarefa: No SS6, cria o `SS6GatewayController` focado exclusivamente em receber *webhooks* externos. A arquitetura exige que o controlador valide os *payloads* e delegue o processamento na `SS6Facade`. O retorno deve ser sempre HTTP 204.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

Análise da Solução: A idempotência do sistema não foi implementada apenas ao nível do controlador, mas sim em profundidade, através de três camadas complementares.

A primeira camada reside na entidade de domínio `GatewayNotification`, que modela explicitamente o ciclo de vida de uma notificação. A constraint de unicidade composta (`gateway`, `notification_id`) declarada diretamente na anotação JPA constitui a garantia estrutural mais forte do sistema — mesmo que a lógica aplicacional falhe, a base de dados rejeita a persistência de notificações duplicadas:

```

// Excerto: GatewayNotification.java (SS6) – entidade de domínio
@Entity
@Table(name = "gateway_notification", uniqueConstraints = {
    @UniqueConstraint(columnNames = { "gateway", "notification_id" })
})
public class GatewayNotification extends PersonAuditableEntity {

    @Column(name = "processed", nullable = false)
    private boolean processed = false;

    @Column(name = "processed_at")
    private LocalDateTime processedAt;

```

```

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "payment_id")
private Payment payment;

public void markAsProcessed(Payment payment) {
    this.processed = true;
    this.processedAt = LocalDateTime.now();
    this.payment = payment;
}
}

```

A segunda e terceira camadas residem no SS6WebhookService, que implementa uma estratégia de defesa em profundidade: uma verificação aplicacional prévia à escrita, seguida de um tratamento defensivo da exceção de integridade que poderia emergir de uma condição de corrida concorrente:

```

// Excerto: SS6WebhookService.java (SS6)
@Service
public class SS6WebhookService {

    @Transactional
    public void process(GatewayNotificationRequest request) {
        // Camada 2: verificação aplicacional – evita escrita desnecessária
        if (gatewayNotificationRepository.existsByGatewayAndNotificationId(
            request.gateway(), request.notificationId())) {
            return; // retorno silencioso – o gateway recebe 204 como se fosse novo
        }

        Payment payment =
paymentRepository.findByExternalId(request.externalPaymentId())
            .orElseThrow(() -> new EntityNotFoundException("Payment not found"));

        ExternalPaymentStatus status = parseExternalStatus(request.payload());

        GatewayNotification notification = new GatewayNotification(
            request.gateway(), request.notificationId(),
            request.externalPaymentId(), request.payload(), request.receivedAt());

        payment.applyExternalStatus(status, request.payload(), request.receivedAt());
        notification.markAsProcessed(payment);

        try {
            gatewayNotificationRepository.save(notification);
            paymentRepository.save(payment);
        } catch (DataIntegrityViolationException ex) {
            // Camada 3: defesa contra race condition concorrente – absorção silenciosa
            return;
        }
    }

    private ExternalPaymentStatus parseExternalStatus(String payload) {
        try {
            JsonNode node = objectMapper.readTree(payload);
            JsonNode statusNode = node.get("status");

```

```

        if (statusNode == null || statusNode.asText().isBlank())
            return ExternalPaymentStatus.PENDING;
        return
ExternalPaymentStatus.valueOf(statusNode.asText().trim().toUpperCase());
    } catch (Exception ex) {
        return ExternalPaymentStatus.PENDING; // fallback defensivo para payload
malformado
    }
}
}

```

A decisão de absorver silenciosamente a `DataIntegrityViolationException` em vez de a propagar é deliberada e clinicamente correta no contexto de *webhooks*: do ponto de vista do *gateway* externo, uma notificação duplicada processada com sucesso e uma notificação duplicada detetada como tal são indistinguíveis — ambas recebem HTTP 204. Esta semântica impede retransmissões infinitas por parte do operador externo.

3.6.8. SS7 (Analytics): Controlo de Acesso e Agregação de Indicadores Executivos

Contexto e Desafio: O módulo de Analytics agrega indicadores de desempenho executivo (KPIs financeiros e desportivos) destinados exclusivamente à gerência do clube. A natureza sensível desta informação exigia que o controlo de acesso fosse aplicado ao nível mais próximo possível do recurso, e que os cálculos de agregação fossem eficientes — delegando ao motor de base de dados o trabalho de contagem e sumariação, em vez de carregar registos para memória.

Prompt utilizado:

Tarefa: Para o SS7 (Analytics), gera o `SS7Controller`. Todos os métodos devem devolver dados agregados (ex: `DashboardResponse`). Utiliza as anotações do Spring Security para garantir que os relatórios de gestão apenas podem ser acedidos pelo perfil `MANAGER`.

- **Package Structure Atual:** ...
- **Código existente para contexto:** ...
- **Documentação específica:** ...
- **Contratos de API (Request / Response DTOs):** ...
- **Nota:** ...

Análise da Solução: A utilização da anotação `@PreAuthorize("hasRole('MANAGER')")` ao nível do método constitui uma aplicação do modelo de segurança *method-level security* do Spring Security, que opera através de um proxy AOP intercetando a invocação antes de esta atingir o controlador. Esta abordagem é preferível à validação programática no corpo do método porque torna a política de acesso declarativa, auditável e independente da lógica de negócio.

```

// Excerto: SS7Controller.java (SS7)
@RestController
@RequestMapping("/api")
public class SS7Controller {

    @PreAuthorize("hasRole('MANAGER')")
    @GetMapping("/dashboard")

```

```

    public ResponseEntity<DashboardResponse> calculateDashboard() {
        return ResponseEntity.ok(ss7Facade.calculateDashboard());
    }
}

```

Os dados servidos por este *endpoint* são calculados pelo `SS7AggregationService`, que agrega métricas em tempo real através de consultas diretas aos repositórios. Os métodos `attendanceRate()` e `paymentRate()` ilustram o padrão adotado: os contadores são calculados ao nível da base de dados via métodos derivados do Spring Data JPA, e a aritmética de precisão fixa é realizada com `BigDecimal` e `RoundingMode.HALF_UP`, garantindo que as taxas percentuais são sempre arredondadas de forma determinística:

```

// Excerto: SS7AggregationService.java (SS7)
@Service
public class SS7AggregationService {

    // Taxa de presença global: presenças confirmadas / total de registos
    public BigDecimal attendanceRate() {
        long present = attendanceRepository.countByPresentTrue();
        long absent  = attendanceRepository.countByPresentFalse();
        long total   = present + absent;
        if (total == 0) return BigDecimal.ZERO;
        return BigDecimal.valueOf(present)
            .divide(BigDecimal.valueOf(total), 2, RoundingMode.HALF_UP);
    }

    // Taxa de pagamento: quotas pagas / total de quotas em qualquer estado
    public BigDecimal paymentRate() {
        long paid = feeRepository.countByStatus(FeeStatus.PAID);
        long total = paid
            + feeRepository.countByStatus(FeeStatus.PENDING)
            + feeRepository.countByStatus(FeeStatus.DEBT)
            + feeRepository.countByStatus(FeeStatus.EXPIRED);
        if (total == 0) return BigDecimal.ZERO;
        return BigDecimal.valueOf(paid)
            .divide(BigDecimal.valueOf(total), 2, RoundingMode.HALF_UP);
    }

    // Contagem de atletas com papel ativo (endDate nulo)
    public Integer countAthletes() {
        return (int) personRoleRepository
            .findByRoleAndEndDateIsNull(Role.ATHLETE).size();
    }

    // Novos atletas registados no mês corrente
    public Integer countNewAthletesThisMonth() {
        LocalDate start = LocalDate.now().withDayOfMonth(1);
        LocalDate end   = start.plusMonths(1).minusDays(1);
        return personRoleRepository
            .findByRoleAndStartDateBetween(Role.ATHLETE, start, end).size();
    }
}

```

A escolha de `BigDecimal` em detrimento de `double` para os cálculos de taxa é uma decisão de correção numérica: operações de divisão em vírgula flutuante de precisão dupla acumulam erros de

representação que, em contexto financeiro e de relatório, produzem resultados incorretos. O uso explícito de `RoundingMode.HALF_UP` torna o comportamento de arredondamento auditável e consistente com as convenções contabilísticas.

3.7. Estratégia de Deployment e Integração Contínua

Para garantir que o sistema transitasse do ambiente de desenvolvimento local para um ambiente de produção acessível aos utilizadores finais, foi desenhada uma estratégia de *deployment* assente em três plataformas complementares: **Vercel** para o *frontend*, **Render** para o *backend* e **Supabase** para a base de dados PostgreSQL. A separação deliberada entre estas três camadas permitiu otimizar cada uma de forma independente, tanto em termos de escalabilidade como de ciclo de atualizações.

3.7.1. Frontend (Vercel)

O cliente web em React, gerido pelo Vite, beneficia de um processo de *build* estático que produz um conjunto de ficheiros otimizados — HTML, CSS e JavaScript minificado. Esta característica tornou o Vercel a escolha natural para o alojamento, pela sua integração nativa com repositórios Git: qualquer *push* para a *branch* principal desencadeia automaticamente um novo *build* e *deploy*, sem intervenção manual e sem custos acrescentados.

Para além da CI/CD transparente, o Vercel oferece distribuição via CDN global, o que se traduz em tempos de carregamento (TTFB) reduzidos independentemente da localização geográfica do utilizador. Um aspeto crítico para o correto funcionamento do React Router em produção foi a configuração de *redirects* na plataforma: sem esta configuração, qualquer acesso direto a uma rota da SPA resultaria num erro 404, uma vez que o servidor tentaria resolver o caminho como um ficheiro estático inexistente. O Vercel resolve este comportamento de forma nativa através do ficheiro `vercel.json`, redirecionando todos os pedidos para o `index.html`:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
```

Variáveis de ambiente no Vercel:

Para que o frontend consiga comunicar corretamente com o backend alojado no Render, foram definidas as seguintes variáveis de ambiente na plataforma:

VITE_API_BASE_URL — URL base da API consumida pelo frontend.

VITE_AUTH_MODE - modo de execução da aplicação, permitindo alternar entre mock e real.

Estas variáveis são injetadas no momento da build do Vite, permitindo que o cliente React conheça o endereço do backend sem que este fique hardcoded no código-fonte.

3.7.2. Backend (Render)

A aplicação Spring Boot foi alojada na plataforma Render, que oferece um modelo de deployment baseado em repositório Git — análogo ao Vercel, mas orientado para serviços com estado e cargas de trabalho contínuas.

Compilação e arranque:

O processo de build é executado pelo Maven, que compila o projeto e empacota a aplicação num ficheiro .jar executável. Este artefacto é então iniciado pelo Render, que gere o ciclo de vida do processo, o reinício automático em caso de falha e a exposição do serviço via HTTPS.

Externalização de configurações sensíveis:

Um princípio central da configuração de produção foi a total externalização de variáveis sensíveis para fora do código-fonte. As credenciais e tokens necessários são injetados exclusivamente através de variáveis de ambiente configuradas no painel do Render. Esta separação entre configuração e código segue a metodologia Twelve-Factor App e impede que informação sensível seja acidentalmente versionada no repositório.

Variáveis de ambiente no Render:

CORS e Segurança:

CORS_ALLOWED_ORIGINS — origens autorizadas para aceder à API.

JWT_SECRET — chave secreta usada para assinar e validar tokens JWT.

JWT_ACCESS_EXPIRATION — tempo de validade do access token, em milissegundos.

JWT_REFRESH_EXPIRATION — tempo de validade do refresh token, em milissegundos.

Base de Dados (Supabase):

SPRING_DATASOURCE_URL — URL de ligação à base de dados PostgreSQL.

SPRING_DATASOURCE_USERNAME — utilizador da base de dados.

SPRING_DATASOURCE_PASSWORD — palavra-passe da base de dados.

JPA Configuration:

SPRING_JPA_HIBERNATE_DDL_AUTO — estratégia de gestão do schema.

SPRING_JPA_SHOW_SQL — ativa ou desativa logging de SQL.

SPRING_JPA_FORMAT_SQL — define se o SQL é formatado no output.

SPRING_JPA_USE_SQL_COMMENTS — ativa ou desativa comentários SQL gerados pelo Hibernate.

Gateway de Pagamentos:

PAYMENT_GATEWAY MOCK_ENABLED — ativa o modo mock do gateway de pagamentos.

Se o sistema evoluir para pagamentos reais, será recomendável configurar variáveis adicionais, como:

PAYMENT_GATEWAY_URL

PAYMENT_GATEWAY_ENTITY

PAYMENT_GATEWAY_API_KEY

Agendamento:

MONTHLY_FEE_GENERATION_CRON — expressão CRON usada para gerar mensalidades automaticamente

3.7.3. Base de Dados (Supabase)

A base de dados PostgreSQL foi alojada na plataforma Supabase, escolhida pelas seguintes vantagens:

Pooler de conexões integrado: O Supabase disponibiliza um connection pooler que gere eficientemente as ligações provenientes do Render, evitando exaustão de conexões e melhorando a latência em cenários de alta concorrência.

SSL/TLS forçado: A configuração `sslmode=require` garante que todas as comunicações entre o backend e a base de dados são encriptadas, cumprindo requisitos de segurança para dados pessoais (RGPD).

Backups automáticos: A plataforma realiza backups diários da base de dados, permitindo recuperação em caso de falha catastrófica ou corrupção de dados.

Gestão do esquema da base de dados:

A sincronização do esquema é assegurada pela propriedade `spring.jpa.hibernate.ddl-auto`, configurada como `update` em produção. Esta abordagem delega ao Hibernate a responsabilidade de detetar divergências entre o modelo de entidades Java e o esquema PostgreSQL existente, aplicando as alterações necessárias de forma não destrutiva no arranque da aplicação. Esta decisão foi adequada para a fase de desenvolvimento e validação do projeto, mantendo o esquema sempre alinhado com o modelo de domínio sem requerer migrações manuais.

3.7.4. Visão Geral do Pipeline e Fluxo de Dados

O diagrama seguinte ilustra o fluxo completo desde o desenvolvimento até à produção, bem como as interações entre as três camadas da arquitetura:

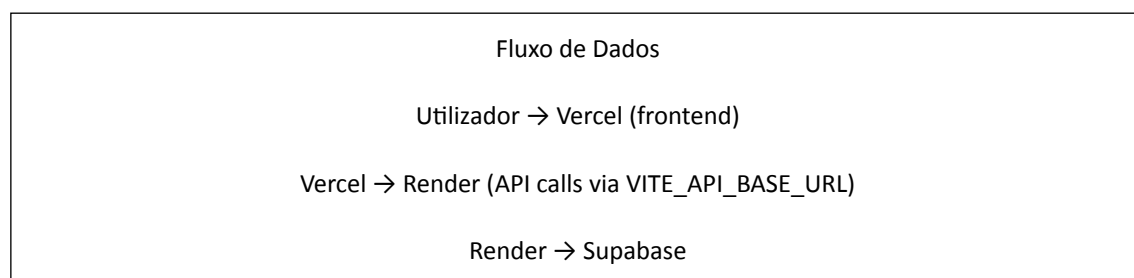


Figura 32: Pipeline de e fluxo de dados entre camadas.

3.7.5. Resumo do Fluxo de Comunicação

Esta arquitetura de deployment garantiu que cada camada da aplicação permanecesse sincronizada com o repositório, eliminando o risco de divergências entre o código em desenvolvimento e o ambiente de produção. A separação clara de responsabilidades — Vercel para o frontend, Render para a lógica de negócio e Supabase para a persistência — permitiu otimizar cada componente de forma independente e assegurou que cada iteração do projeto fosse imediatamente validável pelos utilizadores finais.

3.8. Demonstração do Sistema Final

Esta secção apresenta o sistema desenvolvido no seu estado de produção, acessível em ambiente real através das plataformas Vercel (frontend) e Render (backend). As capturas de ecrã que se seguem foram recolhidas diretamente do ambiente de produção e ilustram os principais fluxos funcionais do clube desportivo, organizadas em duas perspetivas complementares: por perfil de utilizador e por subsistema.

3.8.1. Fluxos por Perfil de Utilizador

A aplicação serve quatro perfis distintos com interfaces e permissões diferenciadas: **Gerente**, **Colaborador**, **Treinador** e **Atleta**. Cada perfil acede a um conjunto restrito de funcionalidades, determinado pelas regras de autorização implementadas no SS1.

3.8.2. Perfil Gerente

O Gestor dispõe do acesso mais abrangente do sistema, incluindo a gestão de recursos humanos, a supervisão financeira e a consulta de indicadores executivos agregados.

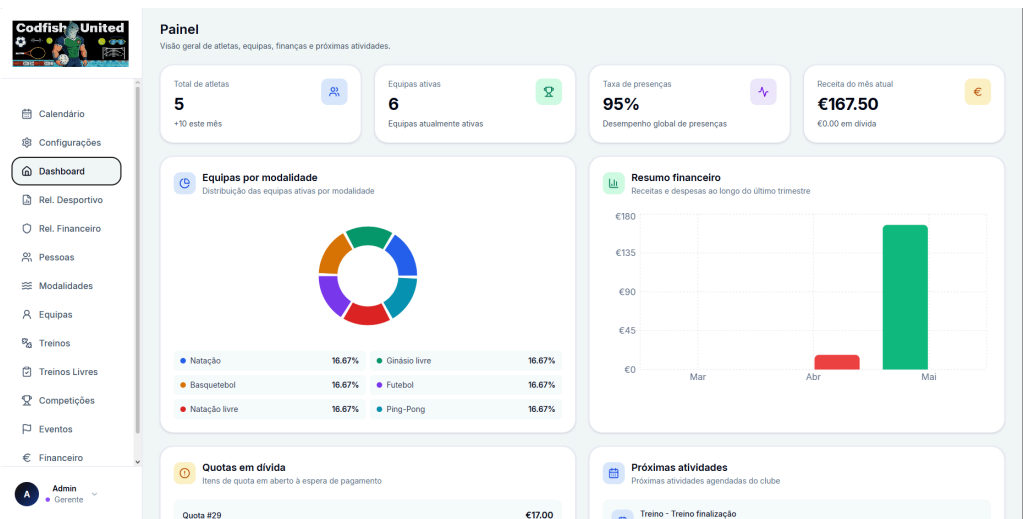


Figura 33: Dashboard executivo do Gestor com indicadores KPI agregados (SS7).

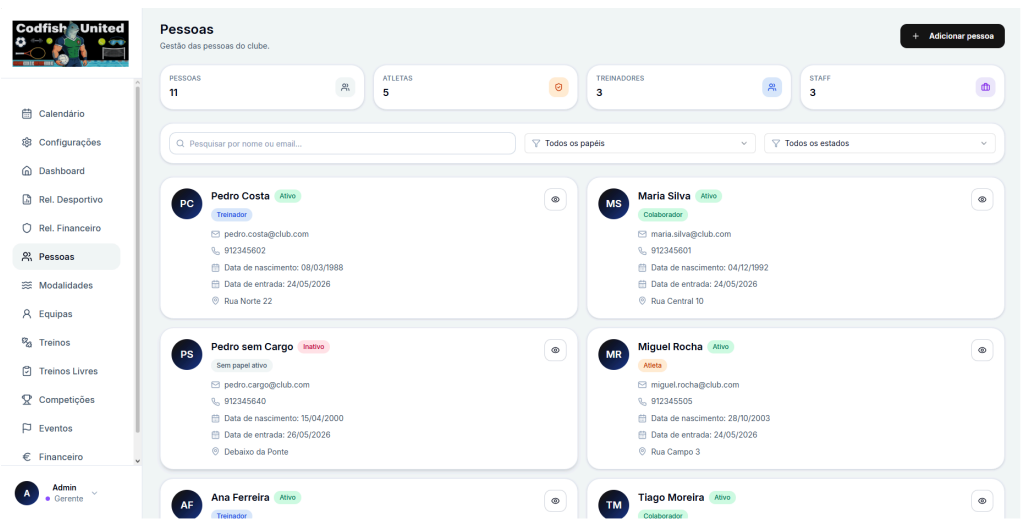


Figura 34: Gestão de pessoas e papéis.

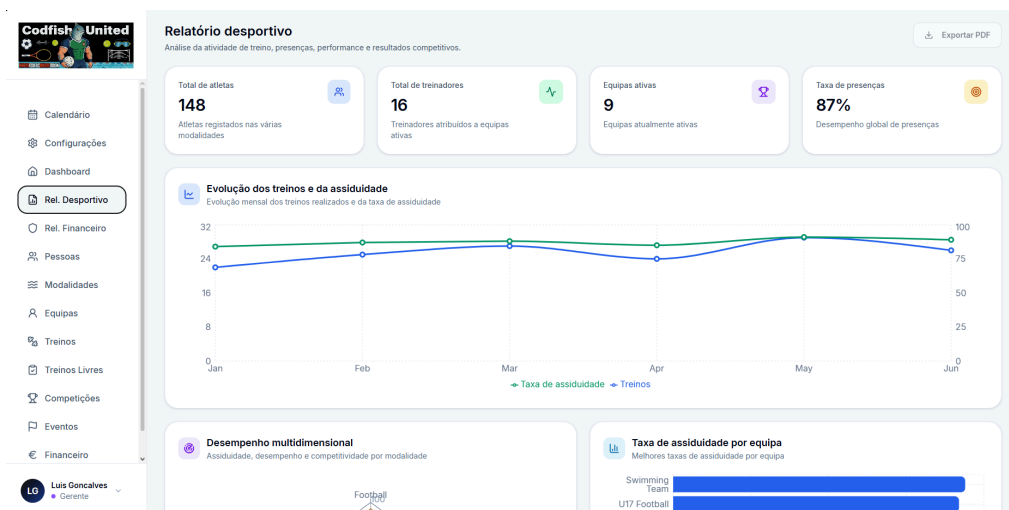


Figura 35: Relatório desportivo do clube.

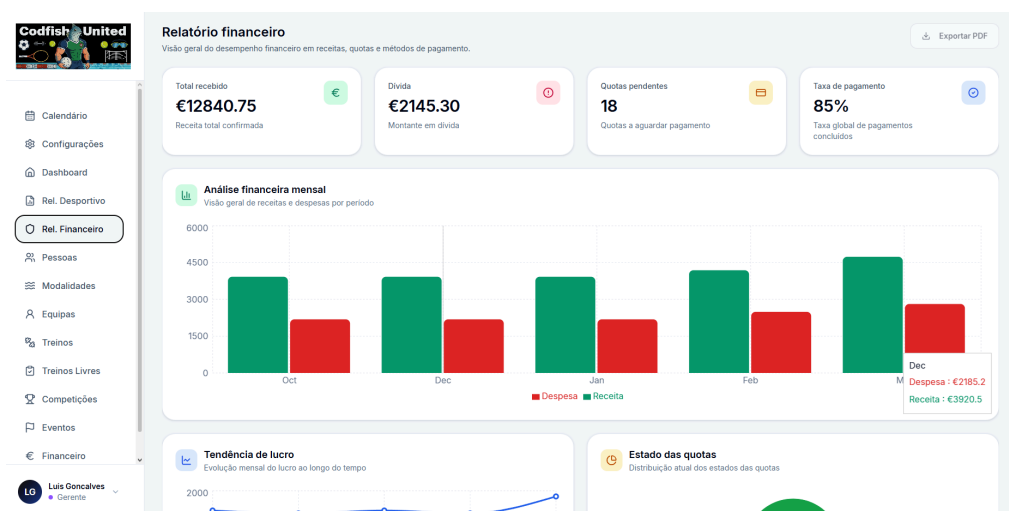


Figura 36: Relatório financeiro do clube.

3.8.3. Perfil Colaborador

O Colaborador tem acesso à gestão administrativa do clube, nomeadamente o registo e edição de pessoas, a criação de modalidades e equipas e a gestão de quotas. Ao contrário do Gestor, não tem acesso aos relatórios executivos nem pode alterar papéis de utilizadores com privilégios superiores.

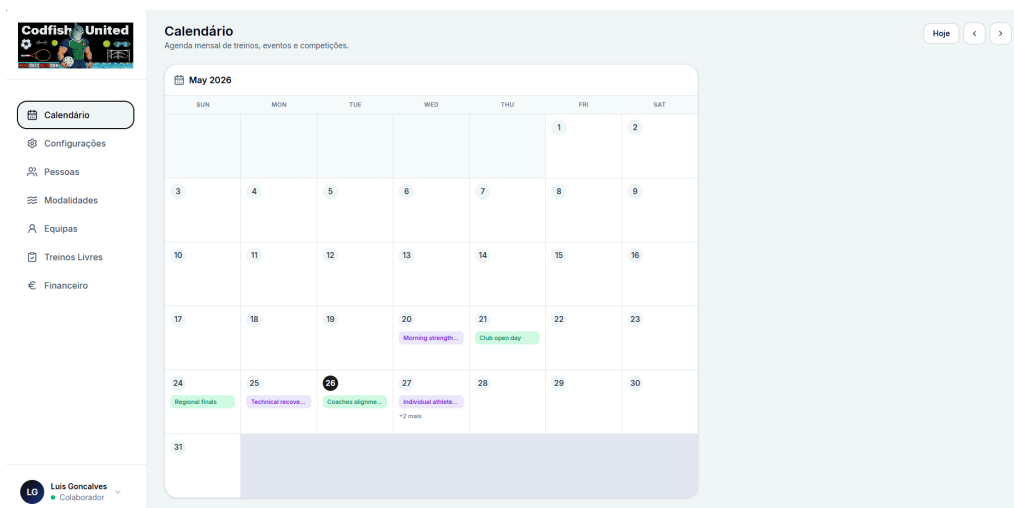


Figura 37: Vista inicial do Colaborador com acesso às funcionalidades administrativas.

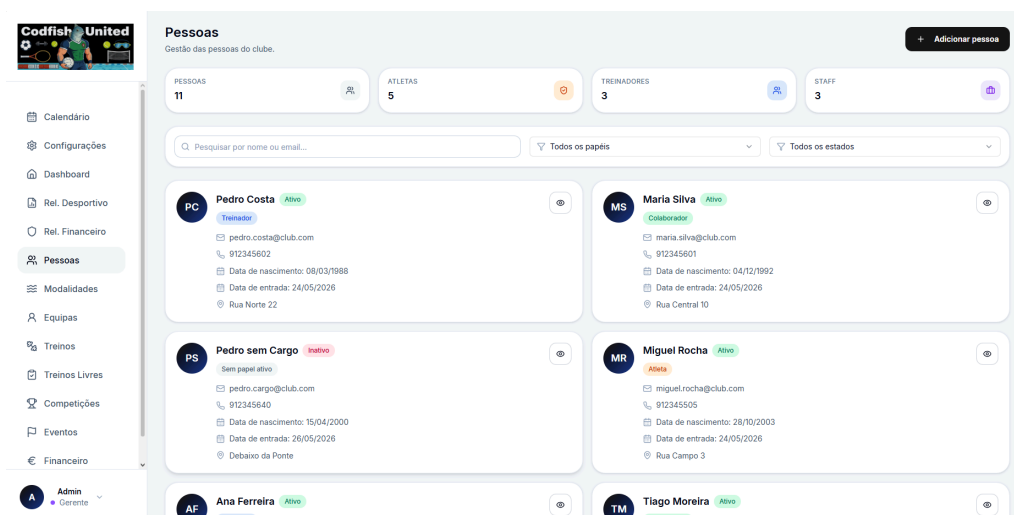


Figura 38: Gestão de pessoas.

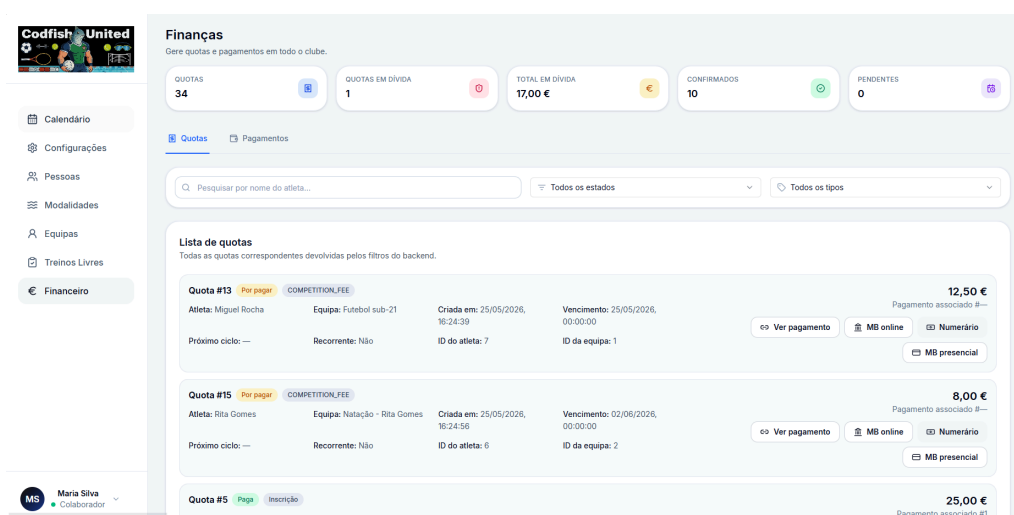


Figura 39: Administração de quotas e pagamentos.

3.8.4. Perfil Treinador

O Treinador tem acesso à gestão das suas equipas, ao agendamento de treinos e eventos e ao registo de presenças dos atletas.

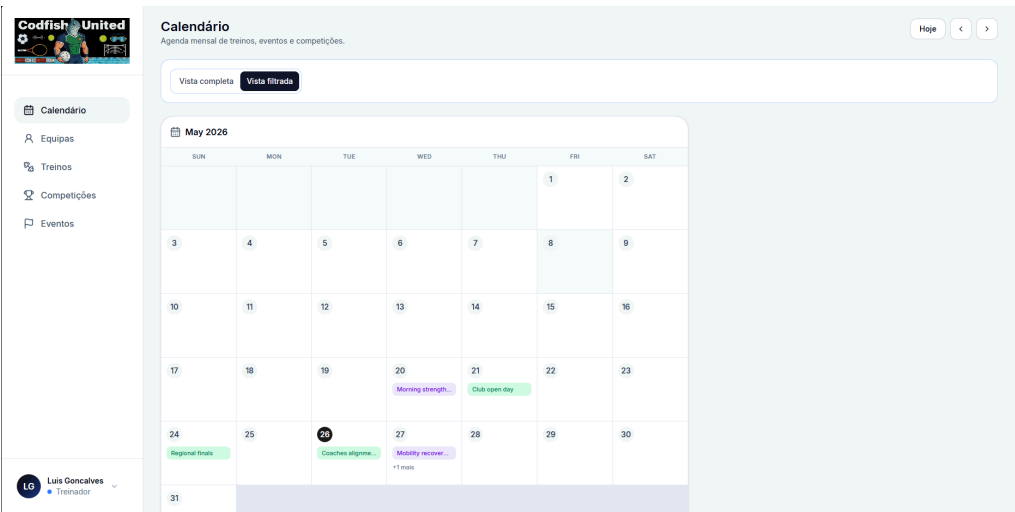


Figura 40: Calendário de treinos.

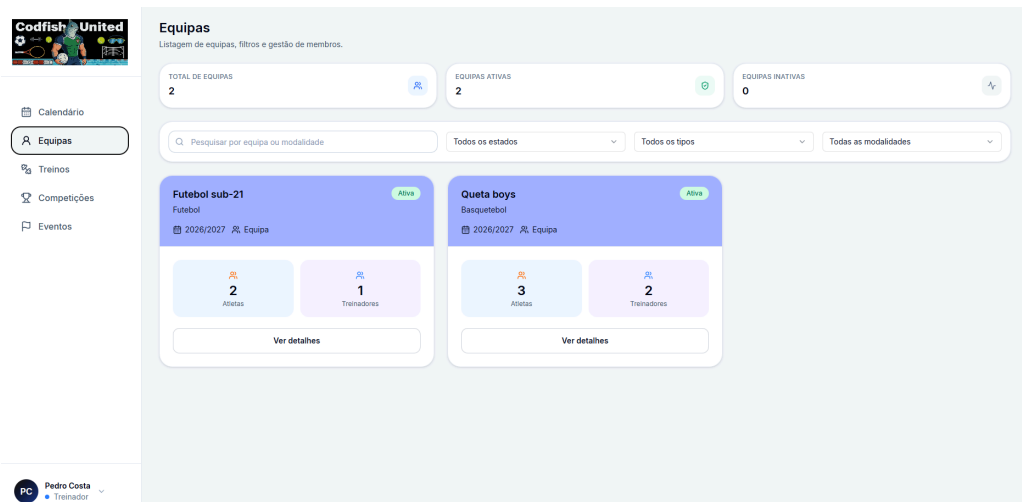


Figura 41: Gestão de plantéis

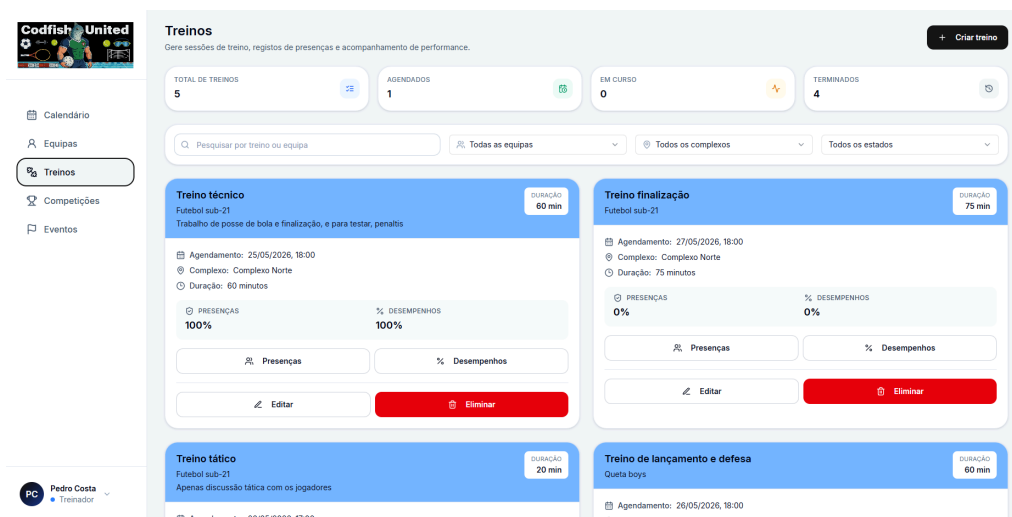


Figura 42: registo de presenças com regra XOR.

3.8.5. Perfil Atleta

O Atleta acede a uma vista simplificada focada na consulta do seu calendário pessoal, historial de presenças e estado financeiro das suas quotas.

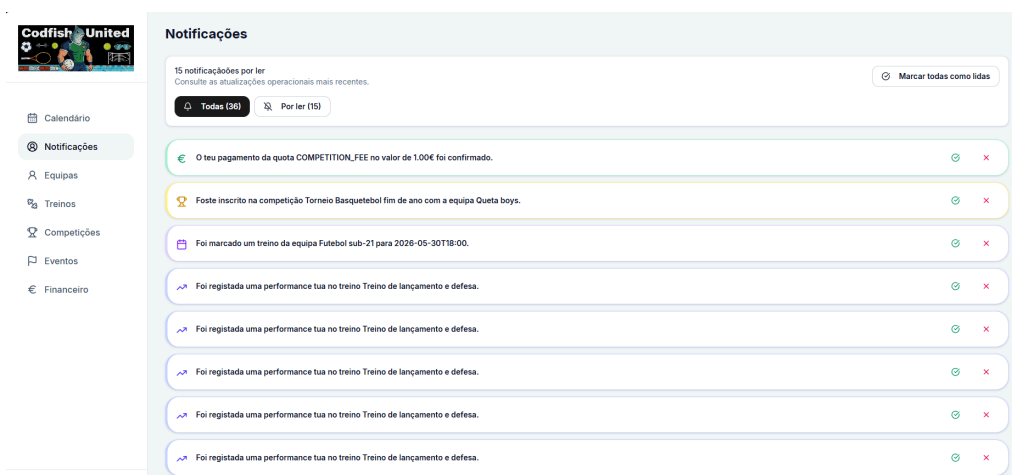


Figura 43: Vista pessoal do Atleta com resumo de atividades e estado de pagamentos.

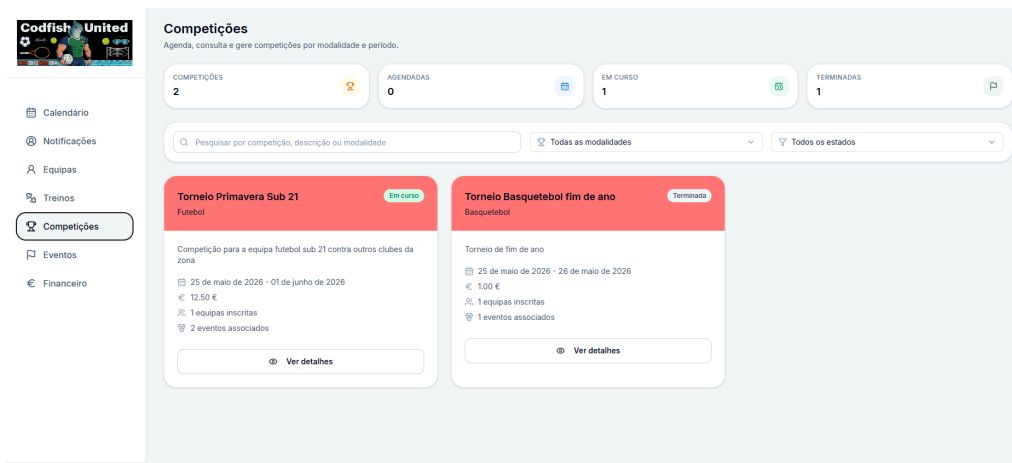


Figura 44: Vista para inscrições de competição.

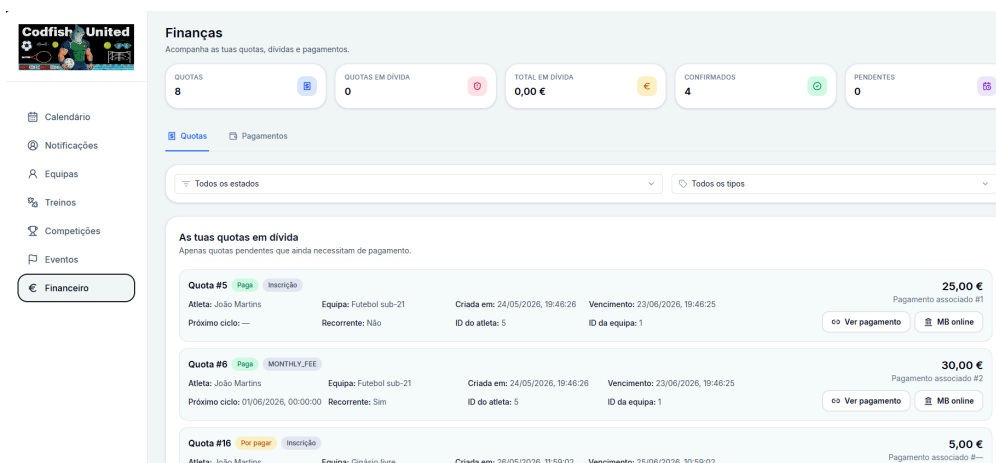


Figura 45: Vista pessoal do Atleta com resumo de atividades e estado de pagamentos.

3.9. Vista por Subsistema

3.9.1. SS1 — Identidade e Autenticação

O fluxo de autenticação é o ponto de entrada de todos os perfis. O sistema valida as credenciais, emite um token JWT e redireciona o utilizador para a interface correspondente ao seu papel.



Área de membro

Inicie sessão para aceder à plataforma do clube.

Email

Palavra-passe

Esqueceu-se da palavra-passe?

[→ Iniciar sessão](#)

[← Voltar à página inicial](#)

Figura 46: Ecrã de autenticação em desktop (identico em mobile).

3.9.2. SS2 — Modalidades Desportivas

O catálogo de modalidades é gerido exclusivamente pelo Gestor. A interface impede a criação de modalidades com designações duplicadas, com validação em tempo real.

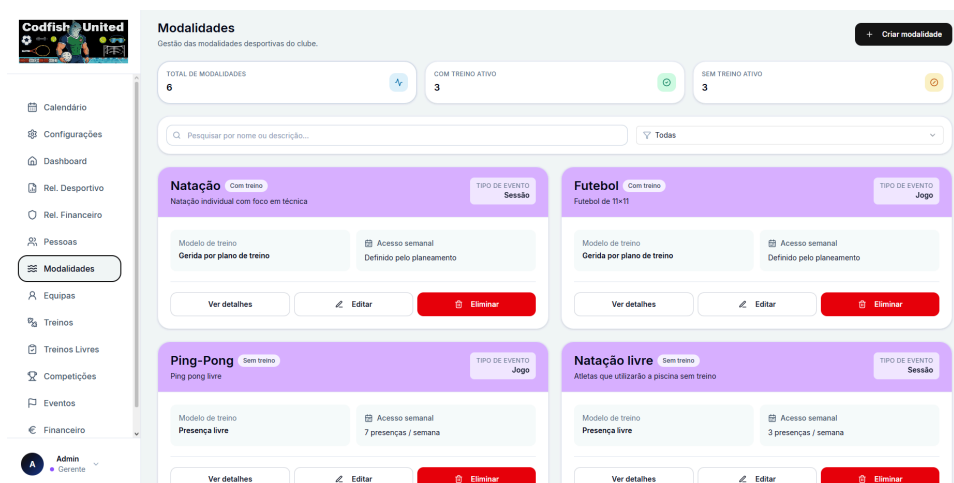


Figura 47: Listagem e gestão do catálogo de modalidades desportivas (SS2).

3.9.3. SS3 — Equipas e Vínculos

A gestão de plantéis permite associar atletas e treinadores a equipas com controlo de datas de início e fim de vínculo, preservando o historial temporal completo.

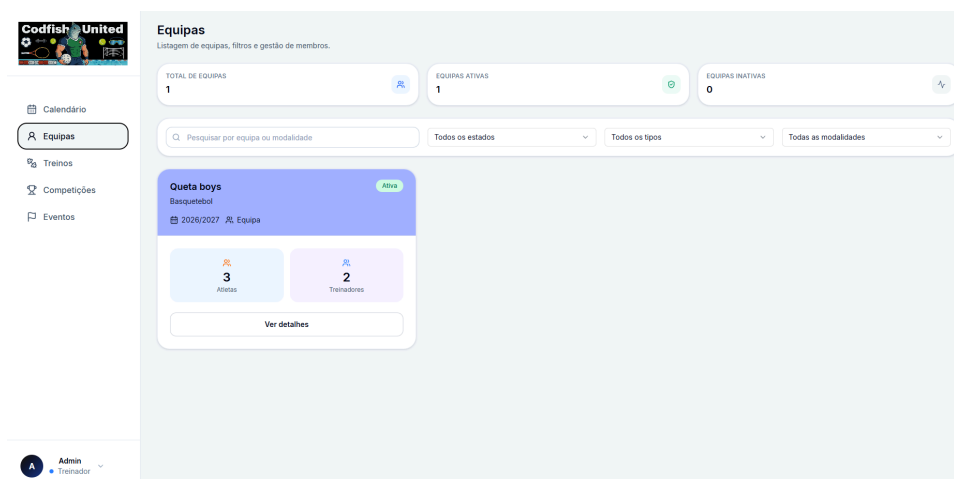


Figura 48: Gestão de plantéis com vínculos temporais ativos e históricos (SS3).

3.9.4. SS4 — Agendamento de Atividades

O motor de calendário suporta filtragem dinâmica por intervalo temporal e distingue entre treinos e eventos competitivos. A vista semanal e mensal é acessível ao Treinador.

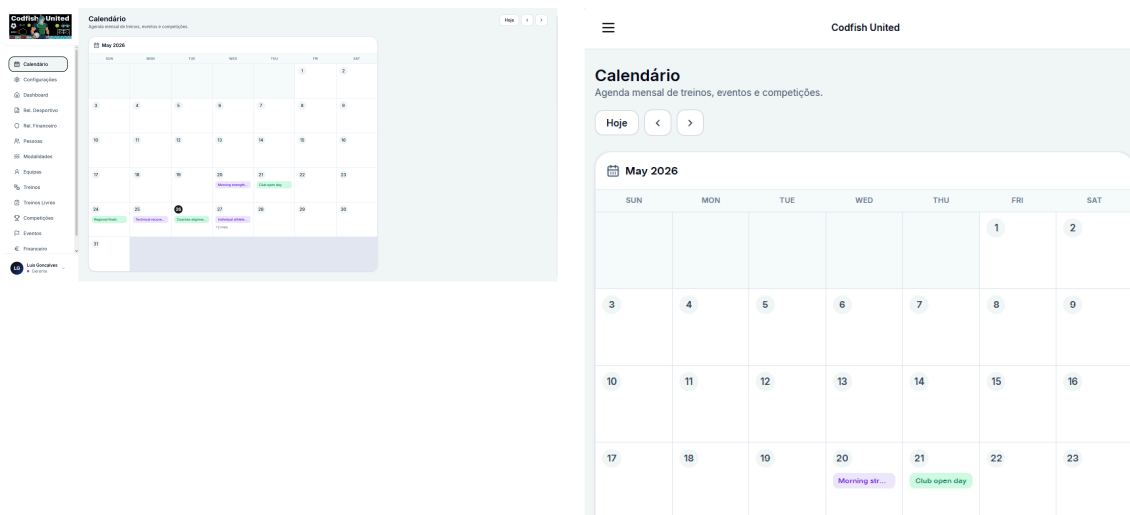


Figura 49: Calendário de atividades em desktop (esquerda) e em dispositivo móvel (direita) (SS4).

3.9.5. SS5 — Registo de Presenças

O formulário de submissão de presenças aplica a regra XOR em tempo real, impedindo a associação simultânea a um Treino e a um Evento.

Codfish United

Calendário

Agenda mensal de treinos, eventos e competições.

Hoje

<

>

May 2026

SUN	MON	TUE	WED	THU	FRI	SAT
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20 Morning str...	21 Club open day	22	23

Calendário

Agenda mensal de treinos, eventos e competições.

May 2026

SUN	MON	TUE	WED	THU	FRI	SAT
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20 Morning str...	21 Club open day	22	23

Figura 50: Interface de registo de presenças com validação XOR ativa (SS5).

3.9.6. SS6 — Módulo Financeiro e Pagamentos

O módulo financeiro gera referências Multibanco através da integração com um gateway externo. O Atleta visualiza a entidade, referência e prazo de pagamento diretamente na aplicação.

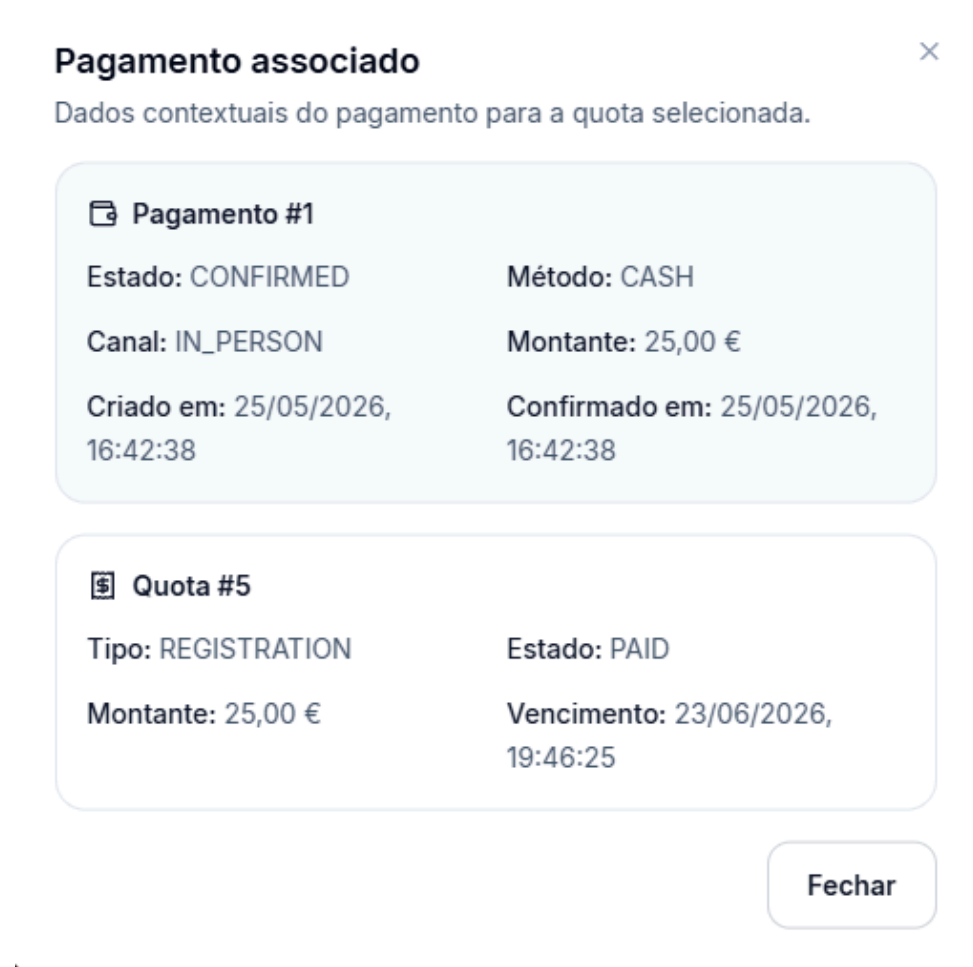


Figura 51: Interface com informação de pagamento

3.9.7. SS7 — Analytics e Relatórios Executivos

O dashboard executivo agrega indicadores financeiros e desportivos em tempo real, acessível exclusivamente ao perfil Gestor. Os dados são calculados por consultas de agregação diretamente na base de dados PostgreSQL.

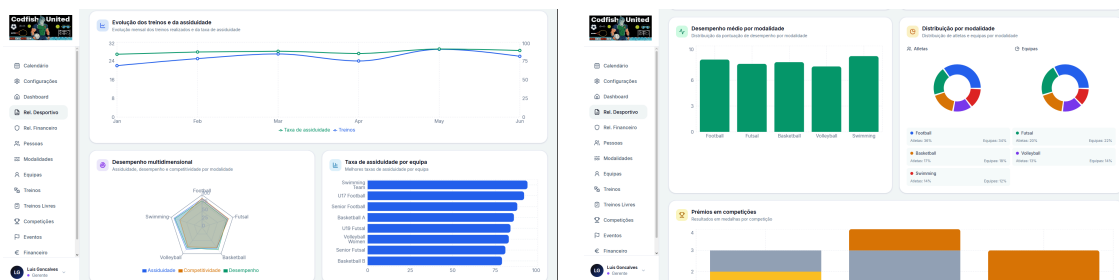


Figura 52: Resumos financeiros

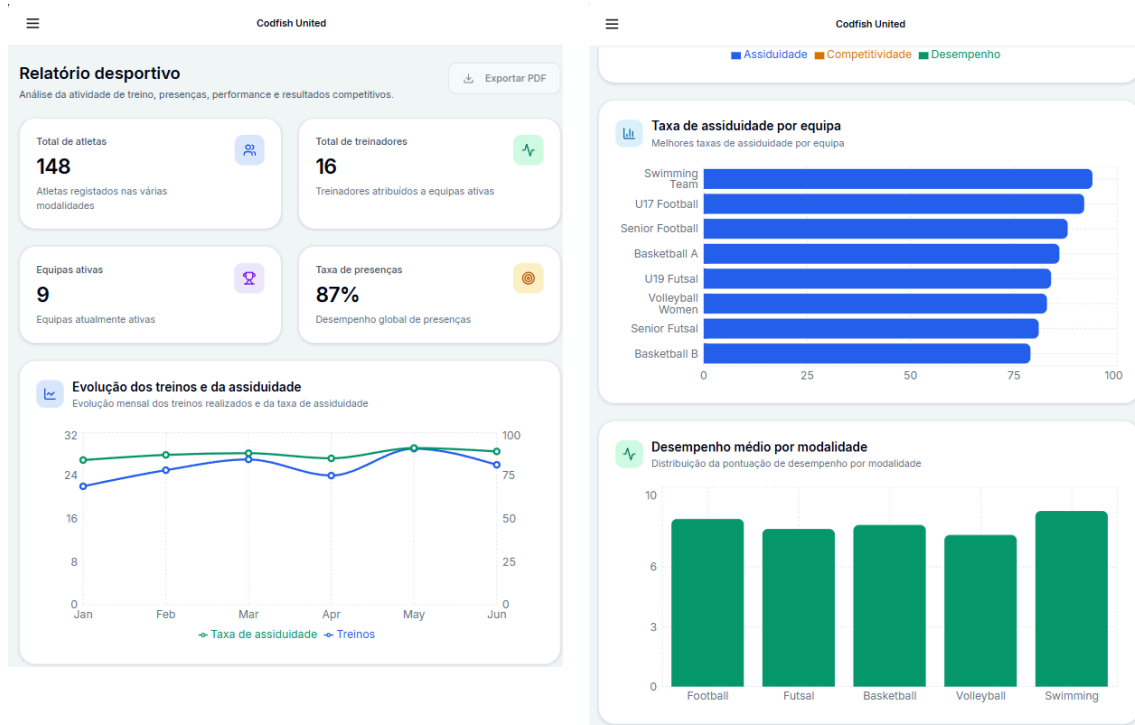


Figura 53: Resumos financeiros em dispositivo móvel

3.10. Síntese

As capturas de ecrã apresentadas atestam a concretização integral dos requisitos funcionais definidos na fase de especificação. O sistema encontra-se operacional em ambiente de produção, servindo os quatro perfis de utilizador — Gerente, Colaborador, Treinador e Atleta — com interfaces adaptadas ao contexto de cada papel e com suporte a dispositivos móveis e desktop.

A coerência entre a arquitetura desenhada (Capítulo 2), a implementação documentada (Capítulo 3) e o produto final aqui demonstrado valida a eficácia da estratégia de desenvolvimento incremental adotada, bem como o contributo da assistência por LLM na aceleração do ciclo de entrega sem comprometer a qualidade arquitetural do sistema.

4. Capítulo 4 - Verificação, Validação e Avaliação da Qualidade do Software

A etapa final do projeto focou-se na garantia de que o Sistema Integrado de Gestão do Codfish United cumpre rigorosamente os requisitos especificados (SRS) e atende aos padrões de qualidade arquiteturais delineados. Dada a natureza do desenvolvimento em paralelo entre a *Single Page Application* (React) e a API (Spring Boot), a estratégia de verificação foi adaptada para focar na resiliência e corretude dos contratos do *backend*, culminando em testes manuais de integração na interface.

4.1. Estratégia de Validação Baseada na API (API-Driven Testing)

Para validar o comportamento funcional e as regras de domínio do sistema sem depender da interface gráfica final, a equipa adotou uma estratégia de testes focada exclusivamente no consumo da API RESTful.

Em vez de testes unitários automatizados, priorizou-se a validação de integração e os Testes de Aceitação do Utilizador (UAT) diretamente na camada de rede. O objetivo foi garantir que o sistema processava corretamente as entradas, impunha as restrições de segurança (RBAC) e traduzia falhas de negócio em códigos HTTP semânticos (ex: 400 Bad Request, 403 Forbidden, 409 Conflict).

4.2. Validação Interativa e Testes de Aceitação via Swagger UI

A principal ferramenta utilizada para a execução dos testes funcionais foi o **Swagger UI**, incorporado nativamente no projeto através da biblioteca `springdoc-openapi`. Uma vez que o desenvolvimento do *frontend* ocorreu de forma faseada, o Swagger assumiu o papel crítico de “aplicação cliente” (um *frontend* provisório).

Esta ferramenta interativa permitiu à equipa simular fluxos completos de utilização:

1. **Testes de Autenticação e Autorização (Segurança):** Através da funcionalidade de injeção de *tokens* JWT da plataforma, a equipa extraiu *tokens* de diferentes perfis (Gerente, Treinador, Colaborador e Atleta). Realizaram-se testes cruzados — por exemplo, tentar aceder ao *endpoint* de criação de modalidades (POST `/api/modalidades`) utilizando um *token* de Atleta. O Swagger permitiu validar visualmente que a API intercetava o pedido e devolvia o erro HTTP 403 (Forbidden), confirmando a robustez da anotação `@PreAuthorize`.
2. **Cenários de Teste de Domínio (Exemplo Prático):** Para testar o Subsistema de Atividades (SS5), foi simulada a submissão de um registo de presença. A equipa injetou intencionalmente um *payload* JSON contendo simultaneamente o `trainingId` e o `eventId`. O Swagger comprovou a eficácia da “Regra XOR”, devolvendo um HTTP 400 com a mensagem semântica proveniente do `GlobalExceptionHandler`, garantindo que não existia corrupção de base de dados.

4.3. Validação da Camada de Apresentação

Após a estabilização da API, a validação estendeu-se à *Single Page Application* desenvolvida em React e alojada na plataforma Vercel. Embora não tenham sido implementados testes *End-to-End* (E2E) automatizados via *scripts* (como Cypress ou Selenium), a equipa executou uma bateria rigorosa de Testes Manuais Exploratórios e Testes de Usabilidade.

Estes testes focaram-se em três vetores essenciais:

- **Responsividade e Contexto Móvel:** Dado o requisito de que os treinadores utilizariam o sistema no campo, o *frontend* foi testado emulando resoluções de *smartphones* e *tablets*. Validou-se que as tabelas de convocatórias e os botões de registo de presença permaneciam acessíveis e sem quebras de *layout*.
- **Validação de Formulários:** Confirmou-se que o React intercetava *inputs* inválidos (como datas de nascimento no futuro ou ausência de campos obrigatórios) antes de estes serem enviados para o servidor, poupando largura de banda e melhorando o *feedback* visual ao utilizador.
- **Gestão de Estado e Sessão:** Testou-se a persistência do *token* JWT no *Local Storage* e o comportamento da interface perante a expiração do mesmo, garantindo que o utilizador era redirecionado suavemente para o ecrã de *login* sem falhas não tratadas na interface.

4.4. Testes de Concorrência e Validação de Optimistic Locking

Uma das validações mais críticas efetuadas durante a fase de testes incidiu sobre a gestão de concorrência. Dado o ambiente colaborativo da aplicação — onde múltiplos atores podem aceder e modificar os mesmos recursos simultaneamente (por exemplo, a edição da mesma ficha de treino por dois membros da equipa técnica) — a equipa implementou e validou o mecanismo de **Optimistic Locking**.

A validação deste comportamento transacional estendeu-se de ponta a ponta (*end-to-end*), testando tanto a resiliência do Servidor como o tratamento de falhas no Cliente:

- **Validação no Backend (Spring Boot e JPA):** O mecanismo foi implementado com recurso à anotação `@Version` nas entidades de domínio críticas. Durante a fase de testes no Swagger UI, a equipa simulou uma condição de corrida (*race condition*), submetendo dois pedidos PUT para o mesmo recurso com a mesma versão base. Validou-se com sucesso que a API processava o primeiro pedido (HTTP 200) e intercetava o segundo, convertendo a exceção do motor de base de dados (`OptimisticLockException`) num erro semântico HTTP 409 (*Conflict*).
- **Validação no Frontend (React):** Na interface gráfica, simulou-se a mesma concorrência utilizando dois separadores de *browser* em simultâneo. A equipa testou a captura do código HTTP 409 através dos intercetores globais do Axios. Validou-se que a *Single Page Application* impede a sobreposição silenciosa de dados, bloqueando o formulário e exibindo uma notificação de sistema contextual (ex: “Os dados foram alterados por outro utilizador. Por favor, atualize a página.”), forçando a revalidação da *query* para obter o estado mais recente.

Este cenário de teste confirmou que o sistema garante a integridade dos dados face a colisões simultâneas, evitando o bloqueio pessimista (que prejudicaria a performance da base de dados) e garantindo uma experiência de utilizador (UX) à prova de falhas na recuperação de dessincronizações.

4.5. Matriz de Rastreabilidade (Requisitos vs. Endpoints)

A conformidade do sistema foi validada mapeando os requisitos funcionais críticos elaborados no Capítulo 1 aos respectivos *endpoints* da API, testados com sucesso:

Req.	Subsistema	Endpoint API	Ferramenta de Validação
R02	SS1 - Identity	POST /api/auth/login	Swagger UI / React
R11	SS2 - Sportscore	POST /api/modalidades	Swagger UI
R18	SS3 - Teams	POST /api/equipas	Swagger UI / React
R29	SS4 - Activities	POST /api/treinos	Swagger UI / React
R30	SS5 - Tracking	POST /api/presencas	Swagger UI (Lotes)
R44	SS6 - Finance	POST /api/gateway/notifications	Swagger UI
R46	SS7 - Analytics	GET /api/dashboard	Swagger UI

4.6. Reflexão Crítica: O LLM como Ferramenta de Teste e QA

Embora a integração da IA tenha acelerado significativamente o desenvolvimento, o processo de auditoria e validação evidenciou as limitações intrínsecas dos grandes modelos de linguagem.

Durante a geração de cenários atípicos (*edge cases*), registaram-se episódios de “alucinação”, onde o modelo sugeriu a verificação de propriedades ou a invocação de métodos de repositório que não existiam na base de código do projeto. Adicionalmente, em certos *prompts* de segurança, o modelo sugeriu configurações obsoletas (próprias de versões anteriores do Spring Security).

Isto comprovou que o LLM não substitui a compreensão profunda do domínio. A IA operou com excelência como um “assistente de sugestões”, mas a validação final, a decisão arquitetural e a garantia de qualidade (QA) exigiram invariavelmente uma revisão técnica manual e criteriosa por parte da equipa de engenharia.

4.7. Avaliação de Qualidade e Requisitos Não Funcionais (ISO/IEC 25010)

No que concerne aos Requisitos Não Funcionais (RNF), particularmente as métricas de tempo de resposta (ex: carregamento de listagens em menos de 2 segundos), comprovou-se que a agressividade destas métricas foi mitigada através de paginação nativa e projeções *Read-Only* na API, comprovado pelas medições de latência do Swagger UI.

Para garantir a sustentabilidade do produto final, a qualidade do *software* foi avaliada tendo como referencial quatro subcaracterísticas fundamentais da norma internacional **ISO/IEC 25010**:

Característica ISO/IEC 25010	Avaliação no Codfish United
Adequação Funcional	O sistema cobre os objetivos do domínio. Os testes exploratórios garantiram a ausência de comportamentos ambíguos nas chamadas à API e no Frontend.
Eficiência de Desempenho	A otimização de consultas JPA e a omissão de entidades pesadas nos DTOs permitiram cumprir as métricas de resposta rápida em ambiente Cloud.
Usabilidade	O React UI foi desenhado com responsividade, enquanto a API previne erros do cliente através do padrão RBAC e mensagens semânticas (GlobalExceptionHandler).
Manutenibilidade	O <i>design</i> em Monólito Modular acentua a forte separação de responsabilidades. Alterações no módulo financeiro (SS6) não impactam a estabilidade dos treinos (SS4).

Conclui-se assim que a plataforma fornece uma fundação transacional robusta, segura e funcionalmente preparada para o ambiente de produção do clube.

5. Capítulo 5 - Conclusões e Trabalho Futuro

A realização deste projeto no âmbito da unidade curricular de Laboratórios de Informática IV permitiu a conceção e o desenvolvimento de uma solução de *software* complexa, desde a fase embrionária de engenharia de requisitos até ao seu *deployment* e validação técnica. O Sistema Integrado de Gestão do Codfish United cumpriu o seu propósito central: digitalizar e desburocratizar a gestão desportiva e financeira de um clube.

5.1. Reflexão Crítica e Desafios

O aspeto mais positivo deste trabalho foi a adoção rigorosa da arquitetura em Monólito Modular. A divisão em sete subsistemas obrigou a equipa a pensar criticamente sobre o acoplamento, resultando numa API onde o sistema financeiro (SS6) e o motor desportivo (SS4/SS5) partilham a base de dados PostgreSQL sem violar as fronteiras de domínio. A utilização de LLMs como assistentes de geração de código, auditores de segurança e planeadores de testes provou ser uma mais-valia vital para o cumprimento dos prazos e elevação da qualidade estrutural.

Em contraponto, o projeto apresentou desafios técnicos significativos. A configuração da segurança *stateless* com **Spring Security** e **JWT**, aliada às políticas de partilha de recursos (CORS) com a *Single Page Application* em **React**, exigiu refatorizações exaustivas e rigorosas. Adicionalmente, a orquestração das transações ACID e a gestão de lógicas temporais complexas (como a validade dos plantéis) revelou a verdadeira dimensão de desenhar sistemas de nível de produção.

5.2. Trabalho Futuro

Apesar da elevada maturidade e estabilidade da plataforma, identificam-se vias claras e fundamentadas para a evolução da arquitetura numa futura “Versão 2.0”:

1. **Evolução Arquitetural para Spring Modulith:** Embora o sistema atual siga os princípios lógicos de um Monólito Modular por convenção estrita de pacotes, a evolução natural passará pela migração para a *framework* **Spring Modulith**. Esta adoção permitirá a verificação arquitetural das fronteiras de domínio em tempo de compilação (evitando dependências cíclicas) e a gestão de eventos assíncronos. Esta refatoração deixará o *backend* estruturalmente preparado para uma eventual transição para arquitetura de **Microserviços** (extraíndo, por exemplo, o módulo Financeiro ou de Analytics para contentores isolados), mantendo os benefícios de *deployment* de um monólito no presente.
2. **Transição do Gateway de Pagamentos para Produção:** Graças à arquitetura de abstração adotada no módulo financeiro (SS6), o sistema encontra-se estruturalmente preparado para processar pagamentos reais. O passo imediato será a substituição do *Mock Gateway* interno por um fornecedor oficial de referências Multibanco (e.g., SIBS ou Ifthenpay). Dado que a arquitetura atual já simula o comportamento

real (incluindo estados transitórios e recepção assíncrona), esta transição exigirá apenas a configuração de credenciais nas variáveis de ambiente, mantendo intacta a máquina de estados e a infraestrutura de idempotência de *webhooks* já testada.

3. **Aplicações Móveis Nativas com Sincronização Offline:** A transposição da *interface* móvel responsiva para uma aplicação nativa (Flutter ou React Native) orientada aos Treinadores. Isto permitiria contornar falhas de conectividade comuns em recintos desportivos através do armazenamento local, realizando a sincronização assíncrona das presenças e avaliações de desempenho com a base de dados central assim que a rede fosse restabelecida.
4. **Recuperação de Palavra-passe por E-mail:** Atualmente, a redefinição de credenciais depende da intervenção de um Colaborador, que força manualmente a alteração através do endpoint administrativo. A evolução natural passará pela implementação de um fluxo de recuperação autónomo e seguro diretamente acessível ao utilizador final. Este mecanismo seguirá o padrão de token de uso único com expiração temporal: ao submeter o seu endereço de e-mail, o sistema gera um token criptograficamente seguro, persiste-o na base de dados com um prazo de validade (ex: 15 minutos) e envia ao utilizador uma hiperligação de redefinição através do servidor SMTP já configurado na infraestrutura. Dado que o módulo de notificações e a integração com serviço de e-mail já se encontram operacionais no sistema atual, esta funcionalidade exigirá apenas a adição de dois endpoints dedicados (POST /api/auth/forgot-password e POST /api/auth/reset-password), uma nova entidade de persistência para os tokens e uma página de redefinição no frontend React — mantendo intactas todas as políticas de segurança e a cadeia de autenticação JWT existente.

Em suma, o Codfish United dispõe agora de uma base tecnológica robusta, segura e escalável, ancorada nas melhores práticas de engenharia de *software* e perfeitamente preparada para absorver as necessidades operacionais reais do clube.

6. Referências Bibliográficas

<https://software-engineering-book.com/>

Arquitetura e Padrões de Software

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Spring Boot / Java

Documentação oficial Spring Boot — <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Documentação oficial Spring Security — <https://docs.spring.io/spring-security/reference/> Documentação

oficial Spring Data JPA — <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/> Documen-

tação oficial Spring Modulith — <https://docs.spring.io/spring-modulith/docs/current/reference/html/>

React / Frontend

Documentação oficial React — <https://react.dev/> Documentação oficial Tailwind CSS — [https://](https://tailwindcss.com/docs/)

tailwindcss.com/docs/

Engenharia de Requisitos

IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications. IEEE. IEEE Std 29148-2018. Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. IEEE.

Usabilidade

Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann. Nielsen Norman Group — <https://www.nngroup.com/articles/ten-usability-heuristics/>

Qualidade de Software

ISO/IEC 25010:2011. Systems and Software Quality Requirements and Evaluation (SQuaRE). ISO.

Segurança e RGPD

Regulamento (UE) 2016/679 do Parlamento Europeu e do Conselho (RGPD) — <https://eur-lex.europa.eu/legal-content/PT/TXT/?uri=CELEX:32016R0679>